<h1 id="RetailPulse:-Global-E-Commerce-Analytics-&-Business-Intelligence">RetailPulse: Global E-Commerce Analytics & Business Intelligence</h1>
<h2 id="Notebook-01---Data-Loading-&-Initial-Exploration">Notebook 01 - Data Loading & Initial Exploration</h2><h3 id="Objective">Objective</h3>

- Load all datasets
- Understand the structure
- Assess data quality
- Prepare for data cleaning

In [2]:
```
# ====================================
# Import Required Libraries
# ====================================
import pandas as pd
import numpy as np

#Display Settings
pd.set_option ('display.max_columns',None)
pd.set_option ('display.max_rows',20)

print("Libraries Imported Successfully")
```

```
Libraries Imported Successfully
```

In [5]:
```
## Section 2: Load All Datasets

customers = pd.read_csv("customers.csv")
orders = pd.read_csv("orders.csv")
order_items = pd.read_csv("order_items.csv")
products = pd.read_csv("products.csv")
payments = pd.read_csv("payment.csv")
reviews = pd.read_csv("reviews.csv")
shipments = pd.read_csv("shipments.csv")
suppliers = pd.read_csv("suppliers.csv")
print ("All dataset imported Successfully")
```

```
All dataset imported Successfully
```

In [7]:
```
## Section 3: Store Them in a Dictionary

datasets = {
    "Customers": customers,
    "Orders": orders,
    "Order Items": order_items,
    "Products": products,
    "Payments": payments,
    "Reviews": reviews,
    "Shipments": shipments,
    "Suppliers": suppliers
}
```

In [12]:
```
## Section 4: Check Dataset Dimensions

for name , df in datasets.items():
    print (f"{name}:,{df.shape}")
```

```
Customers:,(10000, 6)
Orders:,(15000, 4)
Order Items:,(20000, 5)
Products:,(2000, 5)
Payments:,(15000, 5)
Reviews:,(1106, 6)
Shipments:,(15000, 7)
Suppliers:,(100, 6)
```

In [13]:
```
for name, df in datasets.items():
    print("=" * 80)
    print(f"{name} Dataset")
    print("=" * 80)

    display(df.head())
```

```
================================================================================
Customers Dataset
================================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>customer_id</th> <th>first_name</th> <th>last_name</th> <th>address</th> <th>email</th> <th>phone_number</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody> </tbody>

1	James	Smith	123 Main St, Springfield, IL	jsmith1@customer.com	555-190-3233
2	James	Johnson	123 Main St, Springfield, IL	jjohnson2@customer.com	555-525-8357
3	James	Williams	123 Main St, Springfield, IL	jwilliams3@customer.com	555-933-4447
4	James	Brown	123 Main St, Springfield, IL	jbrown4@customer.com	555-897-0326
5	James	Jones	123 Main St, Springfield, IL	jjones5@customer.com	555-762-5836

```
================================================================================
Orders Dataset
================================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>order_id</th> <th>order_date</th> <th>customer_id</th> <th>total_price</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

7324	2023-11-27	2160	955.86
8938	2024-08-31	8497	984.91
4238	2023-12-05	6295	649.18
10944	2024-02-22	2936	54.83
11075	2024-08-14	2994	320.24

```
================================================================================
Order Items Dataset
================================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>order_item_id</th> <th>order_id</th> <th>product_id</th> <th>quantity</th> <th>price_at_purchase</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

1	6550	1032	1	342.92
2	7324	1695	1	955.86
3	11952	1962	1	909.45
4	8938	1958	1	984.91
5	4238	1880	1	649.18

```
================================================================================
Products Dataset
================================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>product_id</th> <th>product_name</th> <th>category</th> <th>price</th> <th>supplier_id</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

1	Office Chair	Furniture	445.01	501
2	Coffee Maker	Home & Kitchen	937.29	502
3	Document Scanner	Electronics	940.02	503
4	Desk Mat	Accessories	76.11	504
5	Tablet Stand	Accessories	388.17	505

```
================================================================================
Payments Dataset
================================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>payment_id</th> <th>order_id</th> <th>payment_method</th> <th>amount</th> <th>transaction_status</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

1	1	Credit Card	232.15	Completed
2	2	Credit Card	454.44	Completed
3	3	Credit Card	26.14	Completed
4	4	Credit Card	985.70	Completed
5	5	Credit Card	733.32	Completed

```
================================================================================
Reviews Dataset
================================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>review_id</th> <th>product_id</th> <th>customer_id</th> <th>rating</th> <th>review_text</th> <th>review_date</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody> </tbody>

1	16	1516	4	Customer service was excellent, product too.	2024-03-03
2	1516	1516	4	Customer service was excellent, product too.	2024-01-08
3	1566	9066	1	Basic functionality but reliable.	2024-03-23
4	66	9066	1	Basic functionality but reliable.	2024-03-25
5	1522	1522	3	Better than similar products I have tried.	2024-03-03

```
================================================================================
Shipments Dataset
================================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>shipment_id</th> <th>order_id</th> <th>shipment_date</th> <th>carrier</th> <th>tracking_number</th> <th>delivery_date</th> <th>shipment_status</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

1	1	2024-10-13	UPS	TRK344284	2024-10-14	Pending
2	2	2024-08-27	DHL	TRK718398	2024-08-28	Delivered
3	3	2024-05-23	FedEx	TRK161368	2024-05-31	Pending
4	4	2024-06-06	UPS	TRK890181	2024-06-10	Pending
5	5	2024-11-03	DHL	TRK681341	2024-11-08	Delivered

```
================================================================================
Suppliers Dataset
================================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>supplier_id</th> <th>supplier_name</th> <th>contact_name</th> <th>address</th> <th>phone_number</th> <th>email</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody> </tbody>

501	Dynamic Systems Group	Donald Benjamin	141 Shore Ln, Island City, MA	(555) 484-6922	dbenjamin@supplier.com
502	Dynamic Systems Group	Nicholas Dennis	161 Harbor Ln, Bay Point, FM	(555) 397-6986	ndennis@supplier.com
503	Mega Suppliers	Catherine Moore	151 Pearl St, Seaside, SD	(555) 240-4096	cmoore@supplier.com
504	Tech Supplies Inc.	Linda Virginia	505 Walnut St, Coast City, OR	(555) 624-2518	lvirginia@supplier.com
505	Ultimate Services	Mark Samuel	585 Lighthouse Rd, Sea Haven, LA	(555) 295-9426	msamuel@supplier.com

In [14]:
```
## Step 5.2: Display Column Names

for name, df in datasets.items():
    print("=" * 80)
    print(f"{name} Columns")
    print("=" * 80)

    print(df.columns.tolist())
    print()
```

```
================================================================================
Customers Columns
================================================================================
['customer_id', 'first_name', 'last_name', 'address', 'email', 'phone_number']

================================================================================
Orders Columns
================================================================================
['order_id', 'order_date', 'customer_id', 'total_price']

================================================================================
Order Items Columns
================================================================================
['order_item_id', 'order_id', 'product_id', 'quantity', 'price_at_purchase']

================================================================================
Products Columns
================================================================================
['product_id', 'product_name', 'category', 'price', 'supplier_id']

================================================================================
Payments Columns
================================================================================
['payment_id', 'order_id', 'payment_method', 'amount', 'transaction_status']

================================================================================
Reviews Columns
================================================================================
['review_id', 'product_id', 'customer_id', 'rating', 'review_text', 'review_dat
e']

================================================================================
Shipments Columns
================================================================================
['shipment_id', 'order_id', 'shipment_date', 'carrier', 'tracking_number', 'deliv
ery_date', 'shipment_status']

================================================================================
Suppliers Columns
================================================================================
['supplier_id', 'supplier_name', 'contact_name', 'address', 'phone_number', 'emai
l']
```

In [15]:
```python
## Step 5.3: Check Dataset Information
for name, df in datasets.items():
    print("=" * 80)
    print(f"{name} Information")
    print("=" * 80)

    df.info()

    print("\n")
```

```
================================================================================
Customers Information
================================================================================
<class 'pandas.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 6 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   customer_id   10000 non-null  int64
 1   first_name    10000 non-null  str
 2   last_name     10000 non-null  str
 3   address       10000 non-null  str
 4   email         10000 non-null  str
 5   phone_number  10000 non-null  str
dtypes: int64(1), str(5)
memory usage: 468.9 KB


================================================================================
Orders Information
================================================================================
<class 'pandas.DataFrame'>
RangeIndex: 15000 entries, 0 to 14999
Data columns (total 4 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   order_id     15000 non-null  int64
 1   order_date   15000 non-null  str
 2   customer_id  15000 non-null  int64
 3   total_price  15000 non-null  float64
dtypes: float64(1), int64(2), str(1)
memory usage: 468.9 KB


================================================================================
Order Items Information
================================================================================
<class 'pandas.DataFrame'>
RangeIndex: 20000 entries, 0 to 19999
Data columns (total 5 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   order_item_id      20000 non-null  int64
 1   order_id           20000 non-null  int64
 2   product_id         20000 non-null  int64
 3   quantity           20000 non-null  int64
 4   price_at_purchase  20000 non-null  float64
dtypes: float64(1), int64(4)
memory usage: 781.4 KB


================================================================================
Products Information
================================================================================
<class 'pandas.DataFrame'>
RangeIndex: 2000 entries, 0 to 1999
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   product_id    2000 non-null   int64
```

```
 1   product_name  2000 non-null   str
 2   category      2000 non-null   str
 3   price         2000 non-null   float64
 4   supplier_id   2000 non-null   int64
dtypes: float64(1), int64(2), str(2)
memory usage: 78.3 KB


================================================================================
Payments Information
================================================================================
<class 'pandas.DataFrame'>
RangeIndex: 15000 entries, 0 to 14999
Data columns (total 5 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   payment_id          15000 non-null  int64
 1   order_id            15000 non-null  int64
 2   payment_method      15000 non-null  str
 3   amount              15000 non-null  float64
 4   transaction_status  15000 non-null  str
dtypes: float64(1), int64(2), str(2)
memory usage: 586.1 KB


================================================================================
Reviews Information
================================================================================
<class 'pandas.DataFrame'>
RangeIndex: 1106 entries, 0 to 1105
Data columns (total 6 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   review_id    1106 non-null   int64
 1   product_id   1106 non-null   int64
 2   customer_id  1106 non-null   int64
 3   rating       1106 non-null   int64
 4   review_text  1106 non-null   str
 5   review_date  1106 non-null   str
dtypes: int64(4), str(2)
memory usage: 52.0 KB


================================================================================
Shipments Information
================================================================================
<class 'pandas.DataFrame'>
RangeIndex: 15000 entries, 0 to 14999
Data columns (total 7 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   shipment_id      15000 non-null  int64
 1   order_id         15000 non-null  int64
 2   shipment_date    15000 non-null  str
 3   carrier          15000 non-null  str
 4   tracking_number  15000 non-null  str
 5   delivery_date    15000 non-null  str
 6   shipment_status  15000 non-null  str
dtypes: int64(2), str(5)
memory usage: 820.4 KB
```

```
================================================================================
Suppliers Information
================================================================================
<class 'pandas.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   supplier_id    100 non-null    int64
 1   supplier_name  100 non-null    str
 2   contact_name   100 non-null    str
 3   address        100 non-null    str
 4   phone_number   100 non-null    str
 5   email          100 non-null    str
dtypes: int64(1), str(5)
memory usage: 4.8 KB
```

In [16]:
```
## Step 5.4: Create a Professional Dataset Summary
summary = pd.DataFrame({
    "Dataset": datasets.keys(),
    "Rows": [df.shape[0] for df in datasets.values()],
    "Columns": [df.shape[1] for df in datasets.values()],
    "Missing Values": [df.isnull().sum().sum() for df in datasets.values()],
    "Duplicate Rows": [df.duplicated().sum() for df in datasets.values()]
})

summary
```

Out[16]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Dataset</th> <th>Rows</th> <th>Columns</th> <th>Missing Values</th> <th>Duplicate Rows</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> </tbody> <tbody></tbody>

	Dataset	Rows	Columns	Missing Values	Duplicate Rows
0	Customers	10000	6	0	0
1	Orders	15000	4	0	0
2	Order Items	20000	5	0	0
3	Products	2000	5	0	0
4	Payments	15000	5	0	0
5	Reviews	1106	6	0	0
6	Shipments	15000	7	0	0
7	Suppliers	100	6	0	0

In [17]:
```
for name, df in datasets.items():
    print("=" * 70)
    print(name)
    print("=" * 70)
```

```
    print(df.dtypes)
    print()
```

```
==========================================================================
Customers
==========================================================================
customer_id     int64
first_name        str
last_name         str
address           str
email             str
phone_number      str
dtype: object

==========================================================================
Orders
==========================================================================
order_id         int64
order_date         str
customer_id      int64
total_price    float64
dtype: object

==========================================================================
Order Items
==========================================================================
order_item_id          int64
order_id               int64
product_id             int64
quantity               int64
price_at_purchase    float64
dtype: object

==========================================================================
Products
==========================================================================
product_id        int64
product_name        str
category            str
price           float64
supplier_id       int64
dtype: object

==========================================================================
Payments
==========================================================================
payment_id              int64
order_id                int64
payment_method            str
amount                float64
transaction_status        str
dtype: object

==========================================================================
Reviews
==========================================================================
review_id      int64
product_id     int64
customer_id    int64
rating         int64
review_text      str
review_date      str
dtype: object
```

```
========================================================================
Shipments
========================================================================
shipment_id        int64
order_id           int64
shipment_date        str
carrier              str
tracking_number      str
delivery_date        str
shipment_status      str
dtype: object

========================================================================
Suppliers
========================================================================
supplier_id      int64
supplier_name      str
contact_name       str
address            str
phone_number       str
email              str
dtype: object
```

In [19]:
```
## Convert Date Columns

orders["order_date"] = pd.to_datetime(orders["order_date"])

reviews["review_date"] = pd.to_datetime(reviews["review_date"])

shipments["shipment_date"] = pd.to_datetime(shipments["shipment_date"])

shipments["delivery_date"] = pd.to_datetime(shipments["delivery_date"])
```

In [20]:
```
## Verify the Conversion
print(orders.dtypes)
print()
print(reviews.dtypes)
print()
print(shipments.dtypes)
```

```
order_id                int64
order_date     datetime64[us]
customer_id             int64
total_price           float64
dtype: object

review_id               int64
product_id              int64
customer_id             int64
rating                  int64
review_text               str
review_date    datetime64[us]
dtype: object

shipment_id                int64
order_id                   int64
shipment_date      datetime64[us]
carrier                      str
tracking_number              str
delivery_date      datetime64[us]
shipment_status              str
dtype: object
```

In [21]:
```
## Validate Primary Keys
primary_keys = {
    "Customers": (customers, "customer_id"),
    "Orders": (orders, "order_id"),
    "Order Items": (order_items, "order_item_id"),
    "Products": (products, "product_id"),
    "Payments": (payments, "payment_id"),
    "Reviews": (reviews, "review_id"),
    "Shipments": (shipments, "shipment_id"),
    "Suppliers": (suppliers, "supplier_id")
}

for table, (df, key) in primary_keys.items():
    print(f"{table}")
    print(f"Unique {key}: {df[key].nunique()}")
    print(f"Total Rows: {len(df)}")
    print("-" * 40)
```

```
Customers
Unique customer_id: 10000
Total Rows: 10000
----------------------------------------
Orders
Unique order_id: 15000
Total Rows: 15000
----------------------------------------
Order Items
Unique order_item_id: 20000
Total Rows: 20000
----------------------------------------
Products
Unique product_id: 2000
Total Rows: 2000
----------------------------------------
Payments
Unique payment_id: 15000
Total Rows: 15000
----------------------------------------
Reviews
Unique review_id: 1106
Total Rows: 1106
----------------------------------------
Shipments
Unique shipment_id: 15000
Total Rows: 15000
----------------------------------------
Suppliers
Unique supplier_id: 100
Total Rows: 100
----------------------------------------
```

In [23]:

```python
# Orders → Customers
invalid_customers = orders[
    ~orders["customer_id"].isin(customers["customer_id"])
]

# Order Items → Orders
invalid_orders = order_items[
    ~order_items["order_id"].isin(orders["order_id"])
]

# Order Items → Products
invalid_products = order_items[
    ~order_items["product_id"].isin(products["product_id"])
]

# Products → Suppliers
invalid_suppliers = products[
    ~products["supplier_id"].isin(suppliers["supplier_id"])
]

# Payments → Orders
invalid_payment_orders = payments[
    ~payments["order_id"].isin(orders["order_id"])
]

# Shipments → Orders
invalid_shipment_orders = shipments[
```

```
    ~shipments["order_id"].isin(orders["order_id"])
]

# Reviews → Customers
invalid_review_customers = reviews[
    ~reviews["customer_id"].isin(customers["customer_id"])
]

# Reviews → Products
invalid_review_products = reviews[
    ~reviews["product_id"].isin(products["product_id"])
]
```

In [24]:

```
validation_results = pd.DataFrame({
    "Relationship": [
        "Orders → Customers",
        "Order Items → Orders",
        "Order Items → Products",
        "Products → Suppliers",
        "Payments → Orders",
        "Shipments → Orders",
        "Reviews → Customers",
        "Reviews → Products"
    ],
    "Invalid Records": [
        len(orders[~orders["customer_id"].isin(customers["customer_id"])]),
        len(order_items[~order_items["order_id"].isin(orders["order_id"])]),
        len(order_items[~order_items["product_id"].isin(products["product_id"])]
        len(products[~products["supplier_id"].isin(suppliers["supplier_id"])]),
        len(payments[~payments["order_id"].isin(orders["order_id"])]),
        len(shipments[~shipments["order_id"].isin(orders["order_id"])]),
        len(reviews[~reviews["customer_id"].isin(customers["customer_id"])]),
        len(reviews[~reviews["product_id"].isin(products["product_id"])])
    ]
})

validation_results["Status"] = validation_results["Invalid Records"].apply(
    lambda x: " Passed" if x == 0 else " Failed"
)

validation_results
```

Out[24]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Relationship</th> <th>Invalid Records</th> <th>Status</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> </tbody> <tbody></tbody>

Orders → Customers	0	Passed
Order Items → Orders	0	Passed
Order Items → Products	0	Passed
Products → Suppliers	0	Passed
Payments → Orders	0	Passed
Shipments → Orders	0	Passed
Reviews → Customers	0	Passed
Reviews → Products	0	Passed

In [26]:
```
## # Merge Orders with Customers
master_df = orders.merge(
    customers,
    on="customer_id",
    how="left"
)

# Merge Order Items
master_df = master_df.merge(
    order_items,
    on="order_id",
    how="left"
)

# Merge Products
master_df = master_df.merge(
    products,
    on="product_id",
    how="left"
)

# Merge Suppliers
master_df = master_df.merge(
    suppliers,
    on="supplier_id",
    how="left"
)

# Merge Payments
master_df = master_df.merge(
    payments,
    on="order_id",
    how="left"
)

# Merge Shipments
master_df = master_df.merge(
    shipments,
```

```
    on="order_id",
    how="left"
)

# Merge Reviews
master_df = master_df.merge(
    reviews,
    on=["customer_id", "product_id"],
    how="left"
)
```

In [27]:
```
## Check the Master Dataset
print(master_df.shape)
master_df.head()
```

```
(20000, 36)
```

Out[27]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>order_id</th> <th>order_date</th> <th>customer_id</th> <th>total_price</th> <th>first_name</th> <th>last_name</th> <th>address_x</th> <th>email_x</th> <th>phone_number_x</th> <th>order_item_id</th> <th>product_id</th> <th>quantity</th> <th>price_at_purchase</th> <th>product_name</th> <th>category</th> <th>price</th> <th>supplier_id</th> <th>supplier_name</th> <th>contact_name</th> <th>address_y</th> <th>phone_number_y</th> <th>email_y</th> <th>payment_id</th> <th>payment_method</th> <th>amount</th> <th>transaction_status</th> <th>shipment_id</th> <th>shipment_date</th> <th>carrier</th> <th>tracking_number</th> <th>delivery_date</th> <th>shipment_status</th> <th>review_id</th> <th>rating</th> <th>review_text</th> <th>review_date</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

7324	2023-11-27	2160	955.86	James	Abigail	242 Garden St, Liberty City, AL	jabigail2160@customer.com	555-888-8342																												
8938	2024-08-31	8497	984.91	John	Dennis	101 Maple Dr, Smallville, KS	jdennis8497@customer.com	555-353-5473																												
4238	2023-12-05	6295	649.18	Mary	Terry	242 Garden St, Liberty City, AL	mterry6295@customer.com	555-988-4082																												
10944	2024-02-22	2936	54.83	James	Jane	474 Marina Ave, Harbor View, VT	jjane2936@customer.com	555-822-5033																												
11075	2024-08-14	2994	320.24	James	Natalie	585 Lighthouse Rd, Sea Haven, LA	jnatalie2994@customer.com	555-164-9390																												

<h2 id="Inspect-the-Master-Dataset">Inspect the Master Dataset</h2>

In [28]:
```
print("Shape of Master Dataset:", master_df.shape)
```

```
Shape of Master Dataset: (20000, 36)
```

In [29]:
```
master_df.head()
```

Out[29]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>order_id</th> <th>order_date</th> <th>customer_id</th> <th>total_price</th> <th>first_name</th> <th>last_name</th> <th>address_x</th> <th>email_x</th> <th>phone_number_x</th> <th>order_item_id</th> <th>product_id</th> <th>quantity</th> <th>price_at_purchase</th> <th>product_name</th> <th>category</th> <th>price</th> <th>supplier_id</th> <th>supplier_name</th> <th>contact_name</th> <th>address_y</th> <th>phone_number_y</th> <th>email_y</th> <th>payment_id</th> <th>payment_method</th> <th>amount</th> <th>transaction_status</th> <th>shipment_id</th> <th>shipment_date</th> <th>carrier</th> <th>tracking_number</th> <th>delivery_date</th> <th>shipment_status</th> <th>review_id</th> <th>rating</th> <th>review_text</th> <th>review_date</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

7324	2023-11-27	2160	955.86	James	Abigail	242 Garden St, Liberty City, AL	jabigail2160@customer.com	555-888-8342																														
8938	2024-08-31	8497	984.91	John	Dennis	101 Maple Dr, Smallville, KS	jdennis8497@customer.com	555-353-5473																																
4238	2023-12-05	6295	649.18	Mary	Terry	242 Garden St, Liberty City, AL	mterry6295@customer.com	555-988-4082																																
10944	2024-02-22	2936	54.83	James	Jane	474 Marina Ave, Harbor View, VT	jjane2936@customer.com	555-822-5033																																
11075	2024-08-14	2994	320.24	James	Natalie	585 Lighthouse Rd, Sea Haven, LA	jnatalie2994@customer.com	555-164-9390																																

In [30]:
```
print(master_df.columns.tolist())
```

```
['order_id', 'order_date', 'customer_id', 'total_price', 'first_name', 'last_nam
e', 'address_x', 'email_x', 'phone_number_x', 'order_item_id', 'product_id', 'qua
ntity', 'price_at_purchase', 'product_name', 'category', 'price', 'supplier_id',
'supplier_name', 'contact_name', 'address_y', 'phone_number_y', 'email_y', 'payme
nt_id', 'payment_method', 'amount', 'transaction_status', 'shipment_id', 'shipmen
t_date', 'carrier', 'tracking_number', 'delivery_date', 'shipment_status', 'revie
w_id', 'rating', 'review_text', 'review_date']
```

In [31]:
```
master_df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 20000 entries, 0 to 19999
Data columns (total 36 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   order_id            20000 non-null  int64
 1   order_date          20000 non-null  datetime64[us]
 2   customer_id         20000 non-null  int64
 3   total_price         20000 non-null  float64
 4   first_name          20000 non-null  str
 5   last_name           20000 non-null  str
 6   address_x           20000 non-null  str
 7   email_x             20000 non-null  str
 8   phone_number_x      20000 non-null  str
 9   order_item_id       20000 non-null  int64
 10  product_id          20000 non-null  int64
 11  quantity            20000 non-null  int64
 12  price_at_purchase   20000 non-null  float64
 13  product_name        20000 non-null  str
 14  category            20000 non-null  str
 15  price               20000 non-null  float64
 16  supplier_id         20000 non-null  int64
 17  supplier_name       20000 non-null  str
 18  contact_name        20000 non-null  str
 19  address_y           20000 non-null  str
 20  phone_number_y      20000 non-null  str
 21  email_y             20000 non-null  str
 22  payment_id          20000 non-null  int64
 23  payment_method      20000 non-null  str
 24  amount              20000 non-null  float64
 25  transaction_status  20000 non-null  str
 26  shipment_id         20000 non-null  int64
 27  shipment_date       20000 non-null  datetime64[us]
 28  carrier             20000 non-null  str
 29  tracking_number     20000 non-null  str
 30  delivery_date       20000 non-null  datetime64[us]
 31  shipment_status     20000 non-null  str
 32  review_id           4 non-null      float64
 33  rating              4 non-null      float64
 34  review_text         4 non-null      str
 35  review_date         4 non-null      datetime64[us]
dtypes: datetime64[us](4), float64(6), int64(8), str(18)
memory usage: 5.5 MB
```

In [32]:
```
master_df.isnull().sum().sort_values(ascending=False)
```

Out[32]:
```
review_date           19996
review_text           19996
rating                19996
review_id             19996
transaction_status        0
                        ...
price_at_purchase         0
product_name              0
category                  0
price                     0
contact_name              0
Length: 36, dtype: int64
```

<h2 id="Create-Business-Metrics">Create Business Metrics</h2>

In [33]:
```
total_revenue = master_df["total_price"].sum()

print(f"Total Revenue: ${total_revenue:,.2f}")
```

```
Total Revenue: $56,786,164.07
```

In [34]:
```
total_orders = master_df["order_id"].nunique()

print(f"Total Orders: {total_orders:,}")
```

```
Total Orders: 15,000
```

In [35]:
```
total_customers = master_df["customer_id"].nunique()

print(f"Total Customers: {total_customers:,}")
```

```
Total Customers: 10,000
```

In [36]:
```
total_products = master_df["product_id"].nunique()

print(f"Total Products: {total_products:,}")
```

```
Total Products: 1,997
```

In [37]:
```
average_order_value = orders["total_price"].mean()

print(f"Average Order Value: ${average_order_value:,.2f}")
```

```
Average Order Value: $2,833.05
```

In [39]:
```
# ========================================
# KPI CALCULATIONS (Correct Version)
# ========================================

# Total Revenue (Use Orders table)
total_revenue = orders["total_price"].sum()

# Total Orders
total_orders = orders["order_id"].nunique()

# Total Customers
total_customers = customers["customer_id"].nunique()

# Total Products
total_products = products["product_id"].nunique()

# Average Order Value
```

```
average_order_value = orders["total_price"].mean()

# Create KPI Summary Table
kpi_summary = pd.DataFrame({
    "KPI": [
        "Total Revenue",
        "Total Orders",
        "Total Customers",
        "Total Products",
        "Average Order Value"
    ],
    "Value": [
        f"${total_revenue:,.2f}",
        total_orders,
        total_customers,
        total_products,
        f"${average_order_value:,.2f}"
    ]
})

kpi_summary
```

Out[39]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>KPI</th> <th>Value</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

Total Revenue	$42,495,704.81
Total Orders	15000
Total Customers	10000
Total Products	2000
Average Order Value	$2,833.05

<h2 id="Section-1:-Sales-Trend-Analysis">Section 1: Sales Trend Analysis</h2><h2 id="Step-1:-Create-Monthly-Sales-Summary">Step 1: Create Monthly Sales Summary</h2>

In [40]:

```
# Create Year, Month and Month Number
orders["Year"] = orders["order_date"].dt.year
orders["Month"] = orders["order_date"].dt.month_name()
orders["Month_Number"] = orders["order_date"].dt.month

# Monthly Sales Summary
monthly_summary = (
    orders
    .groupby(["Year", "Month_Number", "Month"], as_index=False)
    .agg(
        Total_Revenue=("total_price", "sum"),
        Total_Orders=("order_id", "count"),
        Average_Order_Value=("total_price", "mean")
    )
    .sort_values(["Year", "Month_Number"])
)

monthly_summary
```

Out[40]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Year</th> <th>Month_Number</th> <th>Month</th> <th>Total_Revenue</th> <th>Total_Orders</th> <th>Average_Order_Value</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> </tbody> <tbody></tbody>

2023	11	November	2763201.95	988	2796.763107
2023	12	December	3360869.91	1222	2750.302709
2024	1	January	3667447.62	1284	2856.267617
2024	2	February	3300225.42	1173	2813.491407
2024	3	March	3756172.48	1306	2876.089188
2024	4	April	3493861.41	1213	2880.347411
2024	5	May	3590164.64	1267	2833.594822
2024	6	June	3355548.13	1190	2819.788345
2024	7	July	3760470.93	1320	2848.841614
2024	8	August	3737818.32	1308	2857.659266
2024	9	September	3526269.73	1242	2839.186578
2024	10	October	3772435.39	1355	2784.085159
2024	11	November	411218.88	132	3115.294545

<h2 id="Step-2:-Revenue-Trend-Chart">Step 2: Revenue Trend Chart</h2>

In [41]:
```
import matplotlib.pyplot as plt

plt.figure(figsize=(12,6))

plt.plot(
    monthly_summary["Month"],
    monthly_summary["Total_Revenue"],
    marker="o",
    linewidth=2
)

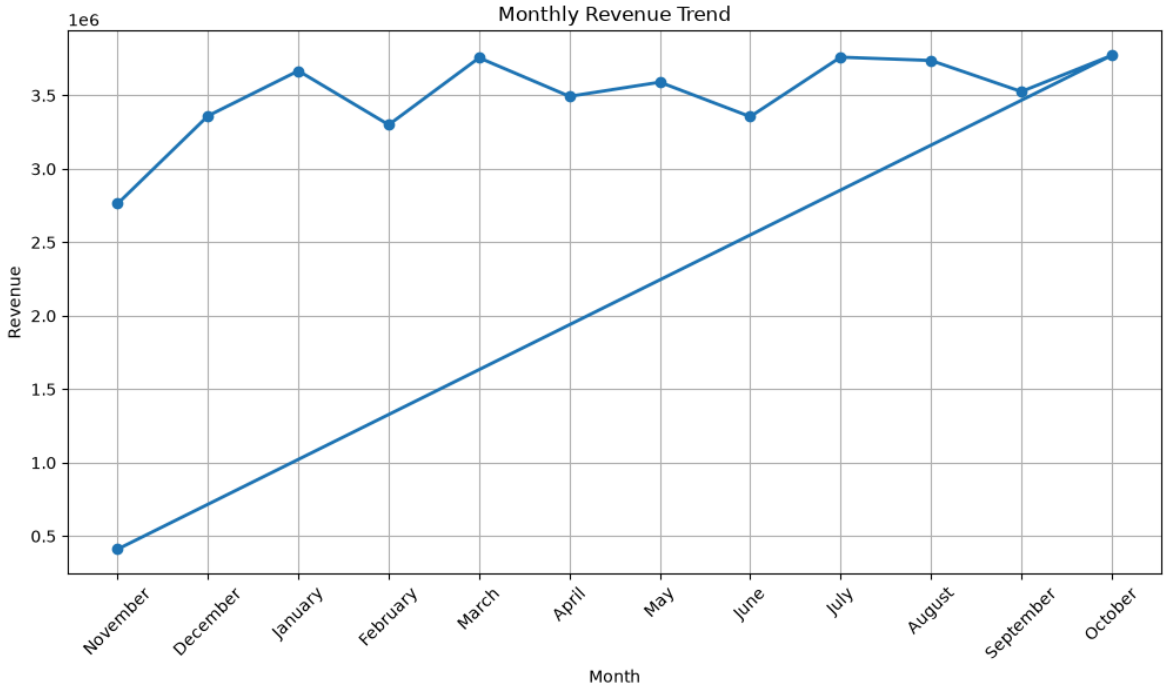
plt.title("Monthly Revenue Trend")
plt.xlabel("Month")
plt.ylabel("Revenue")

plt.xticks(rotation=45)

plt.grid(True)

plt.show()
```

```
Matplotlib is building the font cache; this may take a moment.
```



<h2 id="Step-3:-Monthly-Orders">Step 3: Monthly Orders</h2>

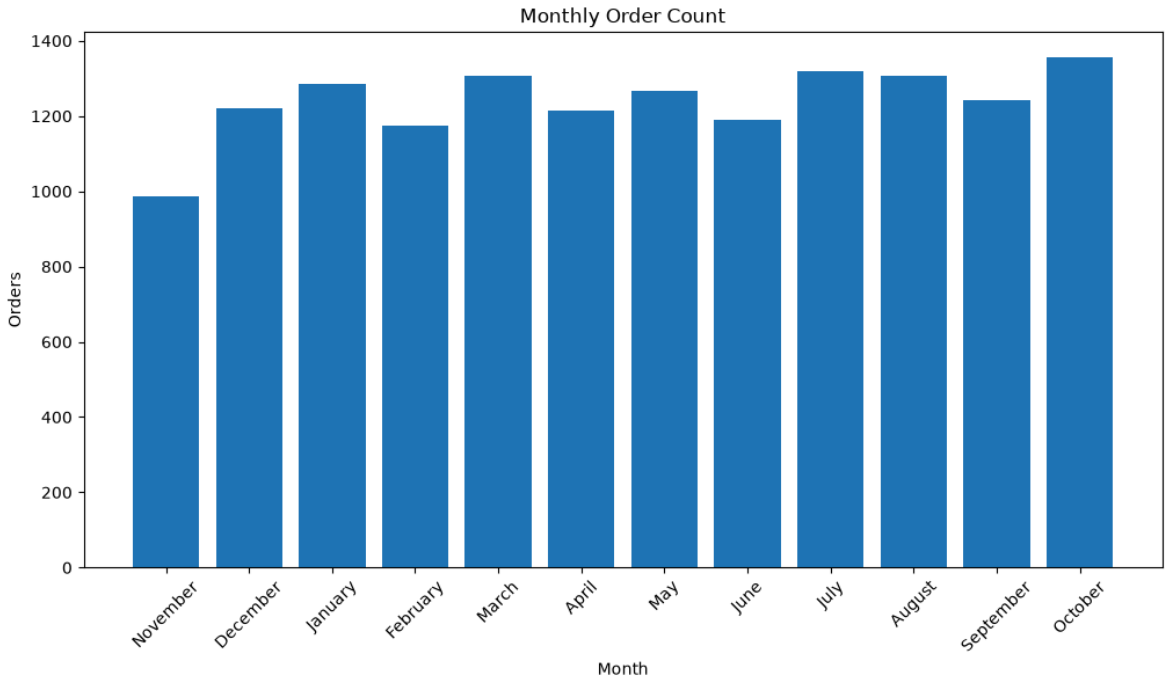
In [42]:
```
plt.figure(figsize=(12,6))

plt.bar(
    monthly_summary["Month"],
    monthly_summary["Total_Orders"]
)

plt.title("Monthly Order Count")
plt.xlabel("Month")
plt.ylabel("Orders")

plt.xticks(rotation=45)

plt.show()
```



<h2 id="Step-4:-Monthly-Average-Order-Value">Step 4: Monthly Average Order Value</h2>

In [43]:
```
plt.figure(figsize=(12,6))

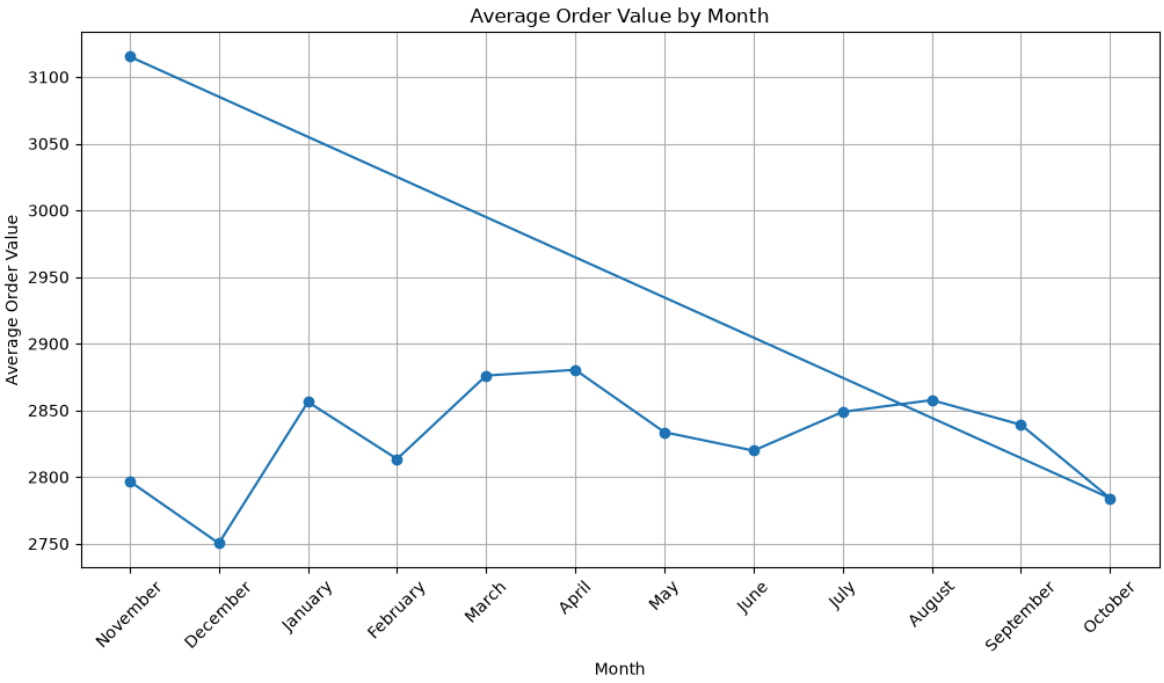
plt.plot(
    monthly_summary["Month"],
    monthly_summary["Average_Order_Value"],
    marker="o"
)

plt.title("Average Order Value by Month")
plt.xlabel("Month")
plt.ylabel("Average Order Value")

plt.xticks(rotation=45)

plt.grid(True)

plt.show()
```



<h2 id="Business-Insights">Business Insights</h2>

- Revenue remained relatively stable throughout 2024, ranging from approximately $3.3Mto$3.8M per month.
- October 2024 recorded the highest revenue and the highest number of orders.
- November 2024 shows significantly lower revenue and order volume, indicating that the dataset likely contains only partial data for that month.
- The Average Order Value (AOV) remained consistent around $2,800–2,900, suggesting stable customer purchasing behavior.

In [44]:
```
# Create a Year-Month label
monthly_summary["Year_Month"] = (
    monthly_summary["Year"].astype(str)
    + "-"
```

```
    + monthly_summary["Month_Number"].astype(str).str.zfill(2)
)
```

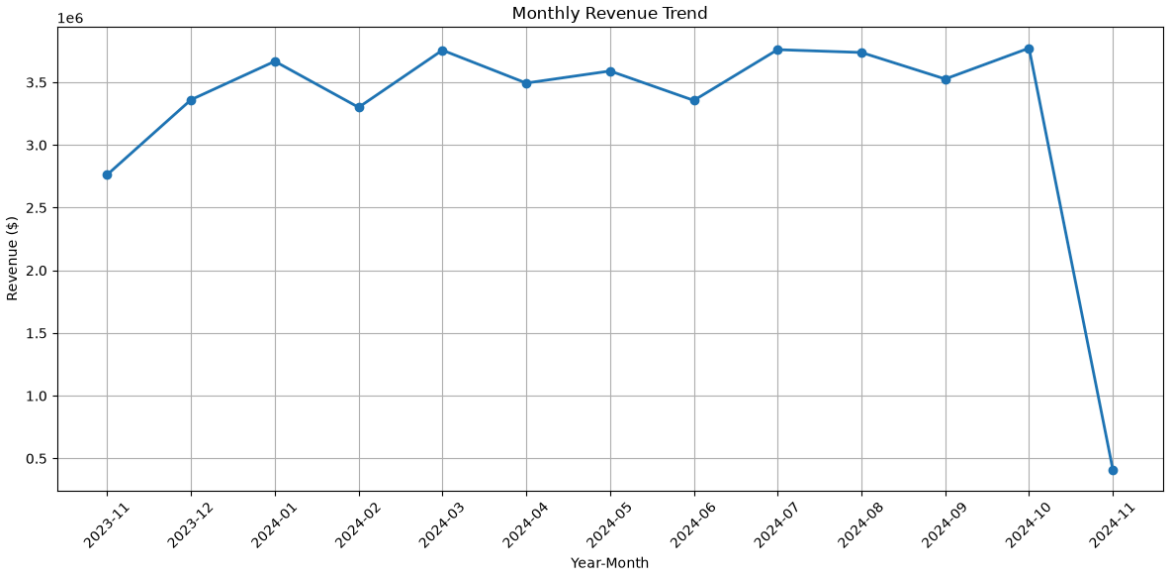
In [45]:
```
import matplotlib.pyplot as plt

plt.figure(figsize=(12,6))

plt.plot(
    monthly_summary["Year_Month"],
    monthly_summary["Total_Revenue"],
    marker="o",
    linewidth=2
)

plt.title("Monthly Revenue Trend")
plt.xlabel("Year-Month")
plt.ylabel("Revenue ($)")
plt.xticks(rotation=45)
plt.grid(True)

plt.tight_layout()
plt.show()
```



<h1 id="Product-Analysis">Product Analysis</h1><h2 id="Business-Questions">Business Questions</h2>

1. Which products generate the highest revenue?
2. Which products sell the highest number of units?
3. Which product categories contribute the highest revenue?
4. Which product categories sell the most units?

These insights help businesses identify their best-performing products and categories, enabling better inventory management and marketing strategies.

In [46]:
```
# ========================================
# Product Sales Dataset
# ========================================

product_sales = (
    order_items
```

```
    .merge(products, on="product_id", how="left")
)

# Revenue generated by each order item
product_sales["Revenue"] = (
    product_sales["quantity"] *
    product_sales["price_at_purchase"]
)

product_sales.head()
```

Out[46]:

	order_item_id	order_id	product_id	quantity	price_at_purchase	product_name	category	price	supplier_id	Revenue
0	1	6550	1032	1	342.92	Storage Shelf	Furniture	680.43	532	342.92
1	2	7324	1695	1	955.86	Desk Lamp	Home & Kitchen	940.82	595	955.86
2	3	11952	1962	1	909.45	Rice Cooker	Home & Kitchen	542.94	562	909.45
3	4	8938	1958	1	984.91	External SSD	Electronics	180.06	558	984.91
4	5	4238	1880	1	649.18	Kitchen Blender	Home & Kitchen	856.78	580	649.18

In [47]:

```
## Top 10 Products by Revenue
top_products_revenue = (
    product_sales
    .groupby("product_name", as_index=False)
    .agg(
        Total_Revenue=("Revenue", "sum"),
        Units_Sold=("quantity", "sum")
    )
    .sort_values("Total_Revenue", ascending=False)
    .head(10)
)

top_products_revenue
```

Out[47]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>product_name</th> <th>Total_Revenue</th> <th>Units_Sold</th> </thead> <tbody> <th>0</th> <th>24</th> <th>35</th> <th>20</th> <th>1</th> <th>27</th> <th>16</th> <th>43</th> <th>3</th> <th>38</th> </tbody> <tbody> </tbody>

4K Monitor	1028666.50	2280
Microphone	981300.29	2001
Standing Desk	974797.23	2030
Kitchen Blender	932734.20	2024
Air Purifier	924883.08	2031
Office Chair	908531.71	1944
Food Processor	903902.22	1965
Water Bottle	903662.53	1948
Bluetooth Headphones	894944.67	2021
Tablet Stand	894409.80	1936

In [48]:
```
## Visualization
import matplotlib.pyplot as plt

plt.figure(figsize=(12,6))

plt.bar(
    top_products_revenue["product_name"],
    top_products_revenue["Total_Revenue"]
)

plt.title("Top 10 Products by Revenue")

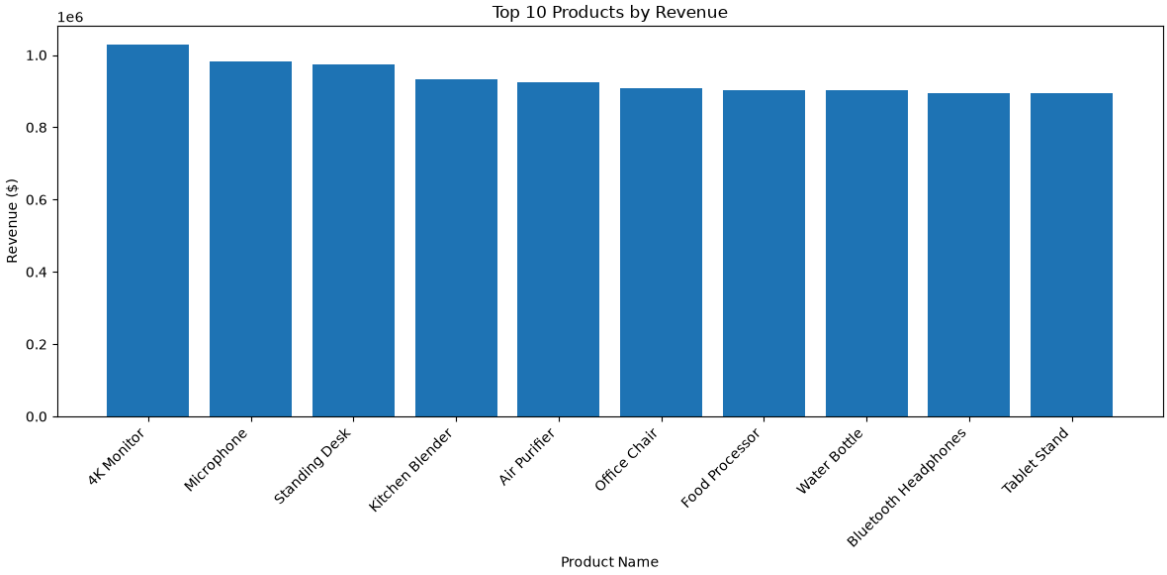
plt.xlabel("Product Name")

plt.ylabel("Revenue ($)")

plt.xticks(rotation=45, ha="right")

plt.tight_layout()

plt.show()
```



<h3 id="Business-Insight">Business Insight</h3>

- These products contribute the highest share of revenue.
- They should receive priority for inventory planning.
- Marketing campaigns should focus on these products.
- These products can be bundled with lower-selling products to increase overall sales.

In [49]:
```
## Top 10 Products by Units Sold
top_products_quantity = (
    product_sales
    .groupby("product_name", as_index=False)
    .agg(
        Units_Sold=("quantity", "sum")
    )
    .sort_values("Units_Sold", ascending=False)
    .head(10)
)

top_products_quantity
```

Out[49]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>product_name</th> <th>Units_Sold</th> </thead> <tbody> <th>0</th> <th>39</th> <th>1</th> <th>35</th> <th>20</th> <th>3</th> <th>37</th> <th>14</th> <th>24</th> <th>25</th> </tbody> <tbody></tbody>

4K Monitor	2280
Throw Pillows	2072
Air Purifier	2031
Standing Desk	2030
Kitchen Blender	2024
Bluetooth Headphones	2021
Storage Shelf	2007
External SSD	2003
Microphone	2001
Monitor Stand	1972

In [50]:
```
#Visualization
plt.figure(figsize=(12,6))

plt.bar(
    top_products_quantity["product_name"],
    top_products_quantity["Units_Sold"]
)

plt.title("Top 10 Products by Units Sold")

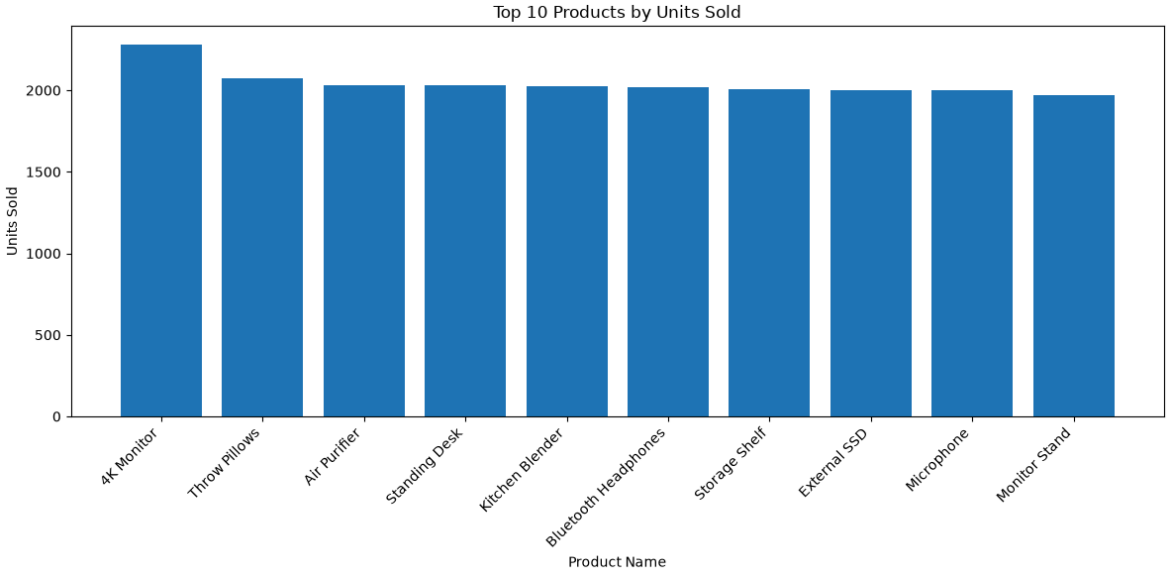
plt.xlabel("Product Name")

plt.ylabel("Units Sold")

plt.xticks(rotation=45, ha="right")

plt.tight_layout()

plt.show()
```



<h3 id="Business-Insight">Business Insight</h3> High unit sales do not always correspond to the highest revenue. Some products generate high revenue because of premium pricing, while others sell in larger quantities at lower prices.

In [51]:
```
## Category-wise Revenue
category_revenue = (
    product_sales
    .groupby("category", as_index=False)
    .agg(
        Revenue=("Revenue", "sum"),
        Units_Sold=("quantity", "sum")
    )
    .sort_values("Revenue", ascending=False)
)

category_revenue
```

Out[51]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>category</th> <th>Revenue</th> <th>Units_Sold</th> </thead> <tbody> <th>1</th> <th>3</th> <th>0</th> <th>2</th> </tbody> <tbody> </tbody>

	Electronics	15247167.14	33704
	Home & Kitchen	11182785.90	24799
	Accessories	9833968.69	21763
	Furniture	6231783.08	13609

In [53]:
```
# ============================================================
# PRODUCT ANALYSIS
# ============================================================
#
# Business Questions
# 1. Which products generate the highest revenue?
# 2. Which products sell the highest number of units?
# 3. Which product categories contribute the highest revenue?
# 4. Which product categories sell the highest number of units?
```

```
# 5. What percentage of total revenue does each category contribute?
#
# Business Objective
# - Identify high-performing products
# - Optimize inventory
# - Improve marketing strategies
# - Analyze category performance
# ============================================================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Chart Settings
plt.rcParams["figure.figsize"] = (12,6)

# ============================================================
# STEP 1 : CREATE PRODUCT SALES DATASET
# ============================================================

product_sales = order_items.merge(
    products,
    on="product_id",
    how="left"
)

# Calculate Revenue
product_sales["Revenue"] = (
    product_sales["quantity"] *
    product_sales["price_at_purchase"]
)

print("="*70)
print("PRODUCT SALES DATASET")
print("="*70)

print("Shape :", product_sales.shape)

display(product_sales.head())

# ============================================================
# STEP 2 : DATASET OVERVIEW
# ============================================================

print("\nDataset Information\n")

product_sales.info()

print("\nSummary Statistics\n")

display(product_sales.describe())

# ============================================================
# BUSINESS QUESTION 1
# TOP 10 PRODUCTS BY REVENUE
# ============================================================

top_products_revenue = (
    product_sales
    .groupby("product_name", as_index=False)
```

```
    .agg(
        Total_Revenue=("Revenue","sum"),
        Units_Sold=("quantity","sum")
    )
    .sort_values(
        "Total_Revenue",
        ascending=False
    )
    .head(10)
)

print("="*70)
print("TOP 10 PRODUCTS BY REVENUE")
print("="*70)

display(top_products_revenue)

# Visualization

plt.figure(figsize=(14,6))

plt.bar(
    top_products_revenue["product_name"],
    top_products_revenue["Total_Revenue"]
)

plt.title("Top 10 Products by Revenue", fontsize=16)

plt.xlabel("Product Name")

plt.ylabel("Revenue ($)")

plt.xticks(rotation=45, ha="right")

plt.tight_layout()

plt.show()

print("""
Business Insight

• These products generate the highest revenue.

• They should receive priority inventory allocation.

• They should remain available throughout the year.

• Marketing campaigns should emphasize these products.
""")

# ============================================================
# BUSINESS QUESTION 2
# TOP 10 PRODUCTS BY UNITS SOLD
# ============================================================

top_products_quantity = (
    product_sales
    .groupby("product_name", as_index=False)
    .agg(
        Units_Sold=("quantity","sum"),
```

```
        Revenue=("Revenue","sum")
    )
    .sort_values(
        "Units_Sold",
        ascending=False
    )
    .head(10)
)

print("="*70)
print("TOP 10 PRODUCTS BY UNITS SOLD")
print("="*70)

display(top_products_quantity)

plt.figure(figsize=(14,6))

plt.bar(
    top_products_quantity["product_name"],
    top_products_quantity["Units_Sold"]
)

plt.title("Top 10 Products by Units Sold", fontsize=16)

plt.xlabel("Product Name")

plt.ylabel("Units Sold")

plt.xticks(rotation=45, ha="right")

plt.tight_layout()

plt.show()

print("""
Business Insight

• High-selling products are not always the highest revenue generators.

• Premium products may sell fewer units but generate higher revenue.
""")

# ============================================================
# BUSINESS QUESTION 3
# CATEGORY-WISE REVENUE
# ============================================================

category_revenue = (
    product_sales
    .groupby("category", as_index=False)
    .agg(
        Total_Revenue=("Revenue","sum"),
        Units_Sold=("quantity","sum")
    )
    .sort_values(
        "Total_Revenue",
        ascending=False
    )
)
```

```
print("="*70)
print("CATEGORY REVENUE")
print("="*70)

display(category_revenue)

plt.figure(figsize=(10,6))

plt.bar(
    category_revenue["category"],
    category_revenue["Total_Revenue"]
)

plt.title("Revenue by Product Category", fontsize=16)

plt.xlabel("Category")

plt.ylabel("Revenue ($)")

plt.xticks(rotation=45)

plt.tight_layout()

plt.show()

# ============================================================
# BUSINESS QUESTION 4
# CATEGORY-WISE UNITS SOLD
# ============================================================

category_quantity = (
    product_sales
    .groupby("category", as_index=False)
    .agg(
        Units_Sold=("quantity","sum")
    )
    .sort_values(
        "Units_Sold",
        ascending=False
    )
)

print("="*70)
print("CATEGORY UNITS SOLD")
print("="*70)

display(category_quantity)

plt.figure(figsize=(10,6))

plt.bar(
    category_quantity["category"],
    category_quantity["Units_Sold"]
)

plt.title("Units Sold by Category", fontsize=16)

plt.xlabel("Category")

plt.ylabel("Units Sold")
```

```
plt.xticks(rotation=45)

plt.tight_layout()

plt.show()

# ============================================================
# BUSINESS QUESTION 5
# REVENUE CONTRIBUTION BY CATEGORY
# ============================================================

category_revenue["Revenue_Percentage"] = (
    category_revenue["Total_Revenue"] /
    category_revenue["Total_Revenue"].sum() * 100
).round(2)

print("="*70)
print("CATEGORY REVENUE CONTRIBUTION")
print("="*70)

display(category_revenue)

plt.figure(figsize=(8,8))

plt.pie(
    category_revenue["Revenue_Percentage"],
    labels=category_revenue["category"],
    autopct="%1.1f%%",
    startangle=90
)

plt.title("Revenue Contribution by Category")

plt.show()

print("""
============================================================
PRODUCT ANALYSIS SUMMARY
============================================================

Key Findings

✓ Identified the highest revenue-generating products.

✓ Identified the highest-selling products.

✓ Compared revenue versus quantity sold.

✓ Evaluated category-wise performance.

✓ Calculated revenue contribution by category.

Business Recommendations

1. Increase inventory for top-selling products.

2. Allocate more marketing budget to high-revenue categories.

3. Improve visibility of low-performing categories.
```

```
4. Bundle premium products with fast-moving products.

5. Monitor category performance regularly.
""")
```

```
======================================================================
PRODUCT SALES DATASET
======================================================================
Shape : (20000, 10)
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>order_item_id</th> <th>order_id</th> <th>product_id</th> <th>quantity</th> <th>price_at_purchase</th> <th>product_name</th> <th>category</th> <th>price</th> <th>supplier_id</th> <th>Revenue</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

1	6550	1032	1	342.92	Storage Shelf	Furniture	680.43	532	342.92
2	7324	1695	1	955.86	Desk Lamp	Home & Kitchen	940.82	595	955.86
3	11952	1962	1	909.45	Rice Cooker	Home & Kitchen	542.94	562	909.45
4	8938	1958	1	984.91	External SSD	Electronics	180.06	558	984.91
5	4238	1880	1	649.18	Kitchen Blender	Home & Kitchen	856.78	580	649.18

```
Dataset Information

<class 'pandas.DataFrame'>
RangeIndex: 20000 entries, 0 to 19999
Data columns (total 10 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   order_item_id      20000 non-null  int64
 1   order_id           20000 non-null  int64
 2   product_id         20000 non-null  int64
 3   quantity           20000 non-null  int64
 4   price_at_purchase  20000 non-null  float64
 5   product_name       20000 non-null  str
 6   category           20000 non-null  str
 7   price              20000 non-null  float64
 8   supplier_id        20000 non-null  int64
 9   Revenue            20000 non-null  float64
dtypes: float64(3), int64(5), str(2)
memory usage: 1.5 MB

Summary Statistics
```

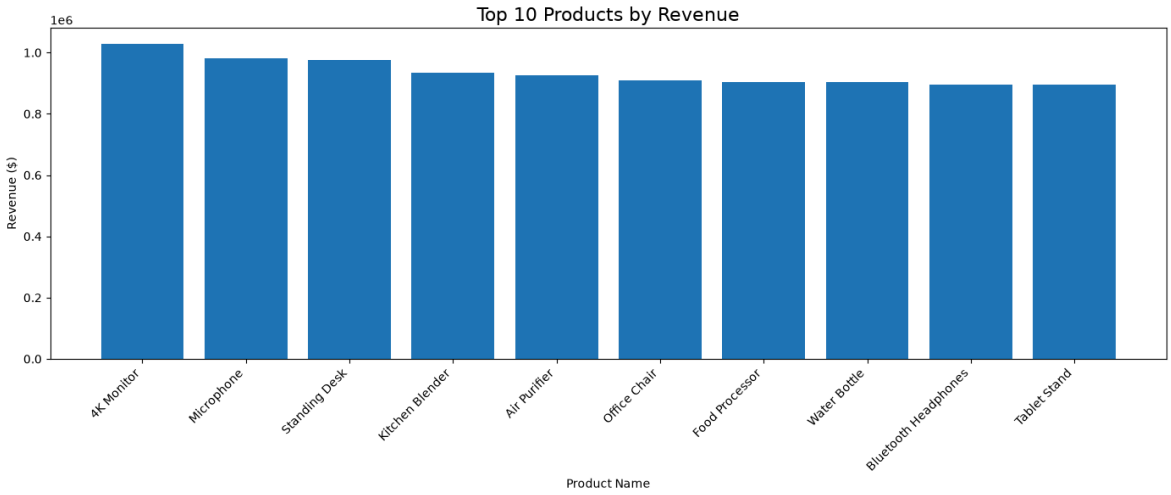
<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>order_item_id</th> <th>order_id</th> <th>product_id</th> <th>quantity</th> <th>price_at_purchase</th> <th>price</th> <th>supplier_id</th> <th>Revenue</th> </thead> <tbody> <th>count</th> <th>mean</th> <th>std</th> <th>min</th> <th>25%</th> <th>50%</th> <th>75%</th> <th>max</th> </tbody> <tbody></tbody>

20000.000000	20000.000000	20000.000000	20000.000000	20000.000000	20000.000000	2000
10000.500000	7500.044900	1000.142750	4.693750	387.014215	503.130107	55
5773.647028	4327.062058	573.545342	2.920648	324.767174	288.664443	2
1.000000	1.000000	1.000000	1.000000	0.000000	0.160000	50
5000.750000	3760.750000	508.000000	2.000000	32.110000	247.517500	52
10000.500000	7486.500000	996.500000	4.000000	347.195000	508.820000	55
15000.250000	11250.250000	1490.000000	7.000000	675.142500	750.340000	57
20000.000000	15000.000000	2000.000000	10.000000	999.820000	999.920000	60

```
======================================================================
TOP 10 PRODUCTS BY REVENUE
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>product_name</th> <th>Total_Revenue</th> <th>Units_Sold</th> </thead> <tbody> <th>0</th> <th>24</th> <th>35</th> <th>20</th> <th>1</th> <th>27</th> <th>16</th> <th>43</th> <th>3</th> <th>38</th> </tbody> <tbody> </tbody>

4K Monitor	1028666.50	2280
Microphone	981300.29	2001
Standing Desk	974797.23	2030
Kitchen Blender	932734.20	2024
Air Purifier	924883.08	2031
Office Chair	908531.71	1944
Food Processor	903902.22	1965
Water Bottle	903662.53	1948
Bluetooth Headphones	894944.67	2021
Tablet Stand	894409.80	1936



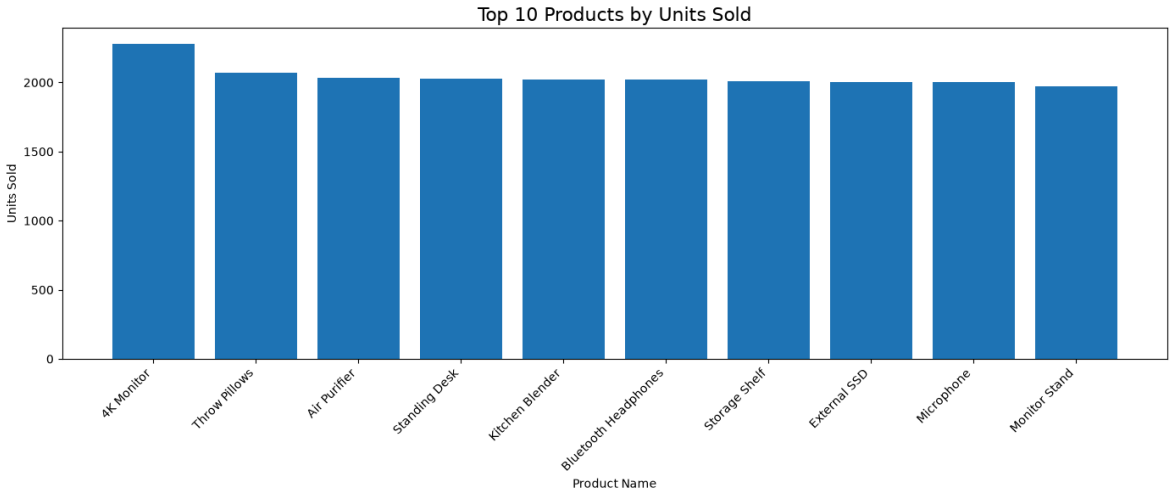
Business Insight

- These products generate the highest revenue.
- They should receive priority inventory allocation.
- They should remain available throughout the year.
- Marketing campaigns should emphasize these products.

```
======================================================================
TOP 10 PRODUCTS BY UNITS SOLD
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>product_name</th> <th>Units_Sold</th> <th>Revenue</th> </thead> <tbody> <th>0</th> <th>39</th> <th>1</th> <th>35</th> <th>20</th> <th>3</th> <th>37</th> <th>14</th> <th>24</th> <th>25</th> </tbody> <tbody></tbody>

4K Monitor	2280	1028666.50
Throw Pillows	2072	879662.03
Air Purifier	2031	924883.08
Standing Desk	2030	974797.23
Kitchen Blender	2024	932734.20
Bluetooth Headphones	2021	894944.67
Storage Shelf	2007	868637.35
External SSD	2003	883169.76
Microphone	2001	981300.29
Monitor Stand	1972	891260.93




```
Business Insight

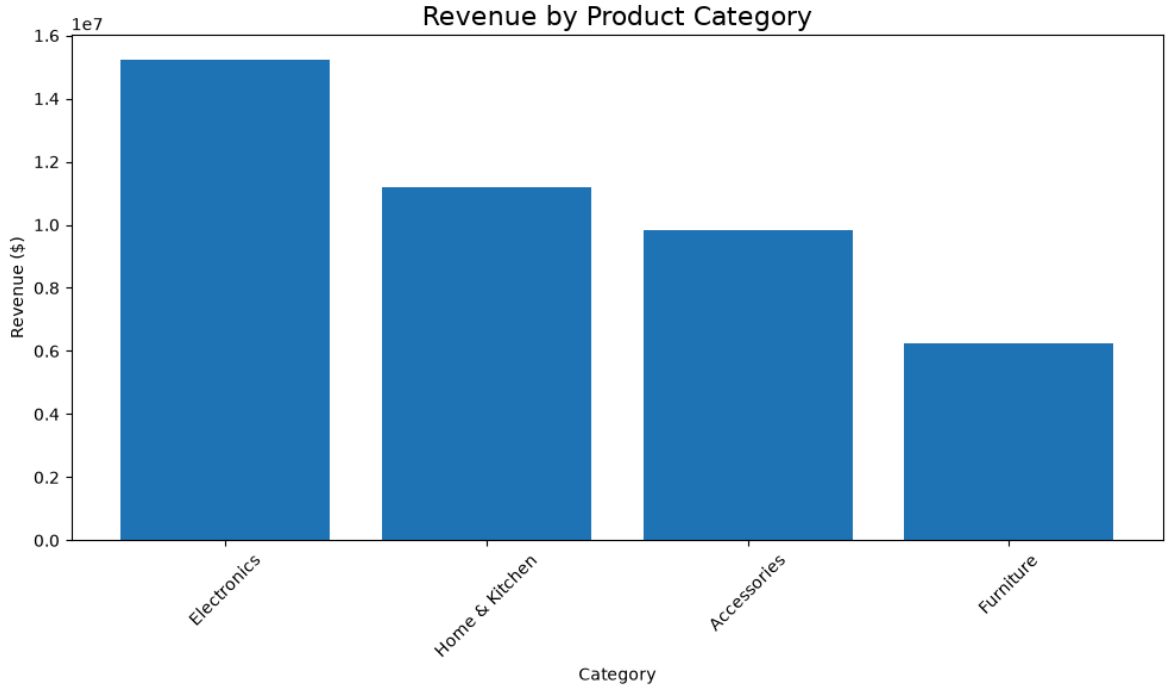
• High-selling products are not always the highest revenue generators.

• Premium products may sell fewer units but generate higher revenue.

======================================================================
CATEGORY REVENUE
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>category</th> <th>Total_Revenue</th> <th>Units_Sold</th> </thead> <tbody> <th>1</th> <th>3</th> <th>0</th> <th>2</th> </tbody> <tbody></tbody>

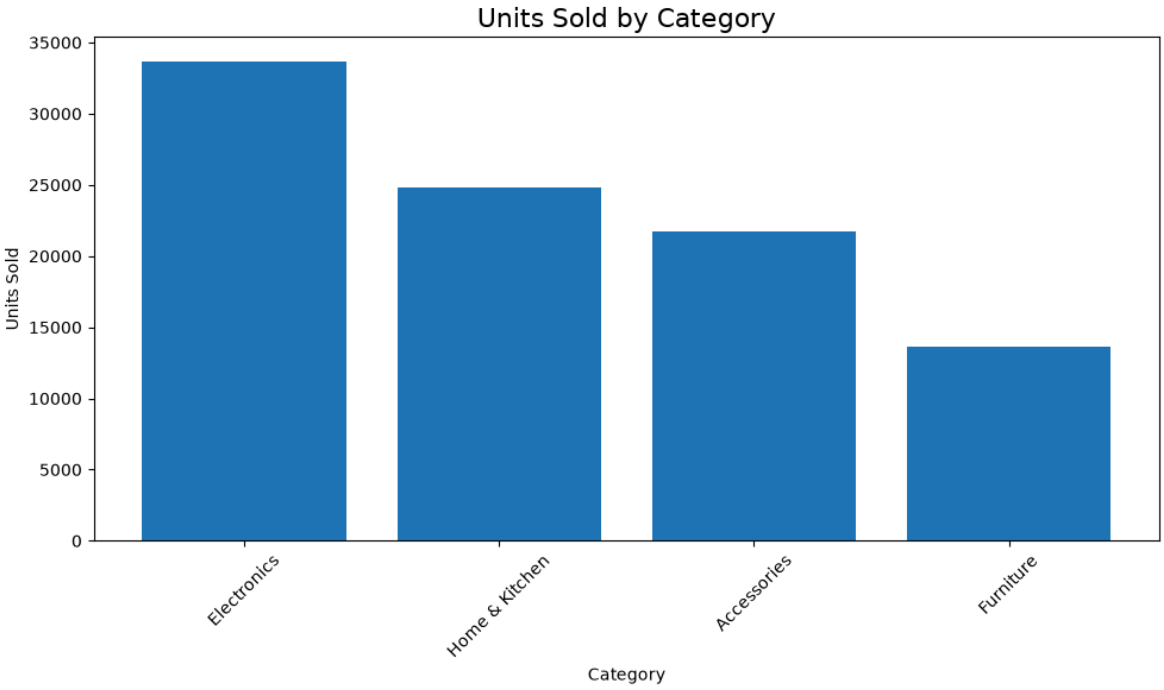
Electronics	15247167.14	33704
Home & Kitchen	11182785.90	24799
Accessories	9833968.69	21763
Furniture	6231783.08	13609



```
======================================================================
CATEGORY UNITS SOLD
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>category</th> <th>Units_Sold</th> </thead> <tbody> <th>1</th> <th>3</th> <th>0</th> <th>2</th> </tbody> <tbody></tbody>

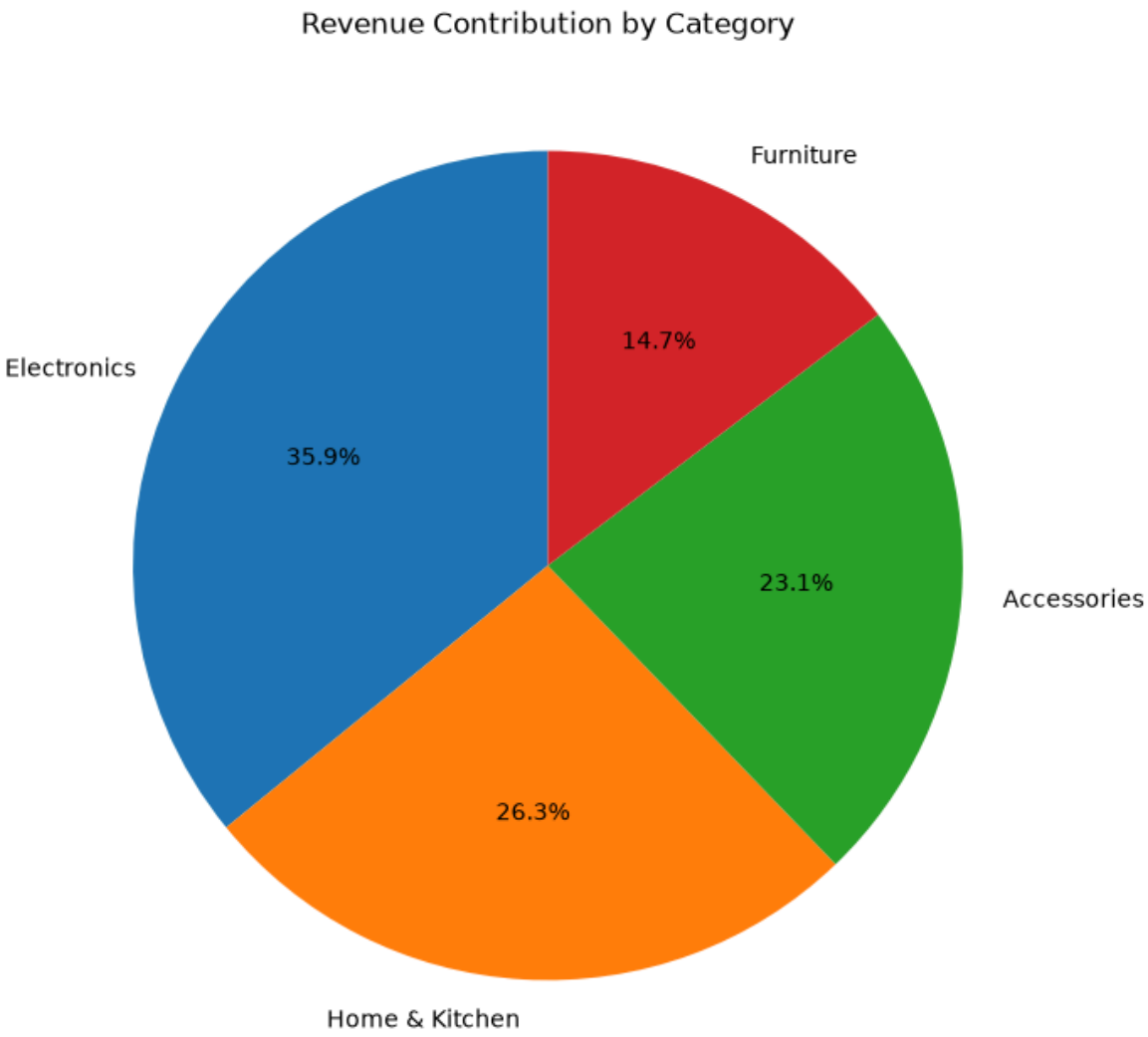
Electronics	33704
Home & Kitchen	24799
Accessories	21763
Furniture	13609




```
======================================================================
CATEGORY REVENUE CONTRIBUTION
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>category</th> <th>Total_Revenue</th> <th>Units_Sold</th> <th>Revenue_Percentage</th> </thead> <tbody> <th>1</th> <th>3</th> <th>0</th> <th>2</th> </tbody> <tbody></tbody>

Electronics	15247167.14	33704	35.88
Home & Kitchen	11182785.90	24799	26.32
Accessories	9833968.69	21763	23.14
Furniture	6231783.08	13609	14.66


Revenue Contribution by Category
Furniture
14.7%
Electronics
35.9%
23.1%
Accessories
26.3%
Home & Kitchen

```
============================================================
PRODUCT ANALYSIS SUMMARY
============================================================

Key Findings

✓ Identified the highest revenue-generating products.

✓ Identified the highest-selling products.

✓ Compared revenue versus quantity sold.

✓ Evaluated category-wise performance.

✓ Calculated revenue contribution by category.

Business Recommendations

1. Increase inventory for top-selling products.

2. Allocate more marketing budget to high-revenue categories.

3. Improve visibility of low-performing categories.

4. Bundle premium products with fast-moving products.

5. Monitor category performance regularly.
```

In [54]:

```
# ============================================================
# CUSTOMER ANALYSIS
# ============================================================
#
# Business Questions
#
# 1. Who are the Top 10 Customers by Revenue?
# 2. Which customers placed the highest number of orders?
# 3. What is the customer spending distribution?
# 4. How much does the average customer spend?
# 5. Which customers contribute 80% of revenue? (Pareto Analysis)
#
# Business Objective
#
# • Identify valuable customers
# • Improve customer retention
# • Design loyalty programs
# • Increase customer lifetime value
# ============================================================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

plt.rcParams["figure.figsize"] = (12,6)

# ============================================================
# STEP 1 : CREATE CUSTOMER SALES DATASET
# ============================================================

customer_sales = (
```

```
    orders
    .merge(customers, on="customer_id", how="left")
)

print("="*70)
print("CUSTOMER SALES DATASET")
print("="*70)

print("Shape :", customer_sales.shape)

display(customer_sales.head())

# ============================================================
# STEP 2 : CUSTOMER OVERVIEW
# ============================================================

print("\nDataset Information\n")

customer_sales.info()

# ============================================================
# BUSINESS QUESTION 1
# TOP 10 CUSTOMERS BY REVENUE
# ============================================================

top_customers = (
    customer_sales
    .groupby(
        ["customer_id","first_name","last_name"],
        as_index=False
    )
    .agg(
        Total_Revenue=("total_price","sum"),
        Total_Orders=("order_id","count")
    )
    .sort_values(
        "Total_Revenue",
        ascending=False
    )
    .head(10)
)

print("="*70)
print("TOP 10 CUSTOMERS BY REVENUE")
print("="*70)

display(top_customers)

plt.figure(figsize=(14,6))

plt.bar(
    top_customers["first_name"] + " " + top_customers["last_name"],
    top_customers["Total_Revenue"]
)

plt.title("Top 10 Customers by Revenue")

plt.xlabel("Customer")

plt.ylabel("Revenue ($)")
```

```
plt.xticks(rotation=45,ha="right")

plt.tight_layout()

plt.show()

print("""
Business Insight

• These customers generate the highest revenue.

• They should be targeted with premium offers.

• Loyalty rewards should prioritize these customers.
""")

# ============================================================
# BUSINESS QUESTION 2
# TOP CUSTOMERS BY NUMBER OF ORDERS
# ============================================================

top_order_customers = (
    customer_sales
    .groupby(
        ["customer_id","first_name","last_name"],
        as_index=False
    )
    .agg(
        Total_Orders=("order_id","count"),
        Revenue=("total_price","sum")
    )
    .sort_values(
        "Total_Orders",
        ascending=False
    )
    .head(10)
)

print("="*70)
print("TOP CUSTOMERS BY ORDERS")
print("="*70)

display(top_order_customers)

plt.figure(figsize=(14,6))

plt.bar(
    top_order_customers["first_name"]+" "+top_order_customers["last_name"],
    top_order_customers["Total_Orders"]
)

plt.title("Top Customers by Number of Orders")

plt.xlabel("Customer")

plt.ylabel("Orders")

plt.xticks(rotation=45,ha="right")
```

```
plt.tight_layout()

plt.show()

print("""
Business Insight

Customers placing frequent orders are ideal candidates
for loyalty and subscription programs.
""")

# ============================================================
# BUSINESS QUESTION 3
# CUSTOMER SPENDING DISTRIBUTION
# ============================================================

customer_spending = (
    customer_sales
    .groupby("customer_id")
    .agg(
        Total_Spending=("total_price","sum")
    )
)

print("="*70)
print("CUSTOMER SPENDING DISTRIBUTION")
print("="*70)

display(customer_spending.head())

plt.figure(figsize=(10,6))

plt.hist(
    customer_spending["Total_Spending"],
    bins=30
)

plt.title("Customer Spending Distribution")

plt.xlabel("Total Spending ($)")

plt.ylabel("Number of Customers")

plt.tight_layout()

plt.show()

print("""
Business Insight

The spending distribution reveals whether revenue
comes from many average customers or a few high-value customers.
""")

# ============================================================
# BUSINESS QUESTION 4
# CUSTOMER SPENDING STATISTICS
# ============================================================

customer_statistics = pd.DataFrame({
```

```
"Metric":[
"Average Spending",
"Median Spending",
"Maximum Spending",
"Minimum Spending"
],

"Value":[

customer_spending["Total_Spending"].mean(),

customer_spending["Total_Spending"].median(),

customer_spending["Total_Spending"].max(),

customer_spending["Total_Spending"].min()

]

})

print("="*70)
print("CUSTOMER SPENDING STATISTICS")
print("="*70)

display(customer_statistics)

# ============================================================
# BUSINESS QUESTION 5
# PARETO ANALYSIS (80-20 RULE)
# ============================================================

pareto = (
    customer_sales
    .groupby(
        ["customer_id","first_name","last_name"],
        as_index=False
    )
    .agg(
        Revenue=("total_price","sum")
    )
)

pareto = pareto.sort_values(
    "Revenue",
    ascending=False
)

pareto["Cumulative Revenue"] = pareto["Revenue"].cumsum()

pareto["Revenue %"] = (
    pareto["Revenue"]/
    pareto["Revenue"].sum()*100
)

pareto["Cumulative %"] = pareto["Revenue %"].cumsum()

display(pareto.head(20))
```

```
plt.figure(figsize=(14,6))

plt.plot(
    pareto.index+1,
    pareto["Cumulative %"],
    linewidth=2
)

plt.axhline(
    y=80,
    linestyle="--"
)

plt.title("Pareto Analysis (80-20 Rule)")

plt.xlabel("Customers")

plt.ylabel("Cumulative Revenue %")

plt.grid(True)

plt.tight_layout()

plt.show()

top_80 = len(
    pareto[
        pareto["Cumulative %"]<=80
    ]
)

print(f"\nCustomers contributing 80% revenue : {top_80}")

print("""
Business Insight

• A small percentage of customers usually contributes
most of the revenue.

• These customers should receive personalized offers,
exclusive discounts and loyalty rewards.
""")

# ============================================================
# CUSTOMER ANALYSIS SUMMARY
# ============================================================

print("""
============================================================
CUSTOMER ANALYSIS SUMMARY
============================================================

Key Findings

✓ Top Revenue Customers Identified

✓ Most Frequent Buyers Identified

✓ Customer Spending Distribution Analyzed
```

```
✓ Spending Statistics Generated

✓ Pareto (80-20) Analysis Completed

Business Recommendations

1. Reward high-value customers.

2. Increase repeat purchase rate.

3. Create loyalty membership.

4. Target medium-value customers for upselling.

5. Monitor customer churn regularly.

============================================================
END OF CUSTOMER ANALYSIS
============================================================
""")
```

```
======================================================================
CUSTOMER SALES DATASET
======================================================================
Shape : (15000, 12)
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>order_id</th> <th>order_date</th> <th>customer_id</th> <th>total_price</th> <th>Year</th> <th>Month</th> <th>Month_Number</th> <th>first_name</th> <th>last_name</th> <th>address</th> <th>email</th> <th>phone_number</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

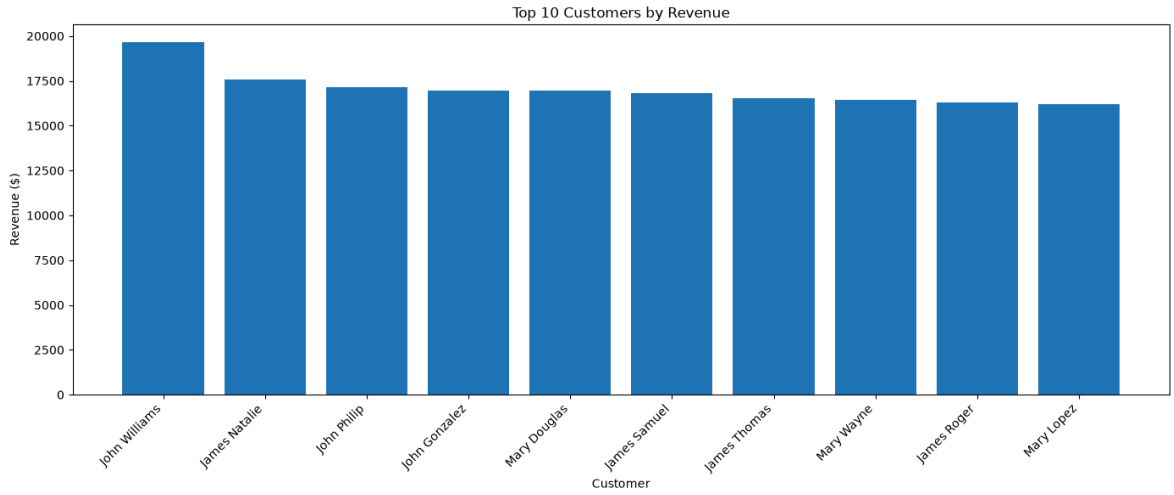
7324	2023-11-27	2160	955.86	2023	November	11	James	Abigail	242 Garden St, Liberty City, AL	jabigail2160@
8938	2024-08-31	8497	984.91	2024	August	8	John	Dennis	101 Maple Dr, Smallville, KS	jdennis8497@
4238	2023-12-05	6295	649.18	2023	December	12	Mary	Terry	242 Garden St, Liberty City, AL	mterry6295@
10944	2024-02-22	2936	54.83	2024	February	2	James	Jane	474 Marina Ave, Harbor View, VT	jjane2936@
11075	2024-08-14	2994	320.24	2024	August	8	James	Natalie	585 Lighthouse Rd, Sea Haven, LA	jnatalie2994@

```
Dataset Information

<class 'pandas.DataFrame'>
RangeIndex: 15000 entries, 0 to 14999
Data columns (total 12 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   order_id      15000 non-null  int64
 1   order_date    15000 non-null  datetime64[us]
 2   customer_id   15000 non-null  int64
 3   total_price   15000 non-null  float64
 4   Year          15000 non-null  int32
 5   Month         15000 non-null  str
 6   Month_Number  15000 non-null  int32
 7   first_name    15000 non-null  str
 8   last_name     15000 non-null  str
 9   address       15000 non-null  str
 10  email         15000 non-null  str
 11  phone_number  15000 non-null  str
dtypes: datetime64[us](1), float64(1), int32(2), int64(2), str(6)
memory usage: 1.3 MB
======================================================================
TOP 10 CUSTOMERS BY REVENUE
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>customer_id</th> <th>first_name</th> <th>last_name</th> <th>Total_Revenue</th> <th>Total_Orders</th> </thead> <tbody> <th>9802</th> <th>4113</th> <th>8940</th> <th>9952</th> <th>7386</th> <th>3612</th> <th>1415</th> <th>5736</th> <th>1732</th> <th>7921</th> </tbody> <tbody> </tbody>

9803	John	Williams	19651.70	2
4114	James	Natalie	17567.00	2
8941	John	Philip	17124.03	2
9953	John	Gonzalez	16956.25	2
7387	Mary	Douglas	16952.30	2
3613	James	Samuel	16813.38	2
1416	James	Thomas	16522.12	2
5737	Mary	Wayne	16442.62	2
1733	James	Roger	16292.75	2
7922	Mary	Lopez	16193.07	2



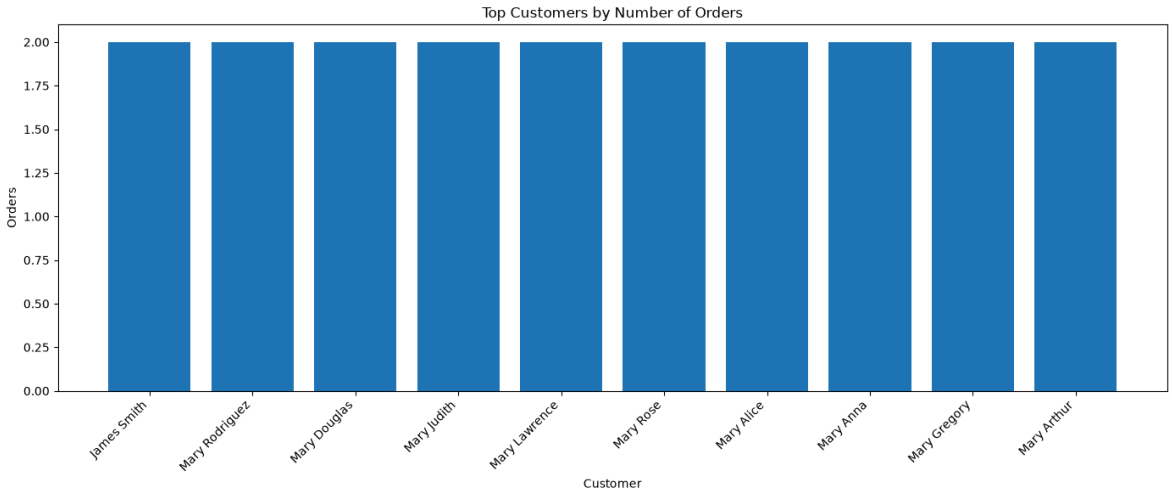

```
Business Insight
```

- These customers generate the highest revenue.
- They should be targeted with premium offers.
- Loyalty rewards should prioritize these customers.

```
======================================================================
TOP CUSTOMERS BY ORDERS
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>customer_id</th> <th>first_name</th> <th>last_name</th> <th>Total_Orders</th> <th>Revenue</th> </thead> <tbody> <th>0</th> <th>4138</th> <th>4166</th> <th>4163</th> <th>7956</th> <th>4161</th> <th>7957</th> <th>4159</th> <th>4158</th> <th>7958</th> </tbody> <tbody> </tbody>

1	James	Smith	2	1096.70
4139	Mary	Rodriguez	2	8393.92
4167	Mary	Douglas	2	10003.60
4164	Mary	Judith	2	6891.96
7957	Mary	Lawrence	2	2794.12
4162	Mary	Rose	2	5039.58
7958	Mary	Alice	2	5366.44
4160	Mary	Anna	2	9565.86
4159	Mary	Gregory	2	1664.32
7959	Mary	Arthur	2	13837.75



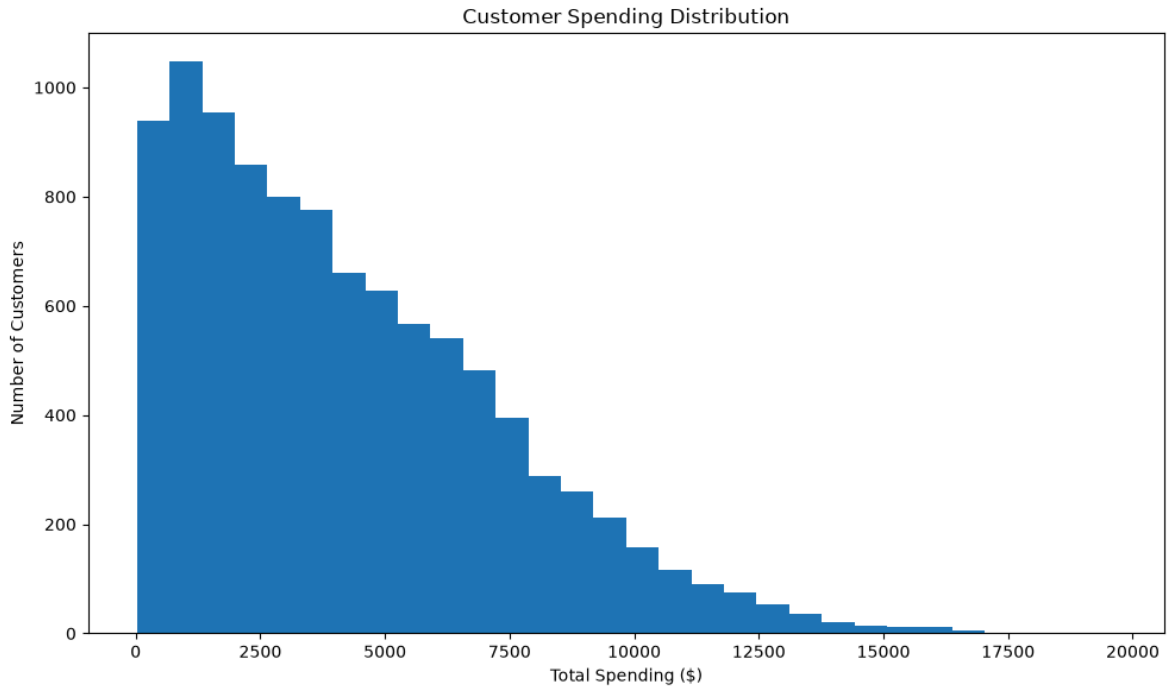

```
Business Insight

Customers placing frequent orders are ideal candidates
for loyalty and subscription programs.

======================================================================
CUSTOMER SPENDING DISTRIBUTION
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Total_Spending</th> <th>customer_id</th> <th></th> </thead> <tbody> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> </tbody> <tbody></tbody>

1096.70	
4378.05	
3177.20	
670.56	
9667.22	



```
Business Insight

The spending distribution reveals whether revenue
comes from many average customers or a few high-value customers.

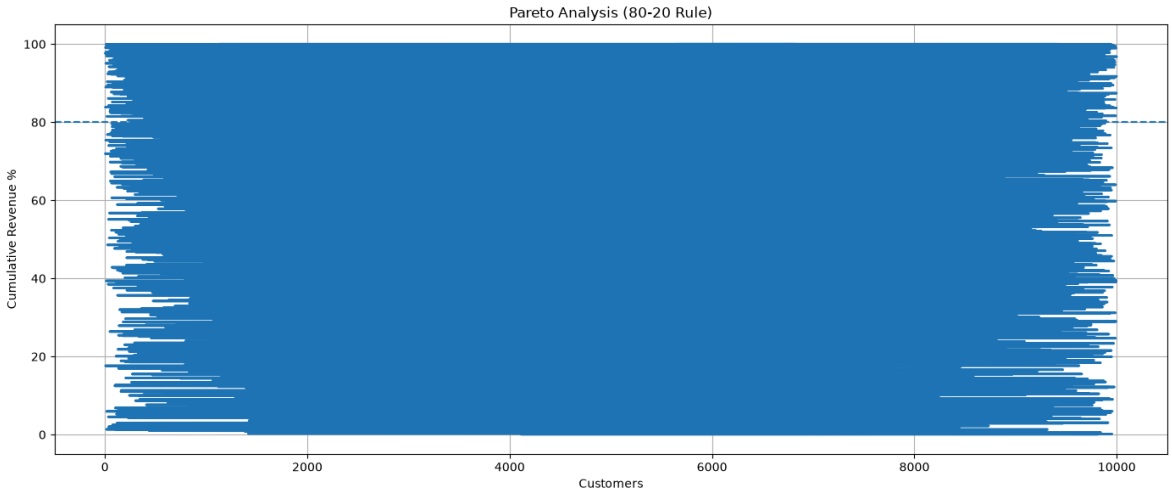
======================================================================
CUSTOMER SPENDING STATISTICS
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Metric</th> <th>Value</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> </tbody> <tbody></tbody>

Average Spending	4249.570481
Median Spending	3625.945000
Maximum Spending	19651.700000
Minimum Spending	24.580000

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>customer_id</th> <th>first_name</th> <th>last_name</th> <th>Revenue</th> <th>Cumulative Revenue</th> <th>Revenue %</th> <th>Cumulative %</th> </thead> <tbody> <th>9802</th> <th>4113</th> <th>8940</th> <th>9952</th> <th>7386</th> <th>3612</th> <th>1415</th> <th>5736</th> <th>1732</th> <th>7921</th> <th>5444</th> <th>5052</th> <th>6991</th> <th>9840</th> <th>2769</th> <th>8370</th> <th>3181</th> <th>2387</th> <th>4539</th> <th>3661</th> </tbody> <tbody></tbody>

9803	John	Williams	19651.70	19651.70	0.046244	0.046244
4114	James	Natalie	17567.00	37218.70	0.041338	0.087582
8941	John	Philip	17124.03	54342.73	0.040296	0.127878
9953	John	Gonzalez	16956.25	71298.98	0.039901	0.167779
7387	Mary	Douglas	16952.30	88251.28	0.039892	0.207671
3613	James	Samuel	16813.38	105064.66	0.039565	0.247236
1416	James	Thomas	16522.12	121586.78	0.038880	0.286115
5737	Mary	Wayne	16442.62	138029.40	0.038692	0.324808
1733	James	Roger	16292.75	154322.15	0.038340	0.363148
7922	Mary	Lopez	16193.07	170515.22	0.038105	0.401253
5445	Mary	Benjamin	16179.95	186695.17	0.038074	0.439327
5053	Mary	Gonzalez	16165.71	202860.88	0.038041	0.477368
6992	Mary	Lauren	16162.25	219023.13	0.038033	0.515401
9841	John	Alexander	16119.30	235142.43	0.037932	0.553332
2770	James	Nicole	16024.80	251167.23	0.037709	0.591041
8371	John	Alexander	15949.86	267117.09	0.037533	0.628574
3182	James	Rose	15885.47	283002.56	0.037381	0.665956
2388	James	Davis	15861.20	298863.76	0.037324	0.703280
4540	Mary	Abigail	15802.72	314666.48	0.037187	0.740467
3662	James	Victoria	15702.48	330368.96	0.036951	0.777417



Customers contributing 80% revenue : 5064

Business Insight

• A small percentage of customers usually contributes most of the revenue.

• These customers should receive personalized offers, exclusive discounts and loyalty rewards.

==========
CUSTOMER ANALYSIS SUMMARY
==========

Key Findings

✓ Top Revenue Customers Identified

✓ Most Frequent Buyers Identified

✓ Customer Spending Distribution Analyzed

✓ Spending Statistics Generated

✓ Pareto (80-20) Analysis Completed

Business Recommendations

1. Reward high-value customers.

2. Increase repeat purchase rate.

3. Create loyalty membership.

4. Target medium-value customers for upselling.

5. Monitor customer churn regularly.

==========
END OF CUSTOMER ANALYSIS
==========

In [55]:
```
# ============================================================
# PAYMENT ANALYSIS
# ============================================================
#
# Business Questions
#
# 1. Which payment method is used the most?
# 2. Which payment method generates the highest revenue?
# 3. What is the transaction success rate?
# 4. What is the transaction failure rate?
# 5. Which payment methods should the business promote?
#
# Business Objective
#
# • Understand customer payment preferences
# • Improve payment success rate
# • Increase customer convenience
# • Reduce payment failures
# ============================================================

import pandas as pd
import matplotlib.pyplot as plt

plt.rcParams["figure.figsize"] = (12,6)

# ============================================================
# STEP 1 : CREATE PAYMENT DATASET
# ============================================================

payment_analysis = (
    payments
    .merge(
        orders,
        on="order_id",
        how="left"
    )
)

print("="*70)
print("PAYMENT DATASET")
print("="*70)

print("Shape :", payment_analysis.shape)

display(payment_analysis.head())

# ============================================================
# STEP 2 : DATASET INFORMATION
# ============================================================

print("\nDataset Information\n")

payment_analysis.info()

# ============================================================
# BUSINESS QUESTION 1
# MOST USED PAYMENT METHOD
# ============================================================
```

```
payment_count = (
    payment_analysis
    .groupby(
        "payment_method",
        as_index=False
    )
    .agg(
        Total_Transactions=("payment_id","count")
    )
    .sort_values(
        "Total_Transactions",
        ascending=False
    )
)

print("="*70)
print("PAYMENT METHOD USAGE")
print("="*70)

display(payment_count)

plt.figure(figsize=(10,6))

plt.bar(
    payment_count["payment_method"],
    payment_count["Total_Transactions"]
)

plt.title("Most Used Payment Methods")

plt.xlabel("Payment Method")

plt.ylabel("Number of Transactions")

plt.tight_layout()

plt.show()

print("""
Business Insight

Customers clearly prefer certain payment methods.

The most frequently used payment method should always
provide a smooth payment experience.
""")

# ============================================================
# BUSINESS QUESTION 2
# REVENUE BY PAYMENT METHOD
# ============================================================

payment_revenue = (
    payment_analysis
    .groupby(
        "payment_method",
        as_index=False
    )
    .agg(
        Total_Revenue=("amount","sum"),
```

```
        Average_Payment=("amount","mean")
    )
    .sort_values(
        "Total_Revenue",
        ascending=False
    )
)

print("="*70)
print("REVENUE BY PAYMENT METHOD")
print("="*70)

display(payment_revenue)

plt.figure(figsize=(10,6))

plt.bar(
    payment_revenue["payment_method"],
    payment_revenue["Total_Revenue"]
)

plt.title("Revenue by Payment Method")

plt.xlabel("Payment Method")

plt.ylabel("Revenue ($)")

plt.tight_layout()

plt.show()

print("""
Business Insight

The payment method generating the highest revenue
should receive continuous support and promotional offers.
""")

# ============================================================
# BUSINESS QUESTION 3
# TRANSACTION STATUS
# ============================================================

transaction_status = (
    payment_analysis
    .groupby(
        "transaction_status",
        as_index=False
    )
    .agg(
        Transactions=("payment_id","count")
    )
)

transaction_status["Percentage"] = (
    transaction_status["Transactions"] /
    transaction_status["Transactions"].sum() * 100
).round(2)

print("="*70)
```

```
print("TRANSACTION STATUS")
print("="*70)

display(transaction_status)

plt.figure(figsize=(8,8))

plt.pie(
    transaction_status["Transactions"],
    labels=transaction_status["transaction_status"],
    autopct="%1.1f%%",
    startangle=90
)

plt.title("Transaction Status Distribution")

plt.show()

print("""
Business Insight

A high success rate indicates a healthy payment system.

Failed transactions should be monitored regularly.
""")

# ============================================================
# BUSINESS QUESTION 4
# PAYMENT METHOD VS TRANSACTION STATUS
# ============================================================

payment_status = pd.crosstab(
    payment_analysis["payment_method"],
    payment_analysis["transaction_status"]
)

print("="*70)
print("PAYMENT METHOD VS TRANSACTION STATUS")
print("="*70)

display(payment_status)

payment_status.plot(
    kind="bar",
    figsize=(12,6)
)

plt.title("Payment Method vs Transaction Status")

plt.xlabel("Payment Method")

plt.ylabel("Transactions")

plt.xticks(rotation=0)

plt.tight_layout()

plt.show()

print("""
```

```
Business Insight

This analysis identifies which payment methods
have the highest number of failed transactions.

These methods should be investigated to improve
customer experience.
""")

# ============================================================
# BUSINESS QUESTION 5
# PAYMENT SHARE
# ============================================================

payment_count["Usage %"] = (
    payment_count["Total_Transactions"] /
    payment_count["Total_Transactions"].sum() * 100
).round(2)

print("="*70)
print("PAYMENT METHOD SHARE")
print("="*70)

display(payment_count)

plt.figure(figsize=(8,8))

plt.pie(
    payment_count["Usage %"],
    labels=payment_count["payment_method"],
    autopct="%1.1f%%",
    startangle=90
)

plt.title("Payment Method Share")

plt.show()

# ============================================================
# PAYMENT ANALYSIS SUMMARY
# ============================================================

print("""
================================================================
PAYMENT ANALYSIS SUMMARY
================================================================

Key Findings

✓ Most Used Payment Method

✓ Highest Revenue Payment Method

✓ Transaction Success Rate

✓ Payment Failure Analysis

✓ Payment Method Distribution

Business Recommendations
```

```
1. Promote the most popular payment method.

2. Reduce payment failures.

3. Improve payment gateway reliability.

4. Provide cashback offers on preferred payment methods.

5. Monitor payment failures weekly.

============================================================
END OF PAYMENT ANALYSIS
============================================================
""")
```

```
======================================================================
PAYMENT DATASET
======================================================================
Shape : (15000, 11)
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>payment_id</th> <th>order_id</th> <th>payment_method</th> <th>amount</th> <th>transaction_status</th> <th>order_date</th> <th>customer_id</th> <th>total_price</th> <th>Year</th> <th>Month</th> <th>Month_Number</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

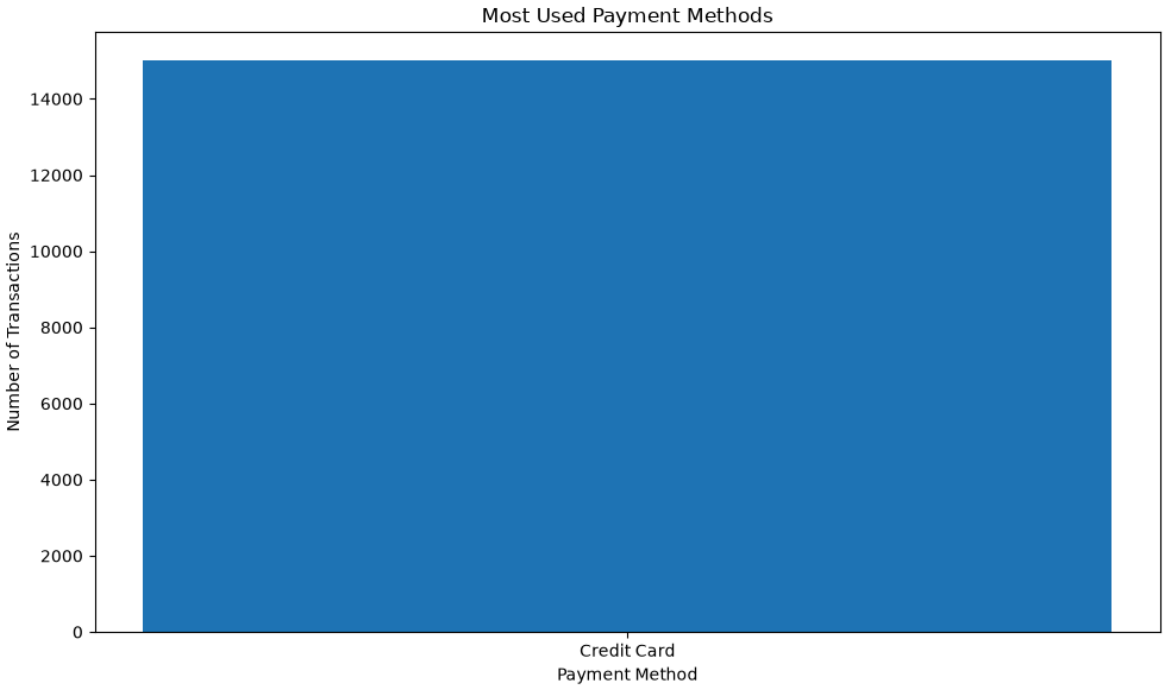
1	1	Credit Card	232.15	Completed	2024-10-10	8002	165.18	2024	October	10
2	2	Credit Card	454.44	Completed	2024-08-25	5097	8314.50	2024	August	8
3	3	Credit Card	26.14	Completed	2024-05-21	4670	1775.76	2024	May	5
4	4	Credit Card	985.70	Completed	2024-06-05	3875	3916.47	2024	June	6
5	5	Credit Card	733.32	Completed	2024-10-29	5507	1478.19	2024	October	10

```
Dataset Information

<class 'pandas.DataFrame'>
RangeIndex: 15000 entries, 0 to 14999
Data columns (total 11 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   payment_id          15000 non-null  int64
 1   order_id            15000 non-null  int64
 2   payment_method      15000 non-null  str
 3   amount              15000 non-null  float64
 4   transaction_status  15000 non-null  str
 5   order_date          15000 non-null  datetime64[us]
 6   customer_id         15000 non-null  int64
 7   total_price         15000 non-null  float64
 8   Year                15000 non-null  int32
 9   Month               15000 non-null  str
 10  Month_Number        15000 non-null  int32
dtypes: datetime64[us](1), float64(2), int32(2), int64(3), str(3)
memory usage: 1.1 MB
========================================================================
PAYMENT METHOD USAGE
========================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>payment_method</th> <th>Total_Transactions</th> </thead> <tbody> <th>0</th> </tbody> <tbody></tbody>

Credit Card	15000



```
Business Insight

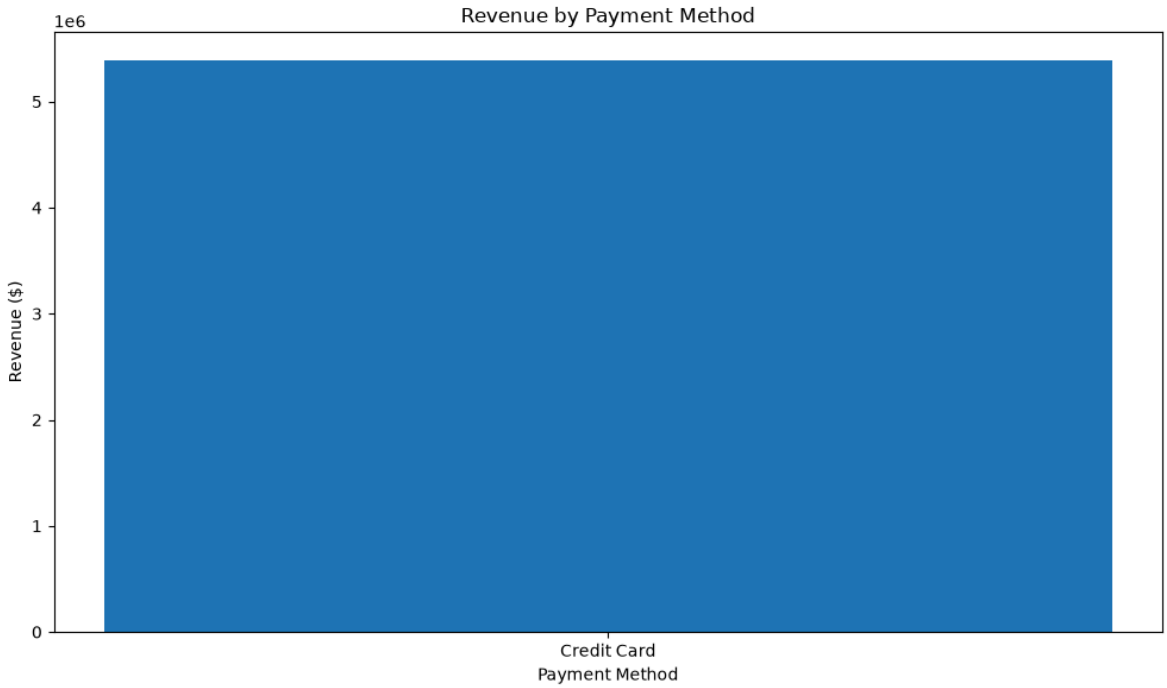
Customers clearly prefer certain payment methods.

The most frequently used payment method should always
provide a smooth payment experience.

======================================================================
REVENUE BY PAYMENT METHOD
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>payment_method</th> <th>Total_Revenue</th> <th>Average_Payment</th> </thead> <tbody> <th>0</th> </tbody> <tbody></tbody>

Credit Card	5384649.73	358.976649	



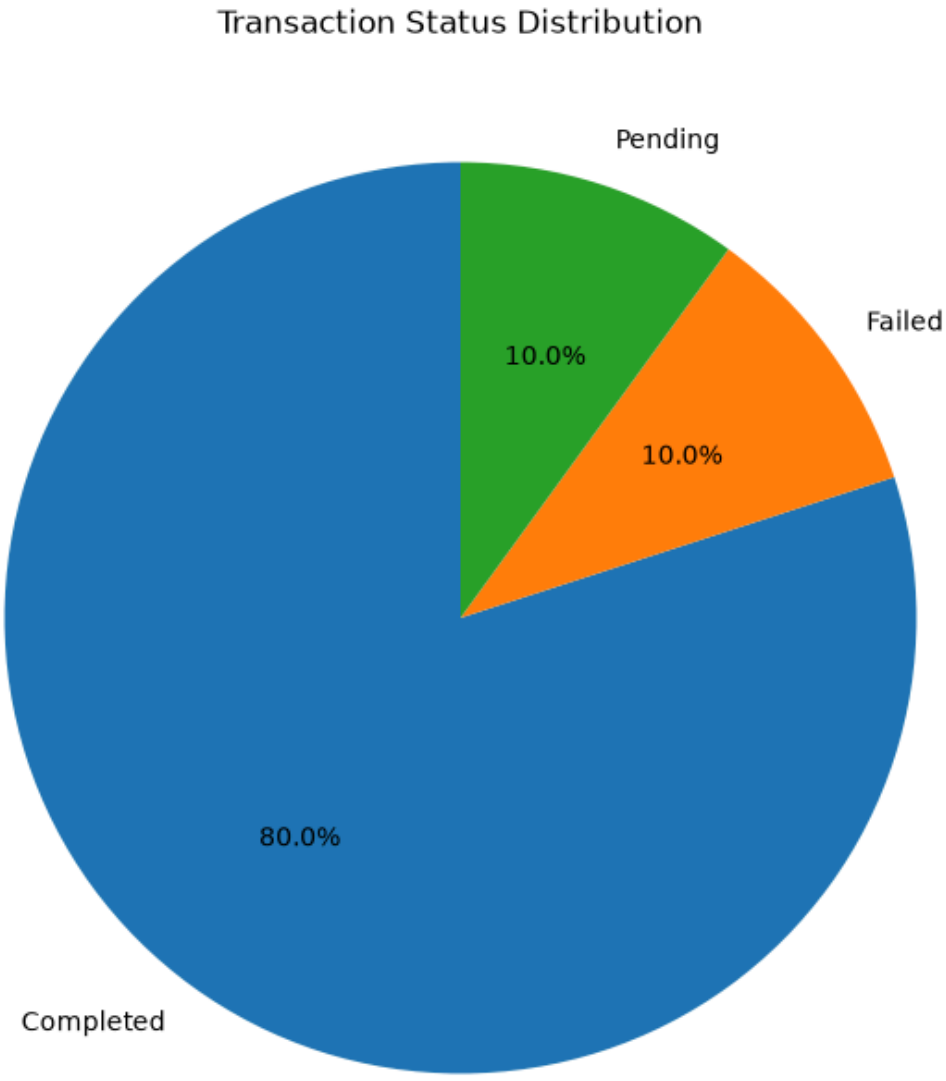

```
Business Insight

The payment method generating the highest revenue
should receive continuous support and promotional offers.

======================================================================
TRANSACTION STATUS
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>transaction_status</th> <th>Transactions</th> <th>Percentage</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> </tbody> <tbody></tbody>

Completed	12000	80.0	
Failed	1500	10.0	
Pending	1500	10.0	




```
Business Insight

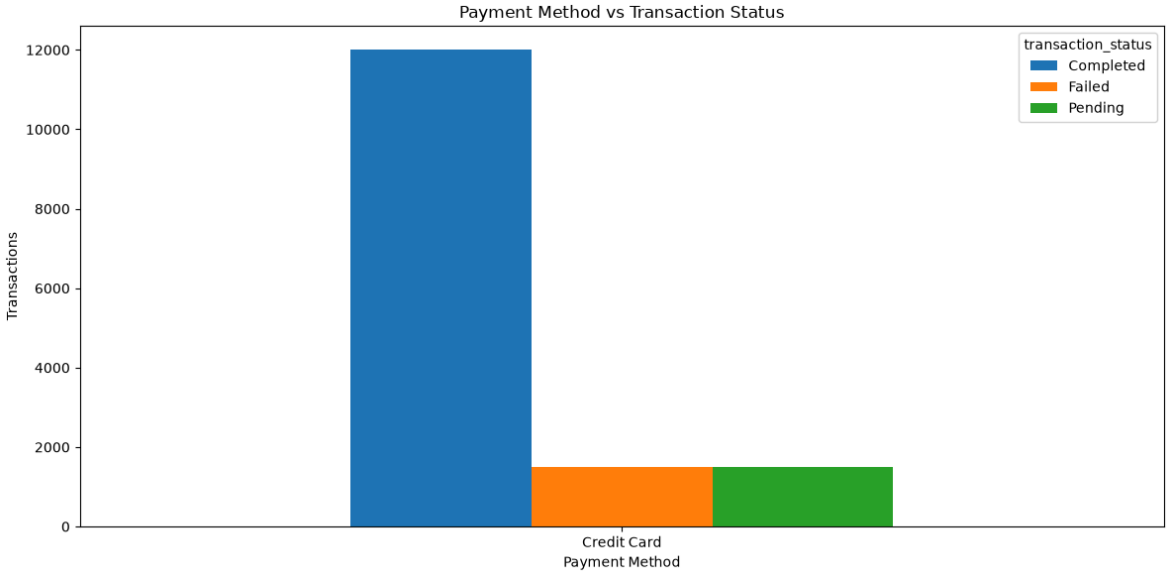
A high success rate indicates a healthy payment system.

Failed transactions should be monitored regularly.

======================================================================
PAYMENT METHOD VS TRANSACTION STATUS
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th>transaction_status</th> <th>Completed</th> <th>Failed</th> <th>Pending</th> <th>payment_method</th> <th></th> <th></th> <th></th> </thead> <tbody> <th>Credit Card</th> </tbody> <tbody></tbody>

```
 12000   1500   1500
```




```
Business Insight

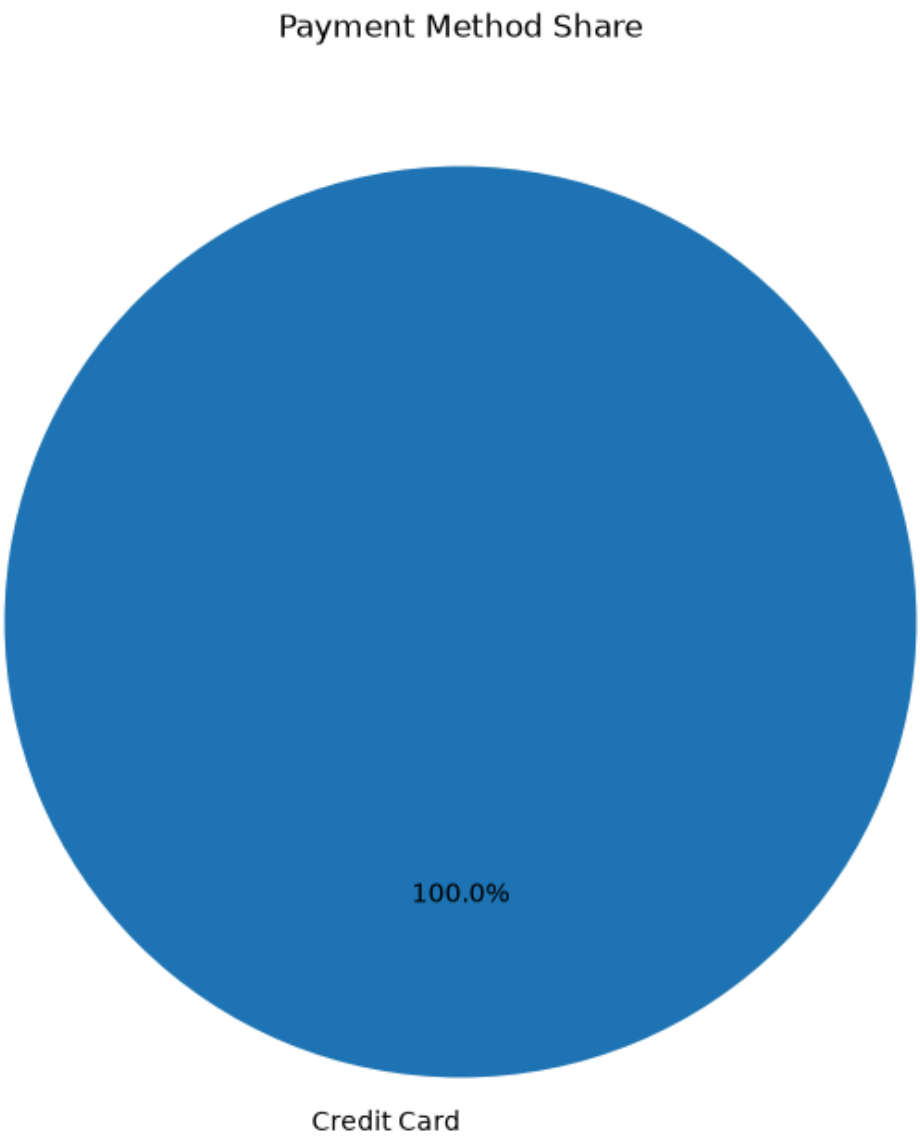
This analysis identifies which payment methods
have the highest number of failed transactions.

These methods should be investigated to improve
customer experience.

======================================================================
PAYMENT METHOD SHARE
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>payment_method</th> <th>Total_Transactions</th> <th>Usage %</th> </thead> <tbody> <th>0</th> </tbody> <tbody></tbody>

Credit Card	15000	100.0	


Payment Method Share
100.0%
Credit Card

```
============================================================
PAYMENT ANALYSIS SUMMARY
============================================================

Key Findings

✓ Most Used Payment Method

✓ Highest Revenue Payment Method

✓ Transaction Success Rate

✓ Payment Failure Analysis

✓ Payment Method Distribution

Business Recommendations

1. Promote the most popular payment method.

2. Reduce payment failures.

3. Improve payment gateway reliability.

4. Provide cashback offers on preferred payment methods.

5. Monitor payment failures weekly.

============================================================
END OF PAYMENT ANALYSIS
============================================================
```

In [56]:

```
# ============================================================
# SHIPPING ANALYSIS
# ============================================================
#
# Business Questions
#
# 1. What is the average delivery time?
# 2. Which shipping carrier delivers the fastest?
# 3. Which shipping carrier delivers the slowest?
# 4. What is the shipment status distribution?
# 5. Which carrier handles the most shipments?
#
# Business Objective
#
# • Improve delivery performance
# • Identify efficient shipping partners
# • Reduce delivery delays
# • Enhance customer satisfaction
#
# ============================================================

import pandas as pd
import matplotlib.pyplot as plt

plt.rcParams["figure.figsize"] = (12,6)

# ============================================================
```

```
# STEP 1 : CREATE SHIPPING DATASET
# ============================================================

shipping_analysis = shipments.merge(
    orders,
    on="order_id",
    how="left"
)

# Delivery Time (Days)

shipping_analysis["Delivery_Days"] = (
    shipping_analysis["delivery_date"] -
    shipping_analysis["shipment_date"]
).dt.days

print("="*70)
print("SHIPPING DATASET")
print("="*70)

print("Shape :", shipping_analysis.shape)

display(shipping_analysis.head())

# ============================================================
# STEP 2 : DATASET INFORMATION
# ============================================================

print("\nDataset Information\n")

shipping_analysis.info()

# ============================================================
# BUSINESS QUESTION 1
# AVERAGE DELIVERY TIME
# ============================================================

average_delivery = shipping_analysis["Delivery_Days"].mean()

print("="*70)
print("AVERAGE DELIVERY TIME")
print("="*70)

print(f"Average Delivery Time : {average_delivery:.2f} Days")

# ============================================================
# BUSINESS QUESTION 2
# DELIVERY TIME BY CARRIER
# ============================================================

carrier_delivery = (
    shipping_analysis
    .groupby(
        "carrier",
        as_index=False
    )
    .agg(
        Average_Delivery=("Delivery_Days","mean"),
        Total_Shipments=("shipment_id","count")
    )
```

```
    .sort_values(
        "Average_Delivery"
    )
)

print("="*70)
print("DELIVERY TIME BY CARRIER")
print("="*70)

display(carrier_delivery)

plt.figure(figsize=(10,6))

plt.bar(
    carrier_delivery["carrier"],
    carrier_delivery["Average_Delivery"]
)

plt.title("Average Delivery Time by Carrier")

plt.xlabel("Carrier")

plt.ylabel("Average Delivery Days")

plt.tight_layout()

plt.show()

print("""
Business Insight

Lower delivery time indicates
better logistics performance.

The fastest carrier should be preferred
for priority deliveries.
""")

# ============================================================
# BUSINESS QUESTION 3
# SHIPMENT STATUS DISTRIBUTION
# ============================================================

shipment_status = (
    shipping_analysis
    .groupby(
        "shipment_status",
        as_index=False
    )
    .agg(
        Shipments=("shipment_id","count")
    )
)

shipment_status["Percentage"] = (
    shipment_status["Shipments"] /
    shipment_status["Shipments"].sum()*100
).round(2)

print("="*70)
```

```
print("SHIPMENT STATUS")
print("="*70)

display(shipment_status)

plt.figure(figsize=(8,8))

plt.pie(
    shipment_status["Shipments"],
    labels=shipment_status["shipment_status"],
    autopct="%1.1f%%",
    startangle=90
)

plt.title("Shipment Status Distribution")

plt.show()

print("""
Business Insight

A high percentage of Delivered shipments
indicates an efficient logistics network.

Delayed or Cancelled shipments
should be monitored closely.
""")

# ============================================================
# BUSINESS QUESTION 4
# SHIPMENTS BY CARRIER
# ============================================================

carrier_shipments = (
    shipping_analysis
    .groupby(
        "carrier",
        as_index=False
    )
    .agg(
        Shipments=("shipment_id","count")
    )
    .sort_values(
        "Shipments",
        ascending=False
    )
)

print("="*70)
print("SHIPMENTS BY CARRIER")
print("="*70)

display(carrier_shipments)

plt.figure(figsize=(10,6))

plt.bar(
    carrier_shipments["carrier"],
    carrier_shipments["Shipments"]
)
```

```
plt.title("Total Shipments by Carrier")

plt.xlabel("Carrier")

plt.ylabel("Number of Shipments")

plt.tight_layout()

plt.show()

print("""
Business Insight

The carrier handling the highest shipment volume
plays a critical role in business operations.

Maintaining strong partnerships with high-volume
carriers is important.
""")

# ============================================================
# BUSINESS QUESTION 5
# DELIVERY TIME DISTRIBUTION
# ============================================================

plt.figure(figsize=(10,6))

plt.hist(
    shipping_analysis["Delivery_Days"],
    bins=20
)

plt.title("Delivery Time Distribution")

plt.xlabel("Delivery Days")

plt.ylabel("Number of Shipments")

plt.tight_layout()

plt.show()

print("""
Business Insight

Most deliveries should cluster around the average.

Orders taking unusually long may indicate
operational bottlenecks.
""")

# ============================================================
# SHIPPING ANALYSIS SUMMARY
# ============================================================

print("""
============================================================
SHIPPING ANALYSIS SUMMARY
============================================================
```

```
Key Findings

✓ Average Delivery Time

✓ Fastest Shipping Carrier

✓ Slowest Shipping Carrier

✓ Shipment Status Distribution

✓ Shipment Volume by Carrier

Business Recommendations

1. Partner more with faster carriers.

2. Investigate delayed shipments.

3. Monitor carrier performance monthly.

4. Improve logistics efficiency.

5. Track delivery KPIs regularly.

============================================================
END OF SHIPPING ANALYSIS
============================================================
""")
```

```
======================================================================
SHIPPING DATASET
======================================================================
Shape : (15000, 14)
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>shipment_id</th> <th>order_id</th> <th>shipment_date</th> <th>carrier</th> <th>tracking_number</th> <th>delivery_date</th> <th>shipment_status</th> <th>order_date</th> <th>customer_id</th> <th>total_price</th> <th>Year</th> <th>Month</th> <th>Month_Number</th> <th>Delivery_Days</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

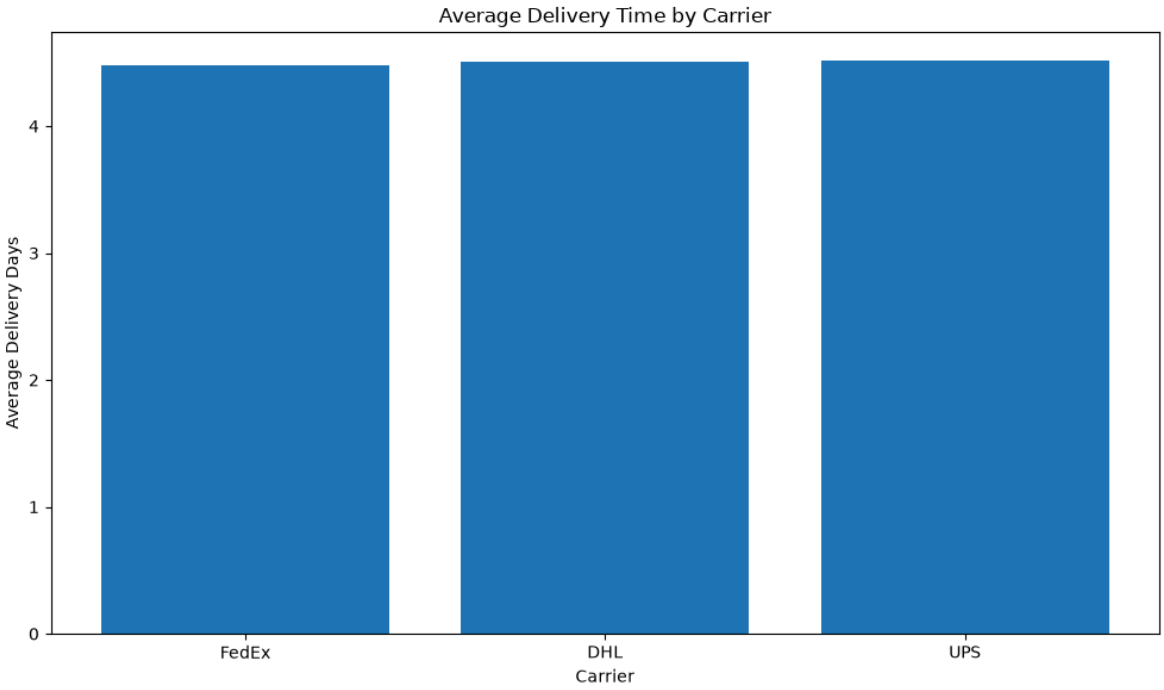
1	1	2024-10-13	UPS	TRK344284	2024-10-14	Pending	2024-10-10	8002	165.18	2024	October	10
2	2	2024-08-27	DHL	TRK718398	2024-08-28	Delivered	2024-08-25	5097	8314.50	2024	August	8
3	3	2024-05-23	FedEx	TRK161368	2024-05-31	Pending	2024-05-21	4670	1775.76	2024	May	5
4	4	2024-06-06	UPS	TRK890181	2024-06-10	Pending	2024-06-05	3875	3916.47	2024	June	6
5	5	2024-11-03	DHL	TRK681341	2024-11-08	Delivered	2024-10-29	5507	1478.19	2024	October	10

```
Dataset Information

<class 'pandas.DataFrame'>
RangeIndex: 15000 entries, 0 to 14999
Data columns (total 14 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   shipment_id      15000 non-null  int64
 1   order_id         15000 non-null  int64
 2   shipment_date    15000 non-null  datetime64[us]
 3   carrier          15000 non-null  str
 4   tracking_number  15000 non-null  str
 5   delivery_date    15000 non-null  datetime64[us]
 6   shipment_status  15000 non-null  str
 7   order_date       15000 non-null  datetime64[us]
 8   customer_id      15000 non-null  int64
 9   total_price      15000 non-null  float64
 10  Year             15000 non-null  int32
 11  Month            15000 non-null  str
 12  Month_Number     15000 non-null  int32
 13  Delivery_Days    15000 non-null  int64
dtypes: datetime64[us](3), float64(1), int32(2), int64(4), str(4)
memory usage: 1.5 MB
======================================================================
AVERAGE DELIVERY TIME
======================================================================
Average Delivery Time : 4.50 Days
======================================================================
DELIVERY TIME BY CARRIER
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>carrier</th> <th>Average_Delivery</th> <th>Total_Shipments</th> </thead> <tbody> <th>1</th> <th>0</th> <th>2</th> </tbody> <tbody></tbody>

FedEx	4.4804	5000	
DHL	4.5038	5000	
UPS	4.5132	5000	




```
Business Insight

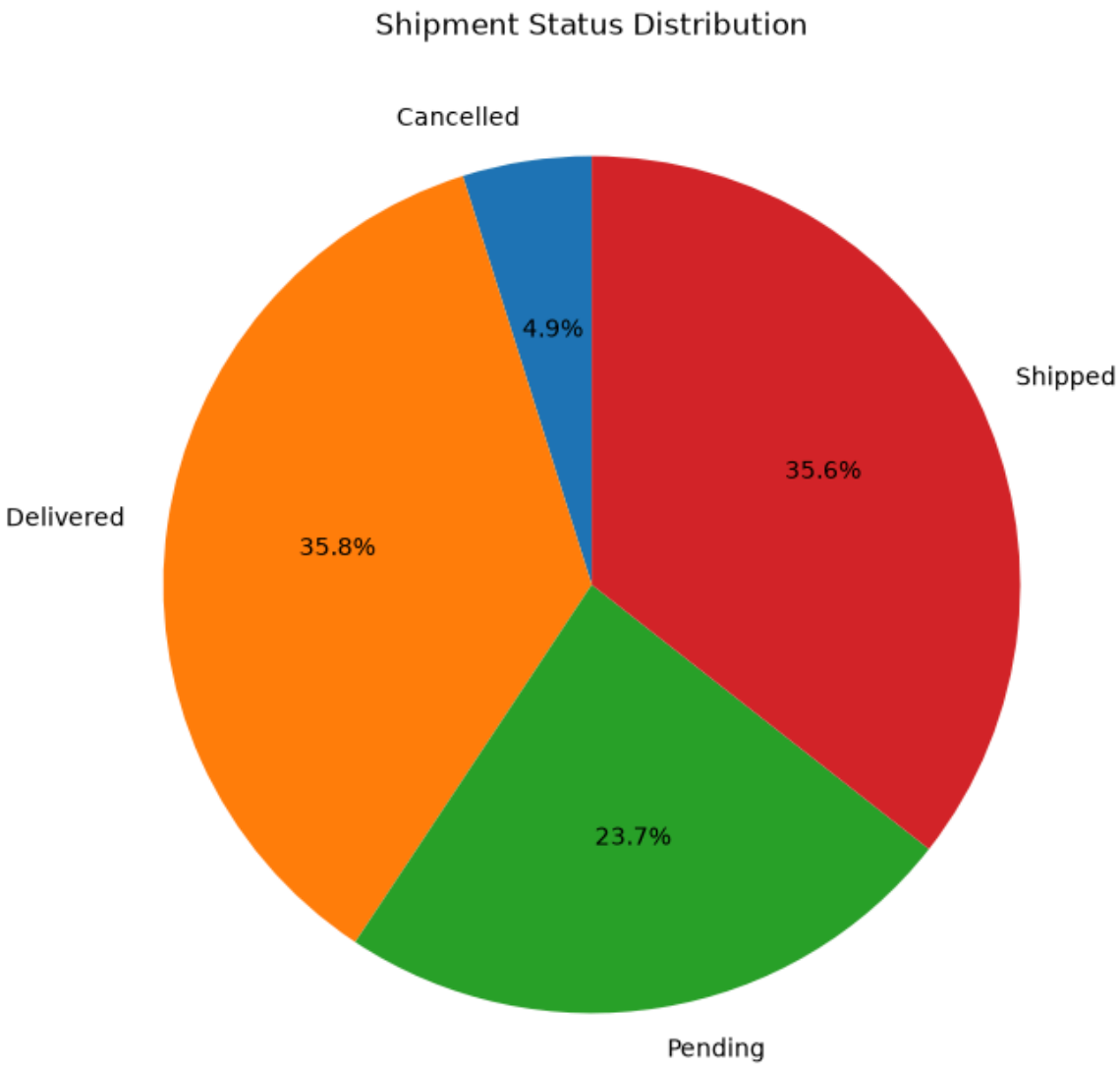
Lower delivery time indicates
better logistics performance.

The fastest carrier should be preferred
for priority deliveries.

======================================================================
SHIPMENT STATUS
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>shipment_status</th> <th>Shipments</th> <th>Percentage</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> </tbody> <tbody> </tbody>

Cancelled	730	4.87
Delivered	5374	35.83
Pending	3559	23.73
Shipped	5337	35.58




```
Business Insight

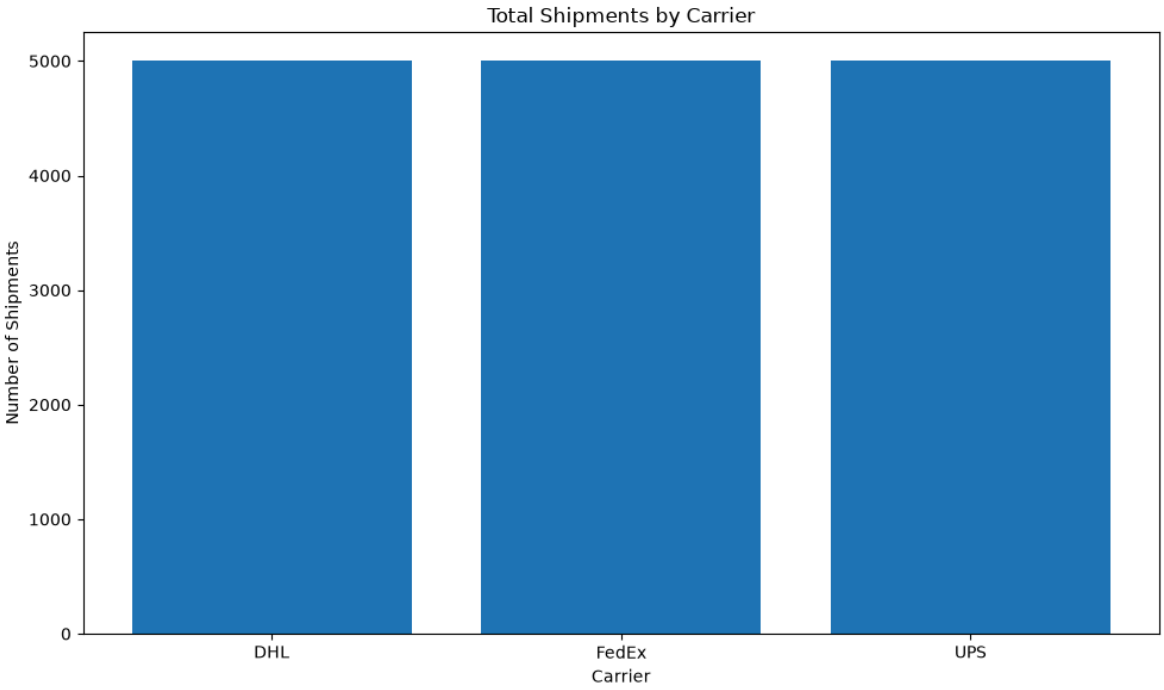
A high percentage of Delivered shipments
indicates an efficient logistics network.

Delayed or Cancelled shipments
should be monitored closely.

======================================================================
SHIPMENTS BY CARRIER
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>carrier</th> <th>Shipments</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> </tbody> <tbody></tbody>

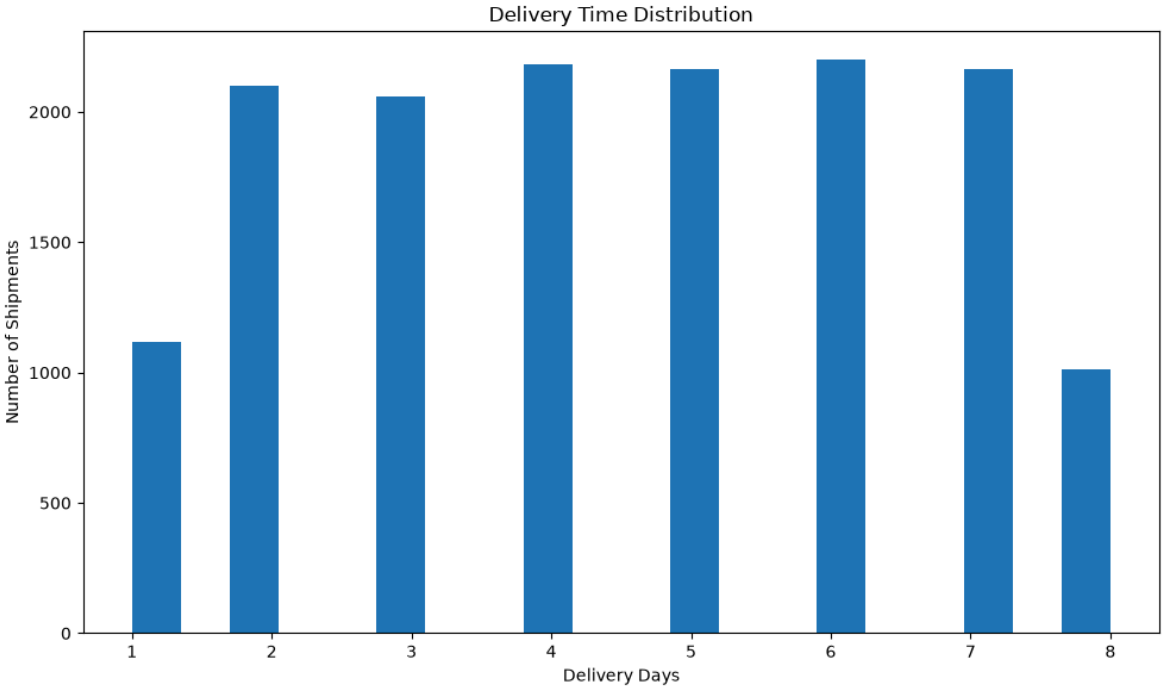
	DHL	5000
	FedEx	5000
	UPS	5000



Business Insight

The carrier handling the highest shipment volume plays a critical role in business operations.

Maintaining strong partnerships with high-volume carriers is important.



```
Business Insight

Most deliveries should cluster around the average.

Orders taking unusually long may indicate
operational bottlenecks.


================================================================
SHIPPING ANALYSIS SUMMARY
================================================================

Key Findings

✓ Average Delivery Time

✓ Fastest Shipping Carrier

✓ Slowest Shipping Carrier

✓ Shipment Status Distribution

✓ Shipment Volume by Carrier

Business Recommendations

1. Partner more with faster carriers.

2. Investigate delayed shipments.

3. Monitor carrier performance monthly.

4. Improve logistics efficiency.

5. Track delivery KPIs regularly.

================================================================
END OF SHIPPING ANALYSIS
================================================================
```

In [57]:
```
# ============================================================
# REVIEW ANALYSIS
# ============================================================
#
# Business Questions
#
# 1. What is the overall average customer rating?
# 2. How are ratings distributed?
# 3. Which products have the highest average ratings?
# 4. Which products have the lowest average ratings?
# 5. Which product categories receive the highest ratings?
#
# Business Objective
#
# • Measure customer satisfaction
# • Identify high-quality products
# • Detect poor-performing products
# • Improve product quality
# • Enhance customer experience
```

```
#
# ============================================================

import pandas as pd
import matplotlib.pyplot as plt

plt.rcParams["figure.figsize"] = (12,6)

# ============================================================
# STEP 1 : CREATE REVIEW DATASET
# ============================================================

review_analysis = reviews.merge(
    products,
    on="product_id",
    how="left"
)

print("="*70)
print("REVIEW DATASET")
print("="*70)

print("Shape :", review_analysis.shape)

display(review_analysis.head())

# ============================================================
# STEP 2 : DATASET INFORMATION
# ============================================================

print("\nDataset Information\n")

review_analysis.info()

# ============================================================
# BUSINESS QUESTION 1
# OVERALL AVERAGE RATING
# ============================================================

average_rating = review_analysis["rating"].mean()

print("="*70)
print("OVERALL CUSTOMER RATING")
print("="*70)

print(f"Average Rating : {average_rating:.2f} / 5")

# ============================================================
# BUSINESS QUESTION 2
# RATING DISTRIBUTION
# ============================================================

rating_distribution = (
    review_analysis
    .groupby(
        "rating",
        as_index=False
    )
    .agg(
        Total_Reviews=("review_id","count")
```

```
    )
)

rating_distribution["Percentage"] = (
    rating_distribution["Total_Reviews"] /
    rating_distribution["Total_Reviews"].sum() * 100
).round(2)

print("="*70)
print("RATING DISTRIBUTION")
print("="*70)

display(rating_distribution)

plt.figure(figsize=(8,6))

plt.bar(
    rating_distribution["rating"].astype(str),
    rating_distribution["Total_Reviews"]
)

plt.title("Customer Rating Distribution")

plt.xlabel("Rating")

plt.ylabel("Number of Reviews")

plt.tight_layout()

plt.show()

print("""
Business Insight

The rating distribution helps understand
overall customer satisfaction.

Higher frequencies of 4 and 5-star ratings
indicate strong customer satisfaction.
""")

# ============================================================
# BUSINESS QUESTION 3
# TOP RATED PRODUCTS
# ============================================================

top_rated_products = (
    review_analysis
    .groupby(
        ["product_id","product_name"],
        as_index=False
    )
    .agg(
        Average_Rating=("rating","mean"),
        Total_Reviews=("review_id","count")
    )
)

top_rated_products = top_rated_products[
    top_rated_products["Total_Reviews"] >= 5
```

```
]

top_rated_products = (
    top_rated_products
    .sort_values(
        ["Average_Rating","Total_Reviews"],
        ascending=[False,False]
    )
    .head(10)
)

print("="*70)
print("TOP RATED PRODUCTS")
print("="*70)

display(top_rated_products)

plt.figure(figsize=(14,6))

plt.bar(
    top_rated_products["product_name"],
    top_rated_products["Average_Rating"]
)

plt.title("Top Rated Products")

plt.xlabel("Product")

plt.ylabel("Average Rating")

plt.xticks(rotation=45,ha="right")

plt.tight_layout()

plt.show()

print("""
Business Insight

Highly rated products should be highlighted
in advertisements and recommendation systems.
""")

# ============================================================
# BUSINESS QUESTION 4
# LOWEST RATED PRODUCTS
# ============================================================

lowest_rated_products = (
    review_analysis
    .groupby(
        ["product_id","product_name"],
        as_index=False
    )
    .agg(
        Average_Rating=("rating","mean"),
        Total_Reviews=("review_id","count")
    )
)
```

```
lowest_rated_products = lowest_rated_products[
    lowest_rated_products["Total_Reviews"] >= 5
]

lowest_rated_products = (
    lowest_rated_products
    .sort_values(
        ["Average_Rating","Total_Reviews"],
        ascending=[True,False]
    )
    .head(10)
)

print("="*70)
print("LOWEST RATED PRODUCTS")
print("="*70)

display(lowest_rated_products)

plt.figure(figsize=(14,6))

plt.bar(
    lowest_rated_products["product_name"],
    lowest_rated_products["Average_Rating"]
)

plt.title("Lowest Rated Products")

plt.xlabel("Product")

plt.ylabel("Average Rating")

plt.xticks(rotation=45,ha="right")

plt.tight_layout()

plt.show()

print("""
Business Insight

Products receiving consistently low ratings
should be investigated for quality issues.
""")

# ============================================================
# BUSINESS QUESTION 5
# CATEGORY-WISE RATINGS
# ============================================================

category_ratings = (
    review_analysis
    .groupby(
        "category",
        as_index=False
    )
    .agg(
        Average_Rating=("rating","mean"),
        Total_Reviews=("review_id","count")
    )
```

```
    .sort_values(
        "Average_Rating",
        ascending=False
    )
)

print("="*70)
print("CATEGORY-WISE RATINGS")
print("="*70)

display(category_ratings)

plt.figure(figsize=(10,6))

plt.bar(
    category_ratings["category"],
    category_ratings["Average_Rating"]
)

plt.title("Average Rating by Category")

plt.xlabel("Category")

plt.ylabel("Average Rating")

plt.xticks(rotation=45)

plt.tight_layout()

plt.show()

print("""
Business Insight

Categories with consistently high ratings
can be used to strengthen the company's brand image.

Low-rated categories should undergo
quality improvement initiatives.
""")

# ============================================================
# REVIEW ANALYSIS SUMMARY
# ============================================================

print("""
============================================================
REVIEW ANALYSIS SUMMARY
============================================================

Key Findings

✓ Overall Customer Rating

✓ Rating Distribution

✓ Top Rated Products

✓ Lowest Rated Products
```

```
✓ Category-wise Rating Performance

Business Recommendations

1. Promote highly rated products.

2. Improve products with poor ratings.

3. Collect customer feedback regularly.

4. Monitor product quality continuously.

5. Reward suppliers producing highly rated products.

============================================================
END OF REVIEW ANALYSIS
============================================================
""")
```

```
======================================================================
REVIEW DATASET
======================================================================
Shape : (1106, 10)
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>review_id</th> <th>product_id</th> <th>customer_id</th> <th>rating</th> <th>review_text</th> <th>review_date</th> <th>product_name</th> <th>category</th> <th>price</th> <th>supplier_id</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody> </tbody>

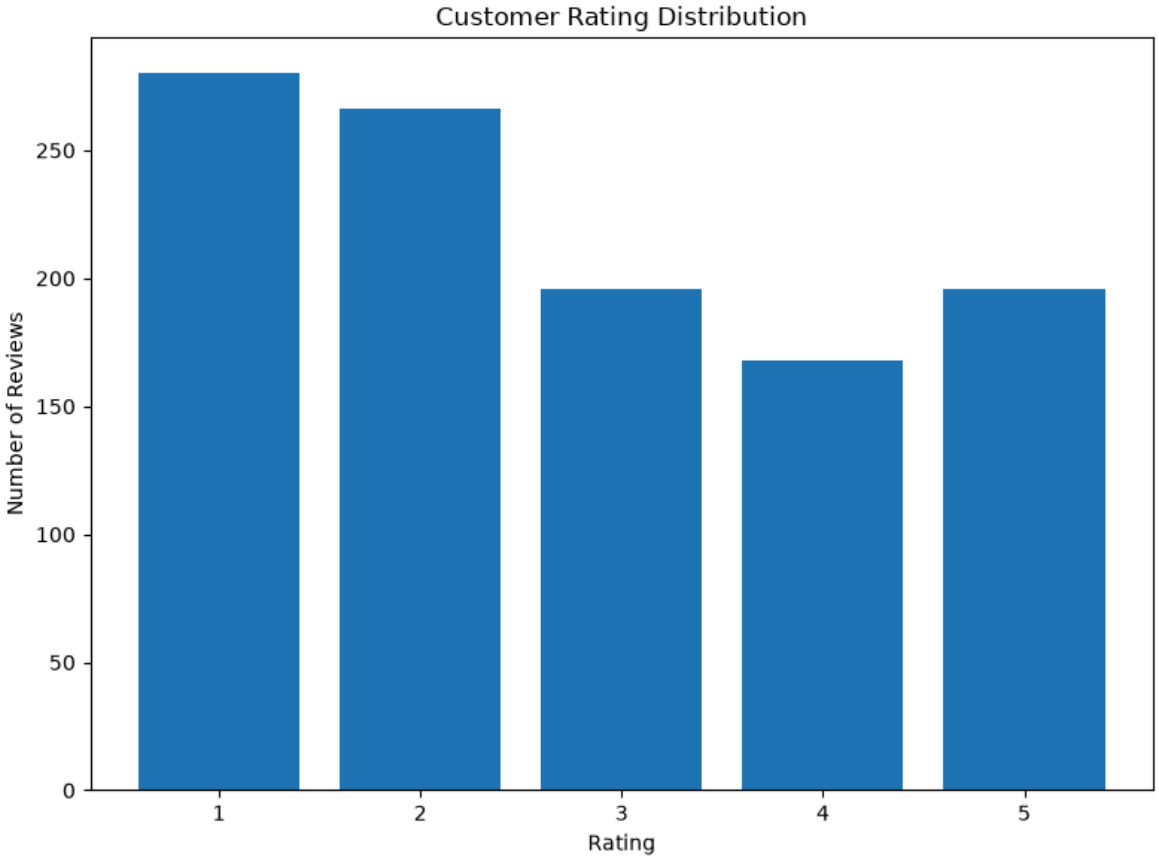
1	16	1516	4	Customer service was excellent, product too.	2024-03-03	Mechanical Keyboard	Electronics	931.97	516
2	1516	1516	4	Customer service was excellent, product too.	2024-01-08	Mechanical Keyboard	Electronics	467.94	516
3	1566	9066	1	Basic functionality but reliable.	2024-03-23	Mechanical Keyboard	Electronics	620.61	566
4	66	9066	1	Basic functionality but reliable.	2024-03-25	Mechanical Keyboard	Electronics	620.87	566
5	1522	1522	3	Better than similar products I have tried.	2024-03-03	Microphone	Electronics	0.20	522

```
Dataset Information

<class 'pandas.DataFrame'>
RangeIndex: 1106 entries, 0 to 1105
Data columns (total 10 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   review_id     1106 non-null   int64
 1   product_id    1106 non-null   int64
 2   customer_id   1106 non-null   int64
 3   rating        1106 non-null   int64
 4   review_text   1106 non-null   str
 5   review_date   1106 non-null   datetime64[us]
 6   product_name  1106 non-null   str
 7   category      1106 non-null   str
 8   price         1106 non-null   float64
 9   supplier_id   1106 non-null   int64
dtypes: datetime64[us](1), float64(1), int64(5), str(3)
memory usage: 86.5 KB
======================================================================
OVERALL CUSTOMER RATING
======================================================================
Average Rating : 2.76 / 5
======================================================================
RATING DISTRIBUTION
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>rating</th> <th>Total_Reviews</th> <th>Percentage</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

1	280	25.32
2	266	24.05
3	196	17.72
4	168	15.19
5	196	17.72




```
Business Insight

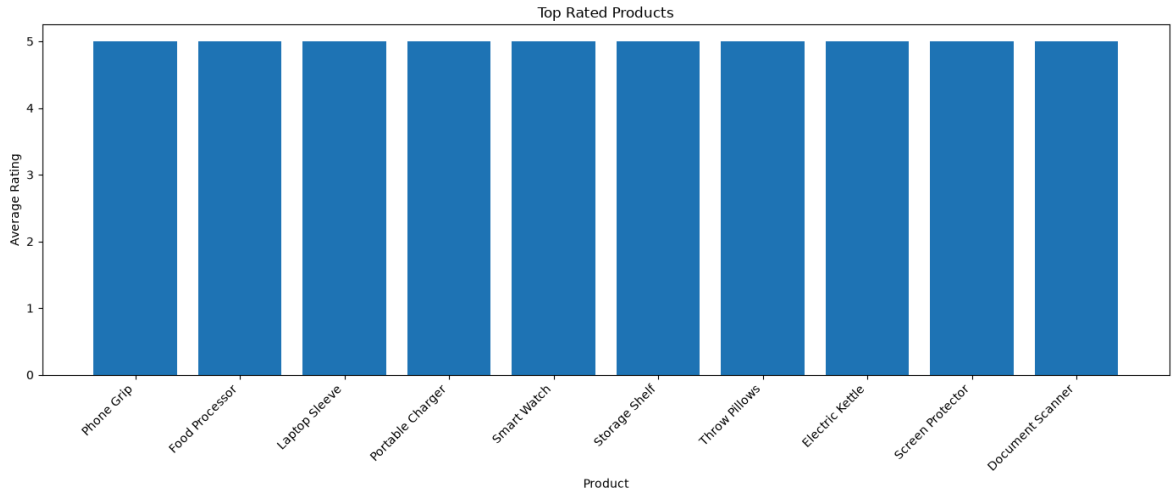
The rating distribution helps understand
overall customer satisfaction.

Higher frequencies of 4 and 5-star ratings
indicate strong customer satisfaction.

======================================================================
TOP RATED PRODUCTS
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>product_id</th> <th>product_name</th> <th>Average_Rating</th> <th>Total_Reviews</th> </thead> <tbody> <th>12</th> <th>16</th> <th>18</th> <th>23</th> <th>26</th> <th>31</th> <th>36</th> <th>41</th> <th>43</th> <th>52</th> </tbody> <tbody></tbody>

13	Phone Grip	5.0	7
17	Food Processor	5.0	7
19	Laptop Sleeve	5.0	7
24	Portable Charger	5.0	7
27	Smart Watch	5.0	7
32	Storage Shelf	5.0	7
37	Throw Pillows	5.0	7
42	Electric Kettle	5.0	7
44	Screen Protector	5.0	7
53	Document Scanner	5.0	7



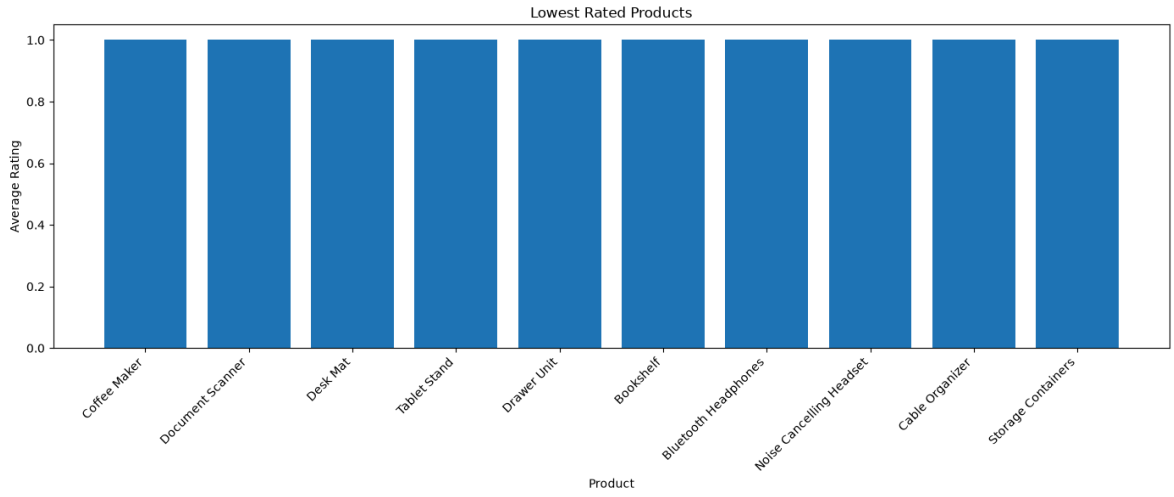

```
Business Insight

Highly rated products should be highlighted
in advertisements and recommendation systems.

======================================================================
LOWEST RATED PRODUCTS
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>product_id</th> <th>product_name</th> <th>Average_Rating</th> <th>Total_Reviews</th> </thead> <tbody> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>22</th> <th>24</th> <th>25</th> <th>27</th> <th>28</th> </tbody> <tbody></tbody>

2	Coffee Maker	1.0	7
3	Document Scanner	1.0	7
4	Desk Mat	1.0	7
5	Tablet Stand	1.0	7
6	Drawer Unit	1.0	7
23	Bookshelf	1.0	7
25	Bluetooth Headphones	1.0	7
26	Noise Cancelling Headset	1.0	7
28	Cable Organizer	1.0	7
29	Storage Containers	1.0	7



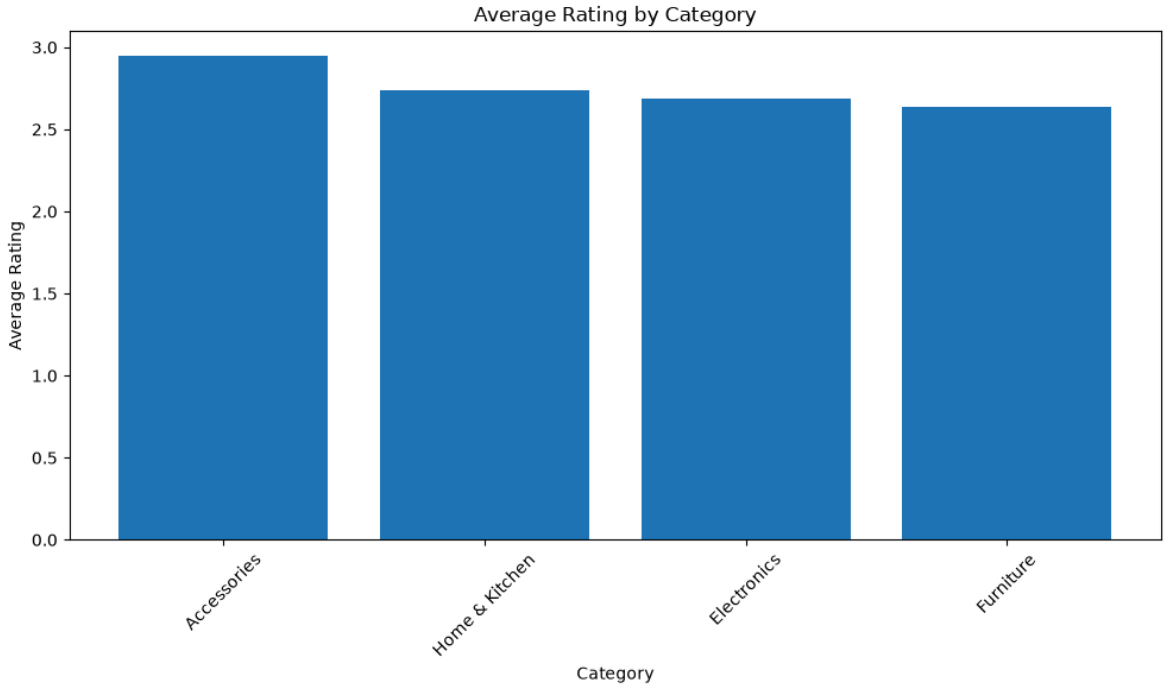

```
Business Insight

Products receiving consistently low ratings
should be investigated for quality issues.

======================================================================
CATEGORY-WISE RATINGS
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>category</th> <th>Average_Rating</th> <th>Total_Reviews</th> </thead> <tbody> <th>0</th> <th>3</th> <th>1</th> <th>2</th> </tbody> <tbody> </tbody>

Accessories	2.950000	280
Home & Kitchen	2.736842	266
Electronics	2.689655	406
Furniture	2.636364	154



```
Business Insight

Categories with consistently high ratings
can be used to strengthen the company's brand image.

Low-rated categories should undergo
quality improvement initiatives.


============================================================
REVIEW ANALYSIS SUMMARY
============================================================

Key Findings

✓ Overall Customer Rating

✓ Rating Distribution

✓ Top Rated Products

✓ Lowest Rated Products

✓ Category-wise Rating Performance

Business Recommendations

1. Promote highly rated products.

2. Improve products with poor ratings.

3. Collect customer feedback regularly.

4. Monitor product quality continuously.

5. Reward suppliers producing highly rated products.

============================================================
END OF REVIEW ANALYSIS
============================================================
```

In [58]:
```
# ============================================================
# SUPPLIER ANALYSIS
# ============================================================
#
# Business Questions
#
# 1. Which suppliers generate the highest revenue?
# 2. Which suppliers provide the most products?
# 3. Which suppliers sell the highest quantity?
# 4. Which suppliers have the highest average product price?
# 5. Which suppliers receive the best customer ratings?
#
# Business Objective
#
# • Identify high-performing suppliers
# • Strengthen supplier relationships
# • Optimize procurement decisions
# • Improve product quality
```

```
#
# ============================================================

import pandas as pd
import matplotlib.pyplot as plt

plt.rcParams["figure.figsize"] = (12,6)

# ============================================================
# STEP 1 : CREATE SUPPLIER DATASET
# ============================================================

supplier_analysis = (
    order_items
    .merge(products, on="product_id", how="left")
    .merge(suppliers, on="supplier_id", how="left")
)

supplier_analysis["Revenue"] = (
    supplier_analysis["quantity"] *
    supplier_analysis["price_at_purchase"]
)

print("="*70)
print("SUPPLIER ANALYSIS DATASET")
print("="*70)

print("Shape :", supplier_analysis.shape)

display(supplier_analysis.head())

# ============================================================
# BUSINESS QUESTION 1
# TOP SUPPLIERS BY REVENUE
# ============================================================

supplier_revenue = (
    supplier_analysis
    .groupby(
        "supplier_name",
        as_index=False
    )
    .agg(
        Total_Revenue=("Revenue","sum"),
        Total_Quantity=("quantity","sum")
    )
    .sort_values(
        "Total_Revenue",
        ascending=False
    )
)

print("="*70)
print("TOP SUPPLIERS BY REVENUE")
print("="*70)

display(supplier_revenue.head(10))

plt.figure(figsize=(14,6))
```

```
plt.bar(
    supplier_revenue.head(10)["supplier_name"],
    supplier_revenue.head(10)["Total_Revenue"]
)

plt.title("Top 10 Suppliers by Revenue")

plt.xlabel("Supplier")

plt.ylabel("Revenue ($)")

plt.xticks(rotation=45,ha="right")

plt.tight_layout()

plt.show()

print("""
Business Insight

Suppliers generating the highest revenue
should be considered strategic partners.
""")

# ============================================================
# BUSINESS QUESTION 2
# PRODUCTS PROVIDED BY EACH SUPPLIER
# ============================================================

supplier_products = (
    supplier_analysis
    .groupby(
        "supplier_name",
        as_index=False
    )
    .agg(
        Total_Products=("product_id","nunique")
    )
    .sort_values(
        "Total_Products",
        ascending=False
    )
)

print("="*70)
print("PRODUCTS PROVIDED BY SUPPLIERS")
print("="*70)

display(supplier_products)

plt.figure(figsize=(12,6))

plt.bar(
    supplier_products["supplier_name"],
    supplier_products["Total_Products"]
)

plt.title("Products Supplied")

plt.xlabel("Supplier")
```

```
plt.ylabel("Number of Products")

plt.xticks(rotation=45)

plt.tight_layout()

plt.show()

# ============================================================
# BUSINESS QUESTION 3
# TOTAL QUANTITY SOLD BY SUPPLIER
# ============================================================

supplier_quantity = (
    supplier_analysis
    .groupby(
        "supplier_name",
        as_index=False
    )
    .agg(
        Quantity_Sold=("quantity","sum")
    )
    .sort_values(
        "Quantity_Sold",
        ascending=False
    )
)

print("="*70)
print("QUANTITY SOLD BY SUPPLIER")
print("="*70)

display(supplier_quantity)

plt.figure(figsize=(12,6))

plt.bar(
    supplier_quantity["supplier_name"],
    supplier_quantity["Quantity_Sold"]
)

plt.title("Quantity Sold by Supplier")

plt.xlabel("Supplier")

plt.ylabel("Units Sold")

plt.xticks(rotation=45)

plt.tight_layout()

plt.show()

# ============================================================
# BUSINESS QUESTION 4
# AVERAGE PRODUCT PRICE
# ============================================================

supplier_price = (
```

```
    supplier_analysis
    .groupby(
        "supplier_name",
        as_index=False
    )
    .agg(
        Average_Product_Price=("price","mean")
    )
    .sort_values(
        "Average_Product_Price",
        ascending=False
    )
)

print("="*70)
print("AVERAGE PRODUCT PRICE")
print("="*70)

display(supplier_price)

plt.figure(figsize=(12,6))

plt.bar(
    supplier_price["supplier_name"],
    supplier_price["Average_Product_Price"]
)

plt.title("Average Product Price by Supplier")

plt.xlabel("Supplier")

plt.ylabel("Average Price ($)")

plt.xticks(rotation=45)

plt.tight_layout()

plt.show()

# ============================================================
# BUSINESS QUESTION 5
# SUPPLIER RATING
# ============================================================

supplier_rating = (
    reviews
    .merge(products,on="product_id")
    .merge(suppliers,on="supplier_id")
)

supplier_rating = (
    supplier_rating
    .groupby(
        "supplier_name",
        as_index=False
    )
    .agg(
        Average_Rating=("rating","mean"),
        Total_Reviews=("review_id","count")
    )
```

```
    .sort_values(
        "Average_Rating",
        ascending=False
    )
)

print("="*70)
print("SUPPLIER RATINGS")
print("="*70)

display(supplier_rating)

plt.figure(figsize=(12,6))

plt.bar(
    supplier_rating["supplier_name"],
    supplier_rating["Average_Rating"]
)

plt.title("Average Rating by Supplier")

plt.xlabel("Supplier")

plt.ylabel("Average Rating")

plt.xticks(rotation=45)

plt.tight_layout()

plt.show()

print("""
Business Insight

Highly rated suppliers consistently deliver
better-quality products and should be preferred
for future procurement.
""")

# ============================================================
# SUPPLIER ANALYSIS SUMMARY
# ============================================================

print("""
================================================================
SUPPLIER ANALYSIS SUMMARY
================================================================

Key Findings

✓ Highest Revenue Suppliers

✓ Suppliers with Most Products

✓ Highest Sales Volume Suppliers

✓ Highest Average Product Price

✓ Best Rated Suppliers
```

```
Business Recommendations

1. Build long-term partnerships with top suppliers.

2. Improve collaboration with low-performing suppliers.

3. Negotiate pricing with premium suppliers.

4. Monitor supplier quality using customer ratings.

5. Review supplier performance regularly.

============================================================
END OF SUPPLIER ANALYSIS
============================================================
""")
```

```
======================================================================
SUPPLIER ANALYSIS DATASET
======================================================================
Shape : (20000, 15)
```

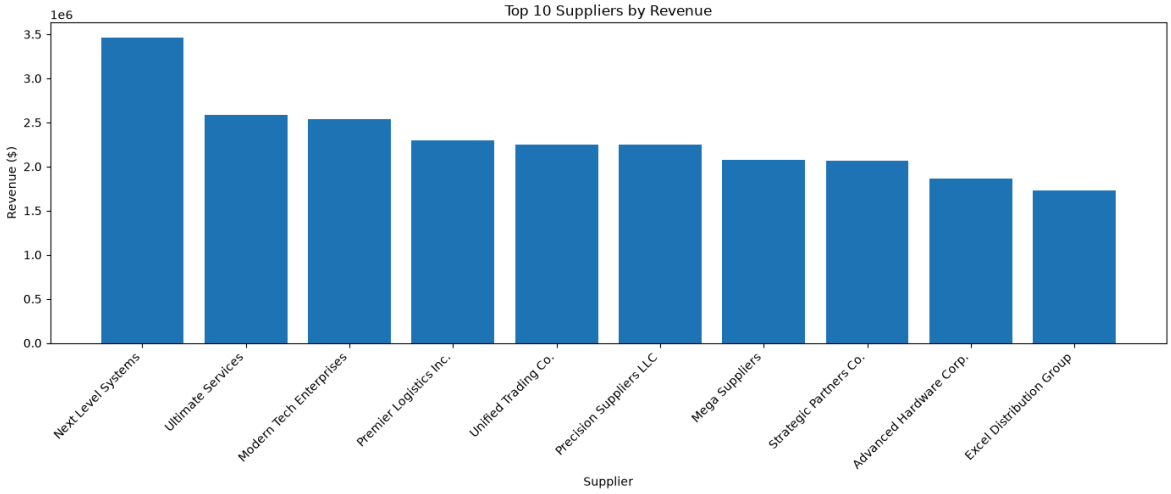
<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>order_item_id</th> <th>order_id</th> <th>product_id</th> <th>quantity</th> <th>price_at_purchase</th> <th>product_name</th> <th>category</th> <th>price</th> <th>supplier_id</th> <th>supplier_name</th> <th>contact_name</th> <th>address</th> <th>phone_number</th> <th>email</th> <th>Revenue</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

1	6550	1032	1	342.92	Storage Shelf	Furniture	680.43	532	Alpha Industries Ltd.	Linda Thomas	92 Harbo Rd, Ba View, Al
2	7324	1695	1	955.86	Desk Lamp	Home & Kitchen	940.82	595	Precision Suppliers LLC	Ryan Arthur	363 Ba St Lakeside IA
3	11952	1962	1	909.45	Rice Cooker	Home & Kitchen	542.94	562	Alpha Industries Ltd.	Barbara Maria	11 Redwoo Ave, Ba Harbo M
4	8938	1958	1	984.91	External SSD	Electronics	180.06	558	Modern Tech Enterprises	Stephen Philip	12 Fores Ln Emeral Height W
5	4238	1880	1	649.18	Kitchen Blender	Home & Kitchen	856.78	580	Summit Solutions Corp.	William Frank	92 Harbo Rd, Ba View, Al

```
======================================================================
TOP SUPPLIERS BY REVENUE
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>supplier_name</th> <th>Total_Revenue</th> <th>Total_Quantity</th> </thead> <tbody> <th>13</th> <th>23</th> <th>12</th> <th>15</th> <th>24</th> <th>14</th> <th>11</th> <th>20</th> <th>1</th> <th>7</th> </tbody> <tbody> </tbody>

Next Level Systems	3458937.63	7531
Ultimate Services	2584189.72	5756
Modern Tech Enterprises	2538935.59	5565
Premier Logistics Inc.	2293785.70	4943
Unified Trading Co.	2253219.08	4908
Precision Suppliers LLC	2252561.35	4896
Mega Suppliers	2075749.35	4650
Strategic Partners Co.	2063347.05	4608
Advanced Hardware Corp.	1860465.68	4081
Excel Distribution Group	1727347.25	3813




```
Business Insight

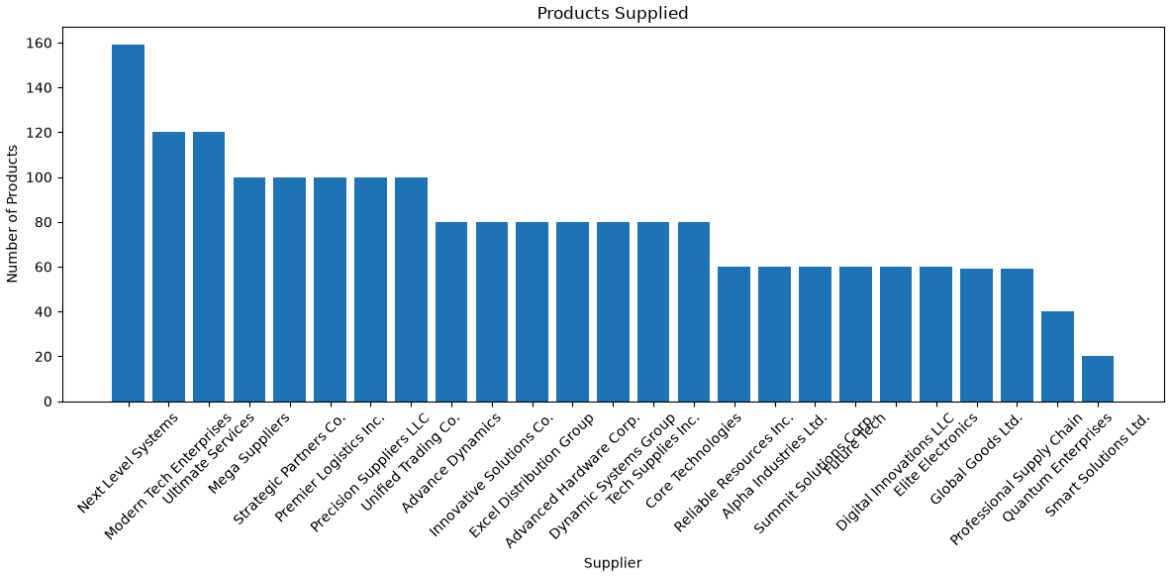
Suppliers generating the highest revenue
should be considered strategic partners.

======================================================================
PRODUCTS PROVIDED BY SUPPLIERS
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>supplier_name</th> <th>Total_Products</th> </thead> <tbody> <th>13</th> <th>12</th> <th>23</th> <th>11</th> <th>20</th> <th>...</th> <th>6</th> <th>9</th> <th>16</th> <th>17</th> <th>19</th> </tbody> <tbody> </tbody>

Next Level Systems	159
Modern Tech Enterprises	120
Ultimate Services	120
Mega Suppliers	100
Strategic Partners Co.	100
...	...
Elite Electronics	60
Global Goods Ltd.	59
Professional Supply Chain	59
Quantum Enterprises	40
Smart Solutions Ltd.	20

25 rows × 2 columns

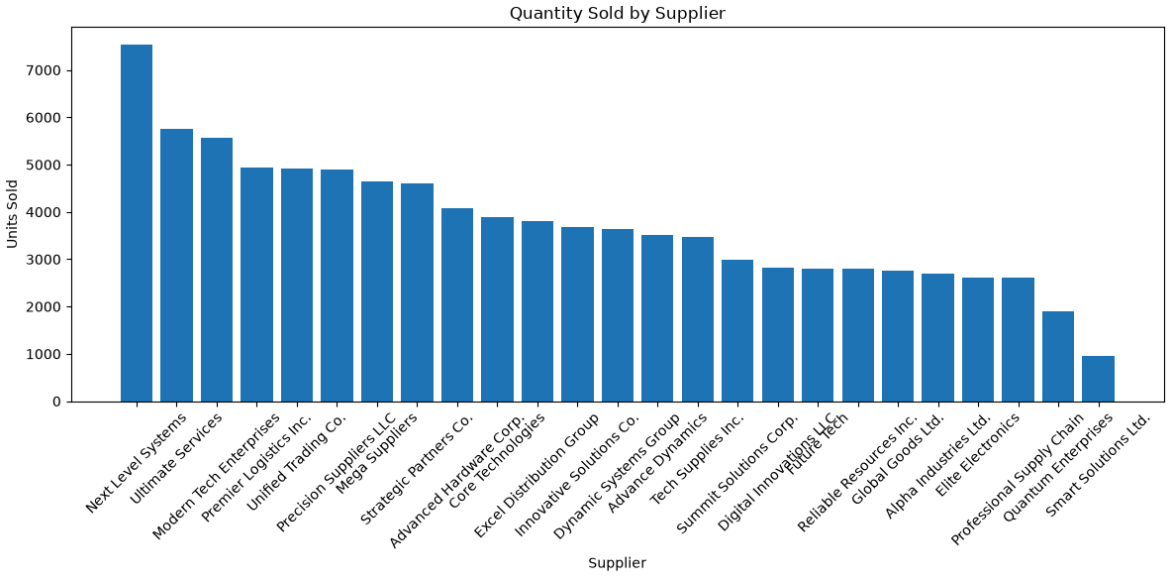



```
======================================================================
QUANTITY SOLD BY SUPPLIER
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>supplier_name</th> <th>Quantity_Sold</th> </thead> <tbody> <th>13</th> <th>23</th> <th>12</th> <th>15</th> <th>24</th> <th>...</th> <th>2</th> <th>6</th> <th>16</th> <th>17</th> <th>19</th> </tbody> <tbody> </tbody>

Next Level Systems	7531
Ultimate Services	5756
Modern Tech Enterprises	5565
Premier Logistics Inc.	4943
Unified Trading Co.	4908
...	...
Alpha Industries Ltd.	2696
Elite Electronics	2621
Professional Supply Chain	2614
Quantum Enterprises	1891
Smart Solutions Ltd.	956

25 rows × 2 columns

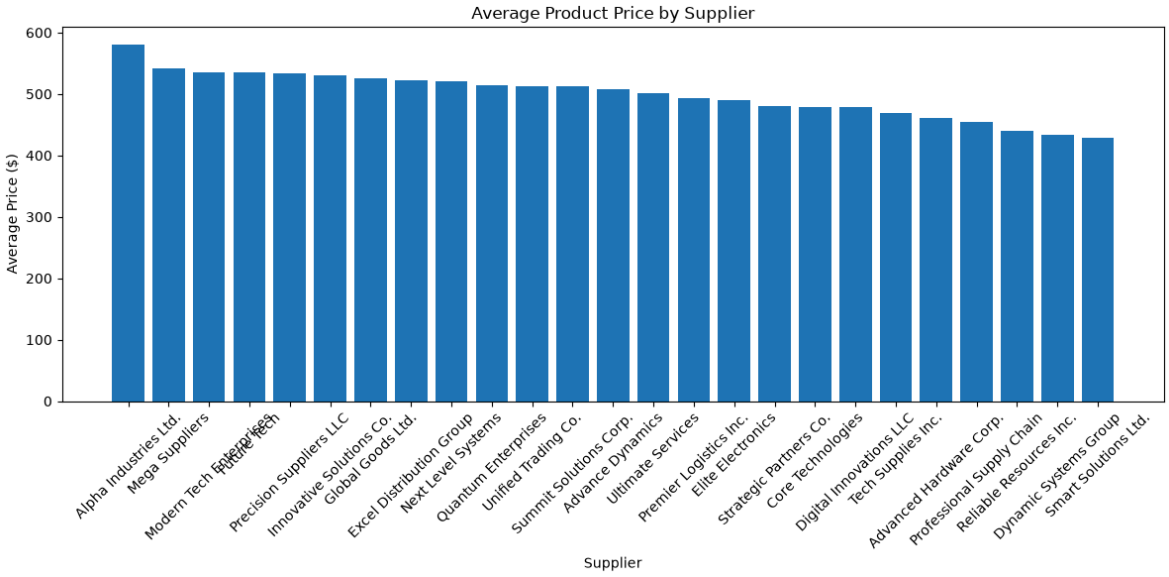



```
======================================================================
AVERAGE PRODUCT PRICE
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>supplier_name</th> <th>Average_Product_Price</th> </thead> <tbody> <th>2</th> <th>11</th> <th>12</th> <th>8</th> <th>14</th> <th>...</th> <th>1</th> <th>16</th> <th>18</th> <th>5</th> <th>19</th> </tbody> <tbody> </tbody>

Alpha Industries Ltd.	579.923339
Mega Suppliers	541.102249
Modern Tech Enterprises	535.440595
Future Tech	534.604000
Precision Suppliers LLC	533.747495
...	...
Advanced Hardware Corp.	461.332625
Professional Supply Chain	454.361060
Reliable Resources Inc.	440.784393
Dynamic Systems Group	432.753016
Smart Solutions Ltd.	428.718911

25 rows × 2 columns

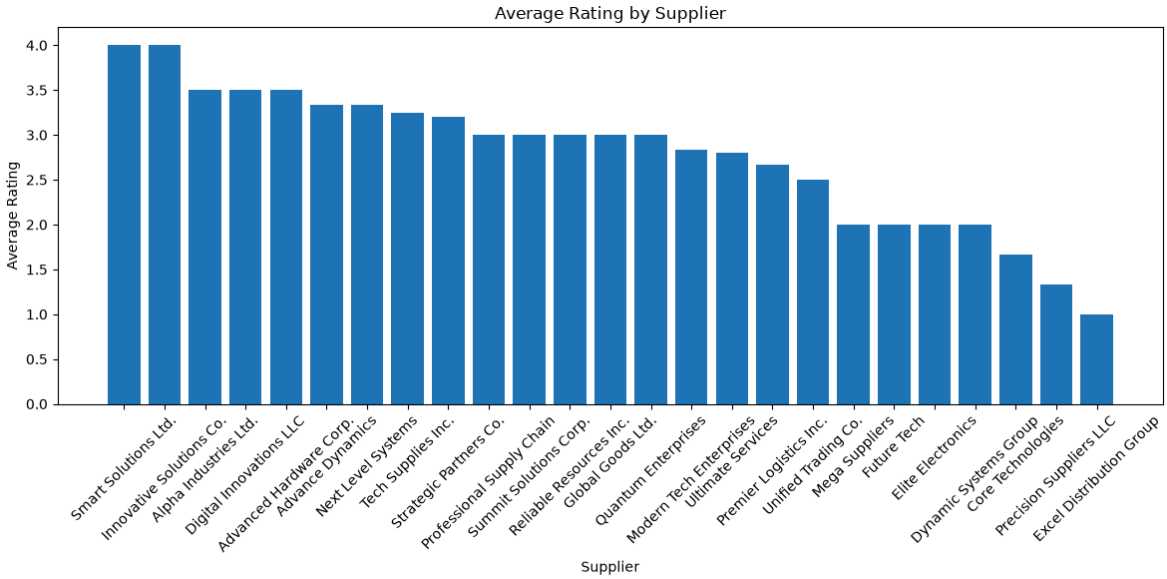



```
======================================================================
SUPPLIER RATINGS
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>supplier_name</th> <th>Average_Rating</th> <th>Total_Reviews</th> </thead> <tbody> <th>19</th> <th>10</th> <th>2</th> <th>4</th> <th>1</th> <th>...</th> <th>6</th> <th>5</th> <th>3</th> <th>14</th> <th>7</th> </tbody> <tbody></tbody>

Smart Solutions Ltd.	4.000000	14
Innovative Solutions Co.	4.000000	56
Alpha Industries Ltd.	3.500000	28
Digital Innovations LLC	3.500000	28
Advanced Hardware Corp.	3.500000	56
...	...	...
Elite Electronics	2.000000	42
Dynamic Systems Group	2.000000	42
Core Technologies	1.666667	42
Precision Suppliers LLC	1.333333	42
Excel Distribution Group	1.000000	42

25 rows × 3 columns



```
Business Insight

Highly rated suppliers consistently deliver
better-quality products and should be preferred
for future procurement.


============================================================
SUPPLIER ANALYSIS SUMMARY
============================================================

Key Findings

✓ Highest Revenue Suppliers

✓ Suppliers with Most Products

✓ Highest Sales Volume Suppliers

✓ Highest Average Product Price

✓ Best Rated Suppliers

Business Recommendations

1. Build long-term partnerships with top suppliers.

2. Improve collaboration with low-performing suppliers.

3. Negotiate pricing with premium suppliers.

4. Monitor supplier quality using customer ratings.

5. Review supplier performance regularly.

============================================================
END OF SUPPLIER ANALYSIS
============================================================
```

In [59]:

```
# ============================================================
# CORRELATION ANALYSIS
# ============================================================
#
# Business Questions
#
# 1. Which variables are highly correlated?
# 2. Does product price influence quantity sold?
# 3. Does order value increase with quantity?
# 4. Which variables have the strongest relationship?
# 5. What insights can correlation provide?
#
# Business Objective
#
# • Understand relationships between business metrics
# • Discover important drivers of revenue
# • Support business decision making
#
# ============================================================
```

```
import pandas as pd
import matplotlib.pyplot as plt

plt.rcParams["figure.figsize"] = (10,8)

# ============================================================
# STEP 1 : CREATE ANALYSIS DATASET
# ============================================================

correlation_df = (
    order_items
    .merge(products,on="product_id",how="left")
)

correlation_df["Revenue"] = (
    correlation_df["quantity"] *
    correlation_df["price_at_purchase"]
)

print("="*70)
print("CORRELATION DATASET")
print("="*70)

display(correlation_df.head())

# ============================================================
# STEP 2 : SELECT NUMERICAL VARIABLES
# ============================================================

numeric_df = correlation_df[[
    "quantity",
    "price",
    "price_at_purchase",
    "Revenue"
]]

print("="*70)
print("NUMERICAL VARIABLES")
print("="*70)

display(numeric_df.head())

# ============================================================
# STEP 3 : CORRELATION MATRIX
# ============================================================

corr_matrix = numeric_df.corr()

print("="*70)
print("CORRELATION MATRIX")
print("="*70)

display(corr_matrix)

# ============================================================
# STEP 4 : HEATMAP
# ============================================================

plt.figure(figsize=(8,6))
```

```
plt.imshow(corr_matrix, cmap="coolwarm")

plt.colorbar()

plt.xticks(
    range(len(corr_matrix.columns)),
    corr_matrix.columns,
    rotation=45
)

plt.yticks(
    range(len(corr_matrix.columns)),
    corr_matrix.columns
)

for i in range(len(corr_matrix.columns)):
    for j in range(len(corr_matrix.columns)):
        plt.text(
            j,
            i,
            round(corr_matrix.iloc[i,j],2),
            ha="center",
            va="center",
            color="black"
        )

plt.title("Correlation Heatmap")

plt.tight_layout()

plt.show()

# ============================================================
# BUSINESS QUESTION 1
# PRICE VS QUANTITY
# ============================================================

plt.figure(figsize=(8,6))

plt.scatter(
    correlation_df["price"],
    correlation_df["quantity"],
    alpha=0.5
)

plt.title("Price vs Quantity Sold")

plt.xlabel("Product Price")

plt.ylabel("Quantity Sold")

plt.tight_layout()

plt.show()

print("""
Business Insight

This visualization helps determine whether
higher-priced products sell fewer units.
```

```
A negative relationship may indicate
price-sensitive customer behavior.
""")

# ============================================================
# BUSINESS QUESTION 2
# QUANTITY VS REVENUE
# ============================================================

plt.figure(figsize=(8,6))

plt.scatter(
    correlation_df["quantity"],
    correlation_df["Revenue"],
    alpha=0.5
)

plt.title("Quantity vs Revenue")

plt.xlabel("Quantity")

plt.ylabel("Revenue")

plt.tight_layout()

plt.show()

print("""
Business Insight

Higher quantities generally increase revenue,
although pricing also plays an important role.
""")

# ============================================================
# BUSINESS QUESTION 3
# PRICE VS REVENUE
# ============================================================

plt.figure(figsize=(8,6))

plt.scatter(
    correlation_df["price"],
    correlation_df["Revenue"],
    alpha=0.5
)

plt.title("Price vs Revenue")

plt.xlabel("Price")

plt.ylabel("Revenue")

plt.tight_layout()

plt.show()

print("""
Business Insight
```

```
Premium-priced products can generate
high revenue even with moderate sales volume.
""")

# ============================================================
# CORRELATION SUMMARY
# ============================================================

print("""
================================================================
CORRELATION ANALYSIS SUMMARY
================================================================

Key Findings

✓ Correlation Matrix Generated

✓ Numerical Relationships Evaluated

✓ Price vs Quantity Analyzed

✓ Quantity vs Revenue Analyzed

✓ Price vs Revenue Analyzed

Business Recommendations

1. Monitor products with strong price sensitivity.

2. Focus on products generating high revenue.

3. Balance pricing and sales volume.

4. Use correlation insights for pricing strategies.

================================================================
END OF CORRELATION ANALYSIS
================================================================
""")
```

```
================================================================================
CORRELATION DATASET
================================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>order_item_id</th> <th>order_id</th> <th>product_id</th> <th>quantity</th> <th>price_at_purchase</th> <th>product_name</th> <th>category</th> <th>price</th> <th>supplier_id</th> <th>Revenue</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

1	6550	1032	1	342.92	Storage Shelf	Furniture	680.43	532	342.92
2	7324	1695	1	955.86	Desk Lamp	Home & Kitchen	940.82	595	955.86
3	11952	1962	1	909.45	Rice Cooker	Home & Kitchen	542.94	562	909.45
4	8938	1958	1	984.91	External SSD	Electronics	180.06	558	984.91
5	4238	1880	1	649.18	Kitchen Blender	Home & Kitchen	856.78	580	649.18

```
======================================================================
NUMERICAL VARIABLES
======================================================================
```

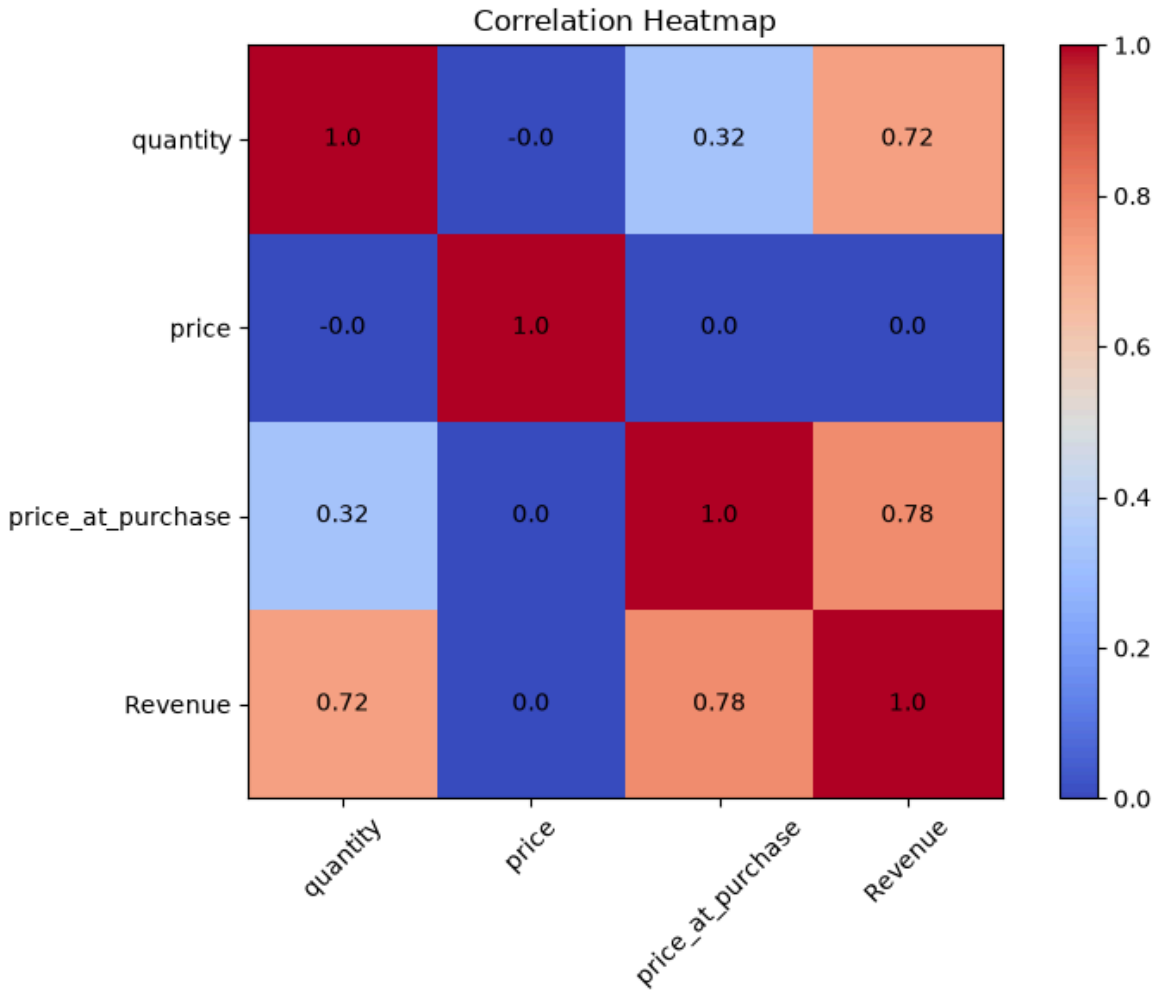
<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>quantity</th> <th>price</th> <th>price_at_purchase</th> <th>Revenue</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

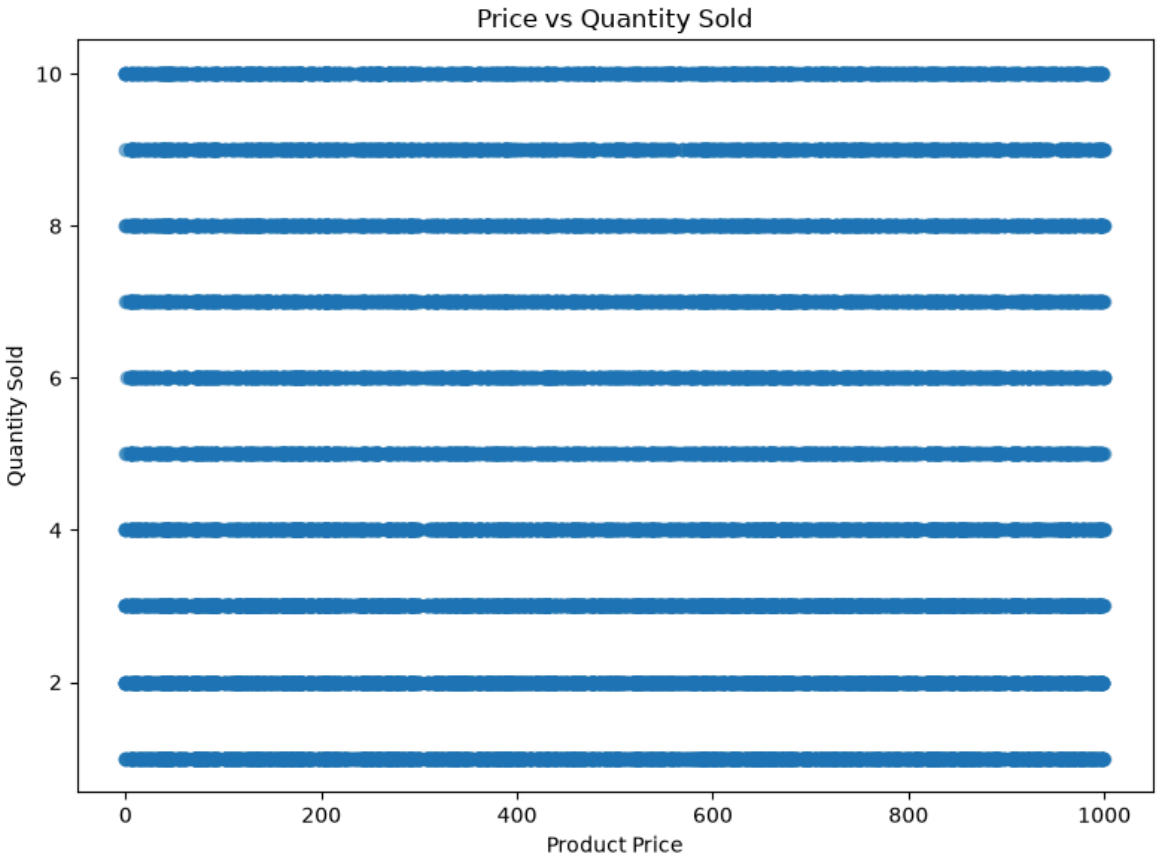
1	680.43	342.92	342.92	
1	940.82	955.86	955.86	
1	542.94	909.45	909.45	
1	180.06	984.91	984.91	
1	856.78	649.18	649.18	

```
======================================================================
CORRELATION MATRIX
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>quantity</th> <th>price</th> <th>price_at_purchase</th> <th>Revenue</th> </thead> <tbody> <th>quantity</th> <th>price</th> <th>price_at_purchase</th> <th>Revenue</th> </tbody> <tbody></tbody>

1.000000	-0.000085	0.324979	0.723404
-0.000085	1.000000	0.002968	0.000746
0.324979	0.002968	1.000000	0.778920
0.723404	0.000746	0.778920	1.000000

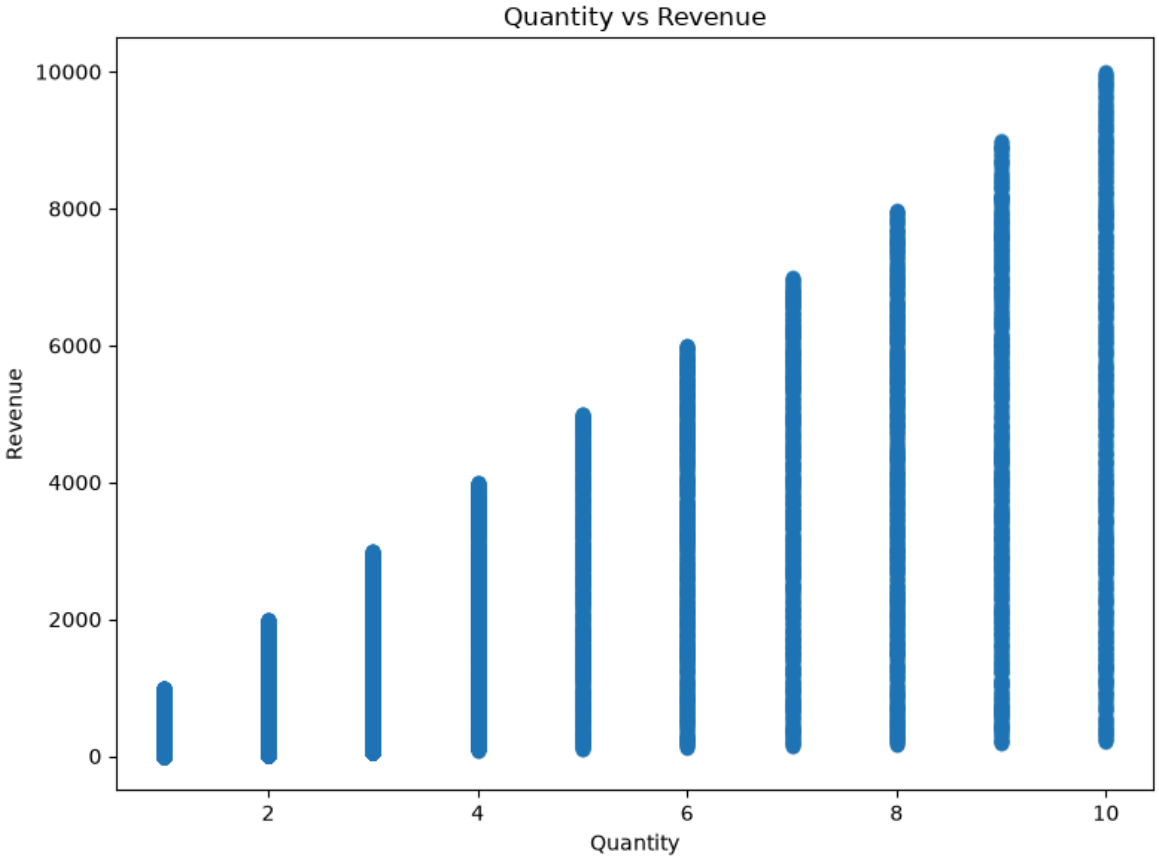

Correlation Heatmap
quantity
price
price_at_purchase
Revenue
1.0
-0.0
0.32
0.72
-0.0
1.0
0.0
0.0
0.32
0.0
1.0
0.78
0.72
0.0
0.78
1.0
quantity
price
price_at_purchase
Revenue
1.0
0.8
0.6
0.4
0.2
0.0


Price vs Quantity Sold
Quantity Sold
10
8
6
4
2
0
200
400
600
800
1000
Product Price

```
Business Insight

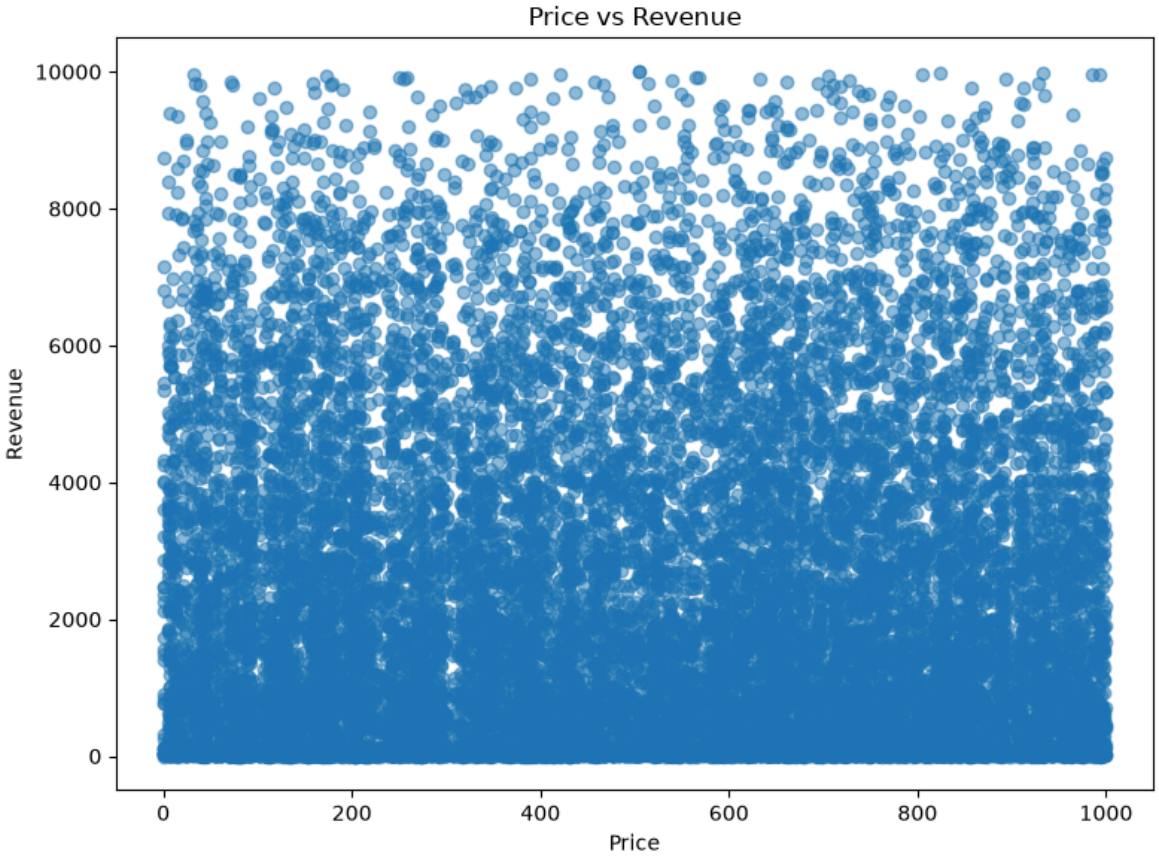
This visualization helps determine whether
higher-priced products sell fewer units.

A negative relationship may indicate
price-sensitive customer behavior.
```




```
Business Insight

Higher quantities generally increase revenue,
although pricing also plays an important role.
```


Price vs Revenue
Revenue
Price
10000
8000
6000
4000
2000
0
0
200
400
600
800
1000

```
Business Insight

Premium-priced products can generate
high revenue even with moderate sales volume.


============================================================
CORRELATION ANALYSIS SUMMARY
============================================================

Key Findings

✓ Correlation Matrix Generated

✓ Numerical Relationships Evaluated

✓ Price vs Quantity Analyzed

✓ Quantity vs Revenue Analyzed

✓ Price vs Revenue Analyzed

Business Recommendations

1. Monitor products with strong price sensitivity.

2. Focus on products generating high revenue.

3. Balance pricing and sales volume.

4. Use correlation insights for pricing strategies.

============================================================
END OF CORRELATION ANALYSIS
============================================================
```

In [60]:

```
# ============================================================
# RFM CUSTOMER SEGMENTATION
# ============================================================
#
# Business Questions
#
# 1. Who are our most valuable customers?
# 2. Which customers purchase frequently?
# 3. Which customers spend the most money?
# 4. Which customers are at risk of churn?
# 5. How should customers be segmented for marketing?
#
# Business Objective
#
# • Identify VIP customers
# • Improve customer retention
# • Increase repeat purchases
# • Design targeted marketing campaigns
#
# ============================================================

import pandas as pd
import matplotlib.pyplot as plt
```

```
plt.rcParams["figure.figsize"] = (12,6)

# ============================================================
# STEP 1 : PREPARE DATA
# ============================================================

rfm_df = orders.copy()

latest_date = rfm_df["order_date"].max()

# ============================================================
# STEP 2 : CREATE RFM TABLE
# ============================================================

rfm = (
    rfm_df
    .groupby("customer_id")
    .agg(
        Recency=("order_date",
                 lambda x: (latest_date - x.max()).days),

        Frequency=("order_id","count"),

        Monetary=("total_price","sum")
    )
    .reset_index()
)

print("="*70)
print("RFM TABLE")
print("="*70)

display(rfm.head())

# ============================================================
# STEP 3 : CREATE RFM SCORES
# ============================================================

rfm["R_Score"] = pd.qcut(
    rfm["Recency"],
    5,
    labels=[5,4,3,2,1]
)

rfm["F_Score"] = pd.qcut(
    rfm["Frequency"].rank(method="first"),
    5,
    labels=[1,2,3,4,5]
)

rfm["M_Score"] = pd.qcut(
    rfm["Monetary"],
    5,
    labels=[1,2,3,4,5]
)

rfm["RFM_Score"] = (
    rfm["R_Score"].astype(str)+
    rfm["F_Score"].astype(str)+
```

```
    rfm["M_Score"].astype(str)
)

display(rfm.head())

# ============================================================
# STEP 4 : CUSTOMER SEGMENTATION
# ============================================================

def customer_segment(row):

    r = int(row["R_Score"])
    f = int(row["F_Score"])
    m = int(row["M_Score"])

    if r >=4 and f>=4 and m>=4:
        return "Champions"

    elif r>=3 and f>=4:
        return "Loyal Customers"

    elif r>=4 and m>=4:
        return "Big Spenders"

    elif r>=4:
        return "Recent Customers"

    elif r<=2 and f>=4:
        return "At Risk"

    elif r<=2 and f<=2:
        return "Lost Customers"

    else:
        return "Potential Loyalists"

rfm["Segment"] = rfm.apply(customer_segment,axis=1)

print("="*70)
print("CUSTOMER SEGMENTS")
print("="*70)

display(rfm.head())

# ============================================================
# BUSINESS QUESTION 1
# CUSTOMER SEGMENT DISTRIBUTION
# ============================================================

segment_summary = (
    rfm
    .groupby(
        "Segment",
        as_index=False
    )
    .agg(
        Customers=("customer_id","count"),
        Revenue=("Monetary","sum")
    )
    .sort_values(
```

```
        "Revenue",
        ascending=False
    )
)

segment_summary["Revenue %"] = (
    segment_summary["Revenue"]/
    segment_summary["Revenue"].sum()*100
).round(2)

display(segment_summary)

# ============================================================
# VISUALIZATION 1
# CUSTOMER SEGMENTS
# ============================================================

plt.figure(figsize=(12,6))

plt.bar(
    segment_summary["Segment"],
    segment_summary["Customers"]
)

plt.title("Customer Segments")

plt.xlabel("Segment")

plt.ylabel("Number of Customers")

plt.xticks(rotation=20)

plt.tight_layout()

plt.show()

# ============================================================
# VISUALIZATION 2
# REVENUE BY SEGMENT
# ============================================================

plt.figure(figsize=(12,6))

plt.bar(
    segment_summary["Segment"],
    segment_summary["Revenue"]
)

plt.title("Revenue by Customer Segment")

plt.xlabel("Segment")

plt.ylabel("Revenue ($)")

plt.xticks(rotation=20)

plt.tight_layout()

plt.show()
```

```
# ============================================================
# VISUALIZATION 3
# PIE CHART
# ============================================================

plt.figure(figsize=(8,8))

plt.pie(
    segment_summary["Customers"],
    labels=segment_summary["Segment"],
    autopct="%1.1f%%",
    startangle=90
)

plt.title("Customer Segment Distribution")

plt.show()

# ============================================================
# TOP CUSTOMERS
# ============================================================

top_customers = rfm.sort_values(
    "Monetary",
    ascending=False
).head(10)

print("="*70)
print("TOP 10 CUSTOMERS")
print("="*70)

display(top_customers)

# ============================================================
# RFM SUMMARY
# ============================================================

print("""
================================================================
RFM ANALYSIS SUMMARY
================================================================

Key Findings

✓ Customer Recency Calculated

✓ Customer Frequency Calculated

✓ Customer Monetary Value Calculated

✓ Customer Segments Created

✓ Revenue Contribution by Segment

Business Recommendations

1. Reward Champions with exclusive benefits.

2. Retain Loyal Customers using loyalty programs.
```

```
3. Reactivate At-Risk customers through personalized offers.

4. Win back Lost Customers using discount campaigns.

5. Focus marketing on high-value customer segments.

================================================================
END OF RFM ANALYSIS
================================================================
""")
```

```
======================================================================
RFM TABLE
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>customer_id</th> <th>Recency</th> <th>Frequency</th> <th>Monetary</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

1	186	2	1096.70	
2	166	1	4378.05	
3	117	1	3177.20	
4	155	1	670.56	
5	103	2	9667.22	

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>customer_id</th> <th>Recency</th> <th>Frequency</th> <th>Monetary</th> <th>R_Score</th> <th>F_Score</th> <th>M_Score</th> <th>RFM_Score</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

1	186	2	1096.70	2	3	1	231	
2	166	1	4378.05	3	1	3	313	
3	117	1	3177.20	3	1	3	313	
4	155	1	670.56	3	1	1	311	
5	103	2	9667.22	4	3	5	435	

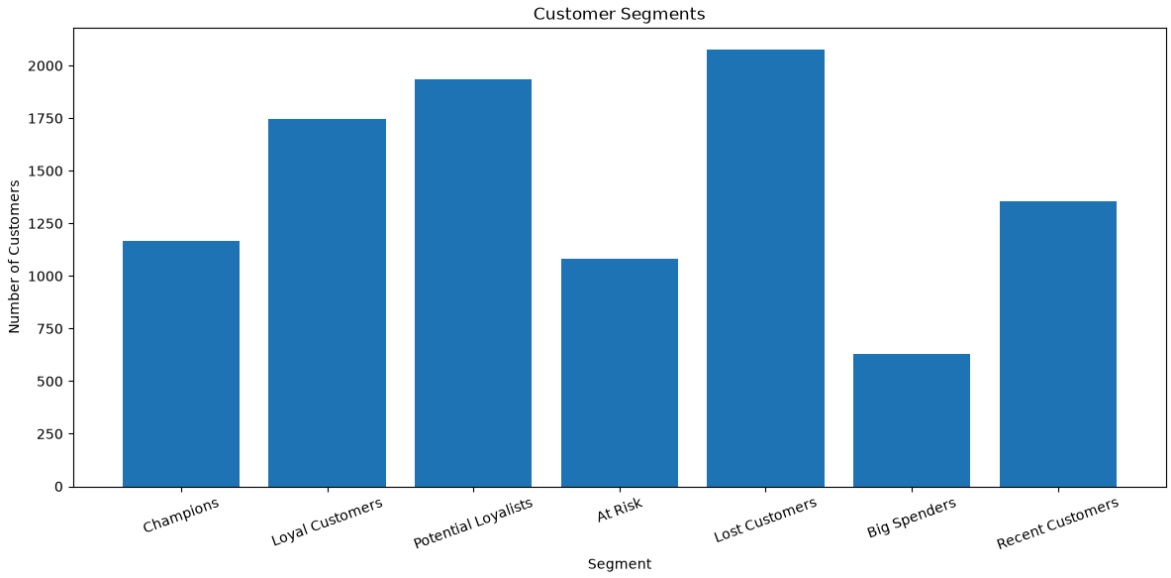
```
======================================================================
CUSTOMER SEGMENTS
======================================================================
```

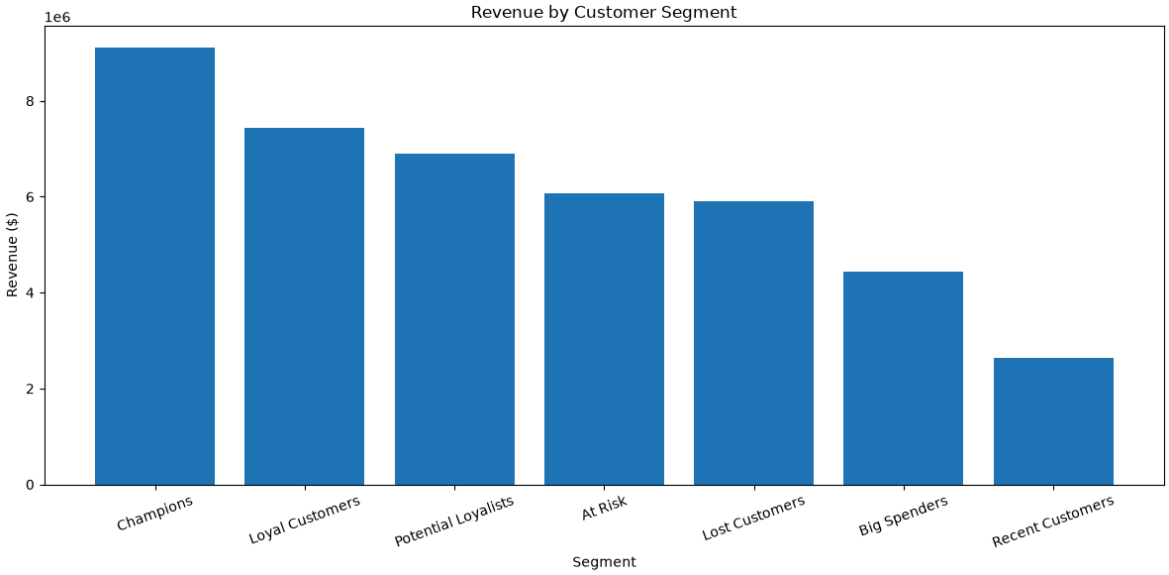
<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>customer_id</th> <th>Recency</th> <th>Frequency</th> <th>Monetary</th> <th>R_Score</th> <th>F_Score</th> <th>M_Score</th> <th>RFM_Score</th> <th>Segment</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

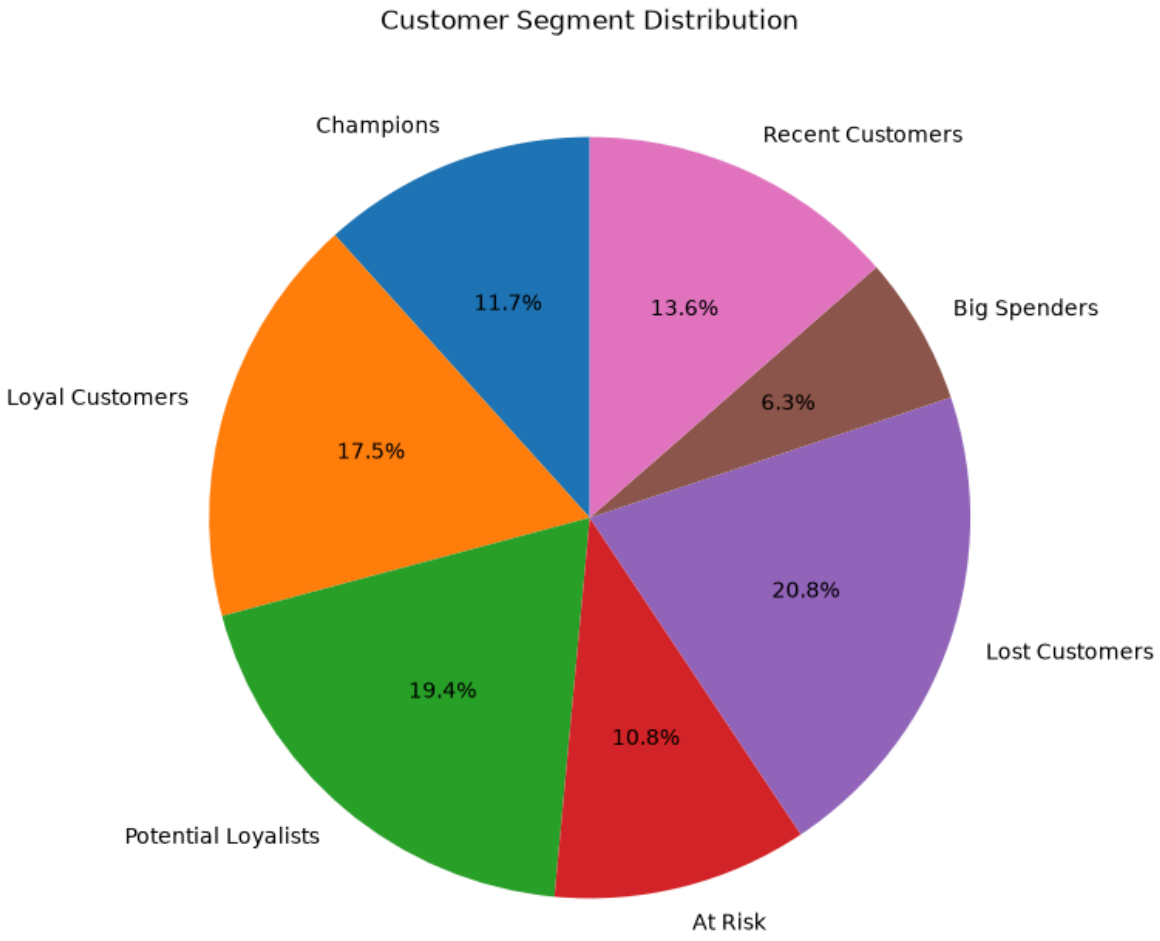
1	186	2	1096.70	2	3	1	231	Potential Loyalists
2	166	1	4378.05	3	1	3	313	Potential Loyalists
3	117	1	3177.20	3	1	3	313	Potential Loyalists
4	155	1	670.56	3	1	1	311	Potential Loyalists
5	103	2	9667.22	4	3	5	435	Big Spenders

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Segment</th> <th>Customers</th> <th>Revenue</th> <th>Revenue %</th> </thead> <tbody> <th>2</th> <th>4</th> <th>5</th> <th>0</th> <th>3</th> <th>1</th> <th>6</th> </tbody> <tbody></tbody>

Champions	1168	9104456.69	21.42
Loyal Customers	1748	7434053.55	17.49
Potential Loyalists	1936	6896913.15	16.23
At Risk	1084	6074610.54	14.29
Lost Customers	2076	5910497.15	13.91
Big Spenders	631	4440837.23	10.45
Recent Customers	1357	2634336.50	6.20








```
======================================================================
TOP 10 CUSTOMERS
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>customer_id</th> <th>Recency</th> <th>Frequency</th> <th>Monetary</th> <th>R_Score</th> <th>F_Score</th> <th>M_Score</th> <th>RFM_Score</th> <th>Segment</th> </thead> <tbody> <th>9802</th> <th>4113</th> <th>8940</th> <th>9952</th> <th>7386</th> <th>3612</th> <th>1415</th> <th>5736</th> <th>1732</th> <th>7921</th> </tbody> <tbody> </tbody>

9803	21	2	19651.70	5	5	5	555	Champions
4114	55	2	17567.00	4	4	5	445	Champions
8941	133	2	17124.03	3	5	5	355	Loyal Customers
9953	136	2	16956.25	3	5	5	355	Loyal Customers
7387	21	2	16952.30	5	5	5	555	Champions
3613	26	2	16813.38	5	4	5	545	Champions
1416	15	2	16522.12	5	3	5	535	Big Spenders
5737	68	2	16442.62	4	4	5	445	Champions
1733	221	2	16292.75	2	3	5	235	Potential Loyalists
7922	88	2	16193.07	4	5	5	455	Champions

```
============================================================
RFM ANALYSIS SUMMARY
============================================================

Key Findings

✓ Customer Recency Calculated

✓ Customer Frequency Calculated

✓ Customer Monetary Value Calculated

✓ Customer Segments Created

✓ Revenue Contribution by Segment

Business Recommendations

1. Reward Champions with exclusive benefits.

2. Retain Loyal Customers using loyalty programs.

3. Reactivate At-Risk customers through personalized offers.

4. Win back Lost Customers using discount campaigns.

5. Focus marketing on high-value customer segments.

============================================================
END OF RFM ANALYSIS
============================================================
```

In [61]:

```python
# ============================================================
# ABC INVENTORY ANALYSIS
# ============================================================
#
# Business Questions
#
# 1. Which products generate the majority of revenue?
# 2. Which products belong to A, B, and C classes?
# 3. What percentage of products contribute 80% of revenue?
# 4. Which products require the highest inventory priority?
# 5. How should inventory be managed?
#
# Business Objective
#
# • Optimize inventory management
# • Prioritize high-value products
# • Reduce inventory carrying costs
# • Improve profitability
#
# ============================================================

import pandas as pd
import matplotlib.pyplot as plt

plt.rcParams["figure.figsize"] = (12,6)

# ============================================================
```

```
# STEP 1 : CREATE PRODUCT REVENUE TABLE
# ============================================================

abc = (
    order_items
    .merge(products,on="product_id",how="left")
)

abc["Revenue"] = (
    abc["quantity"] *
    abc["price_at_purchase"]
)

abc = (
    abc
    .groupby(
        ["product_id","product_name"],
        as_index=False
    )
    .agg(
        Revenue=("Revenue","sum"),
        Quantity=("quantity","sum")
    )
)

print("="*70)
print("PRODUCT REVENUE DATASET")
print("="*70)

display(abc.head())

# ============================================================
# STEP 2 : SORT PRODUCTS BY REVENUE
# ============================================================

abc = abc.sort_values(
    "Revenue",
    ascending=False
).reset_index(drop=True)

# ============================================================
# STEP 3 : CUMULATIVE REVENUE
# ============================================================

abc["Revenue %"] = (
    abc["Revenue"] /
    abc["Revenue"].sum() * 100
)

abc["Cumulative Revenue %"] = (
    abc["Revenue %"].cumsum()
)

display(abc.head())

# ============================================================
# STEP 4 : ABC CLASSIFICATION
# ============================================================

def classify(value):
```

```
    if value <= 80:
        return "A"

    elif value <= 95:
        return "B"

    else:
        return "C"

abc["ABC_Class"] = abc["Cumulative Revenue %"].apply(classify)

display(abc.head())

# ============================================================
# BUSINESS QUESTION 1
# CLASS SUMMARY
# ============================================================

abc_summary = (
    abc
    .groupby(
        "ABC_Class",
        as_index=False
    )
    .agg(
        Products=("product_id","count"),
        Revenue=("Revenue","sum")
    )
)

abc_summary["Revenue %"] = (
    abc_summary["Revenue"] /
    abc_summary["Revenue"].sum() * 100
).round(2)

print("="*70)
print("ABC SUMMARY")
print("="*70)

display(abc_summary)

# ============================================================
# VISUALIZATION 1
# PRODUCTS BY CLASS
# ============================================================

plt.figure(figsize=(8,5))

plt.bar(
    abc_summary["ABC_Class"],
    abc_summary["Products"]
)

plt.title("Number of Products by ABC Class")

plt.xlabel("ABC Class")

plt.ylabel("Products")
```

```
plt.tight_layout()

plt.show()

# ============================================================
# VISUALIZATION 2
# REVENUE BY CLASS
# ============================================================

plt.figure(figsize=(8,5))

plt.bar(
    abc_summary["ABC_Class"],
    abc_summary["Revenue"]
)

plt.title("Revenue by ABC Class")

plt.xlabel("ABC Class")

plt.ylabel("Revenue ($)")

plt.tight_layout()

plt.show()

# ============================================================
# VISUALIZATION 3
# PIE CHART
# ============================================================

plt.figure(figsize=(8,8))

plt.pie(
    abc_summary["Revenue"],
    labels=abc_summary["ABC_Class"],
    autopct="%1.1f%%",
    startangle=90
)

plt.title("Revenue Contribution by ABC Class")

plt.show()

# ============================================================
# BUSINESS QUESTION 2
# TOP A CLASS PRODUCTS
# ============================================================

top_A = abc[
    abc["ABC_Class"]=="A"
].head(15)

print("="*70)
print("TOP A CLASS PRODUCTS")
print("="*70)

display(top_A)

# ============================================================
```

```
# BUSINESS QUESTION 3
# PRODUCT CLASS DISTRIBUTION
# ============================================================

class_distribution = (
    abc["ABC_Class"]
    .value_counts()
    .reset_index()
)

class_distribution.columns = [
    "ABC_Class",
    "Number_of_Products"
]

display(class_distribution)

# ============================================================
# ABC ANALYSIS SUMMARY
# ============================================================

print("""
====================================================================
ABC ANALYSIS SUMMARY
====================================================================

Key Findings

✓ Products classified into A, B and C categories

✓ Revenue contribution calculated

✓ Cumulative revenue analyzed

✓ High-value products identified

✓ Inventory priorities established

Business Recommendations

A-Class Products
• Maintain high inventory levels
• Avoid stock-outs
• Monitor sales daily

B-Class Products
• Review inventory weekly
• Maintain moderate stock

C-Class Products
• Maintain minimum inventory
• Consider promotional offers
• Review product profitability

====================================================================
END OF ABC ANALYSIS
====================================================================
""")
```

```
======================================================================
PRODUCT REVENUE DATASET
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>product_id</th> <th>product_name</th> <th>Revenue</th> <th>Quantity</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

1	Office Chair	280.20	2	
2	Coffee Maker	16112.32	33	
3	Document Scanner	17084.55	52	
4	Desk Mat	35444.44	78	
5	Tablet Stand	17673.31	38	

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>product_id</th> <th>product_name</th> <th>Revenue</th> <th>Quantity</th> <th>Revenue %</th> <th>Cumulative Revenue %</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

697	Water Bottle	63822.93	92	0.150187	0.150187	
1186	Standing Desk	61073.42	117	0.143717	0.293903	
1012	Rice Cooker	59233.99	98	0.139388	0.433292	
1181	Wall Clock	57196.17	107	0.134593	0.567884	
1844	Screen Protector	56872.60	104	0.133831	0.701716	

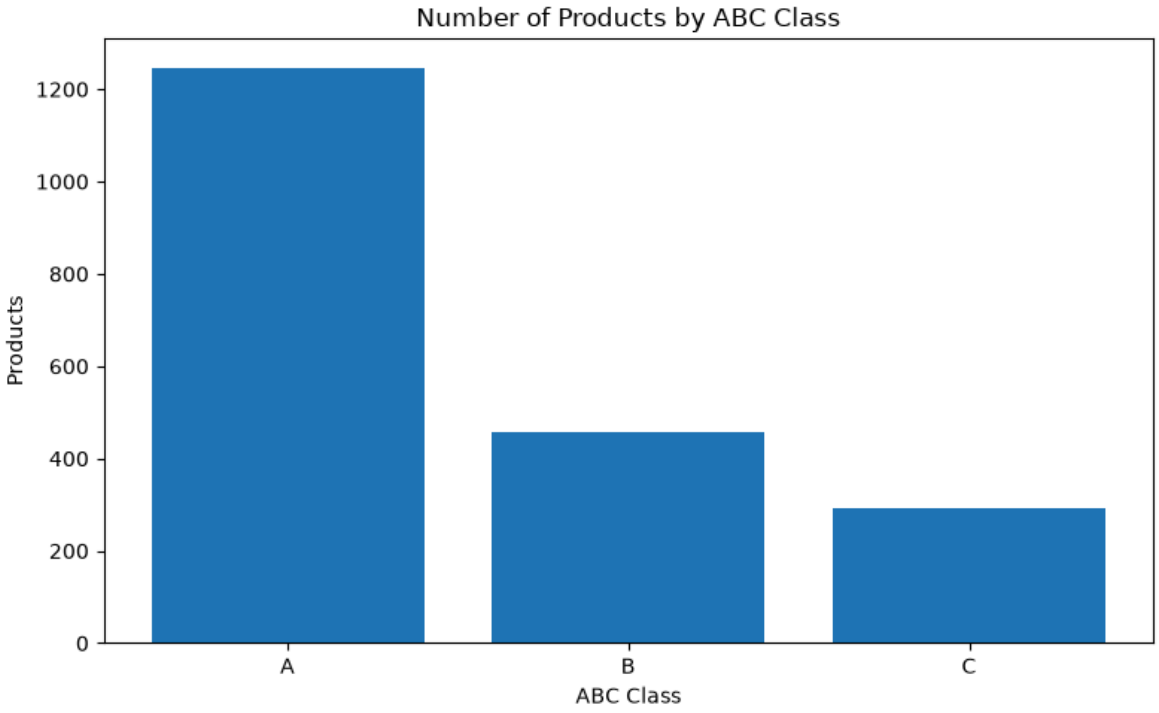
<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>product_id</th> <th>product_name</th> <th>Revenue</th> <th>Quantity</th> <th>Revenue %</th> <th>Cumulative Revenue %</th> <th>ABC_Class</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

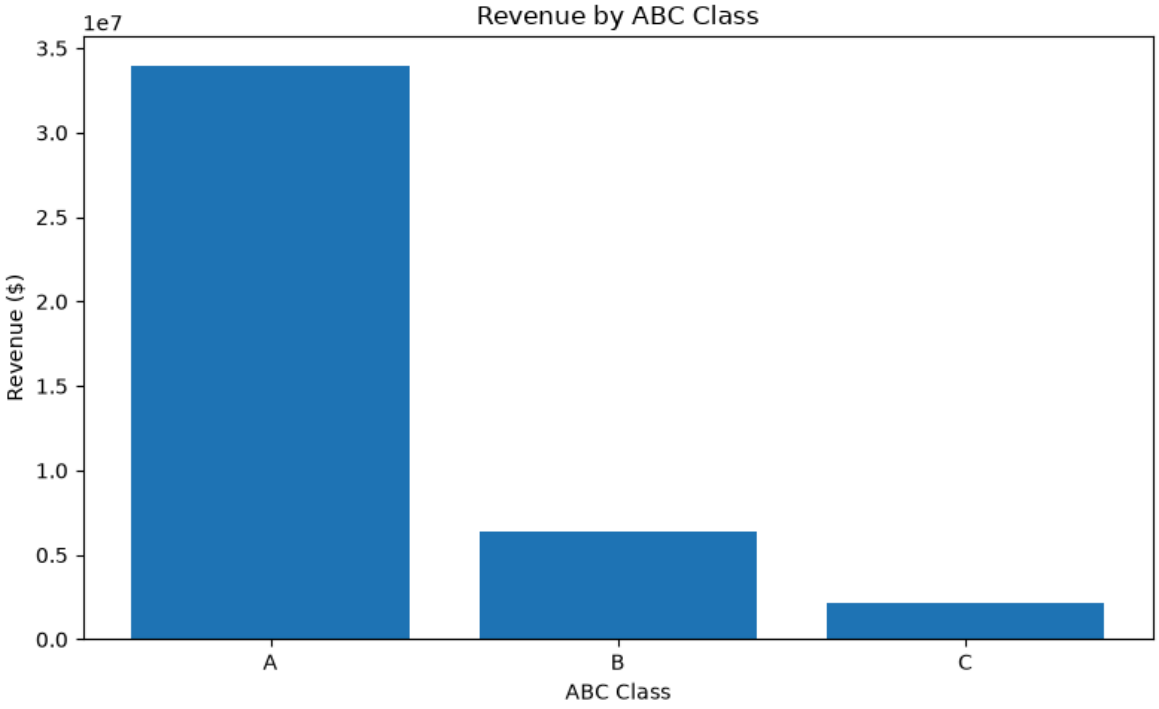
697	Water Bottle	63822.93	92	0.150187	0.150187	A	
1186	Standing Desk	61073.42	117	0.143717	0.293903	A	
1012	Rice Cooker	59233.99	98	0.139388	0.433292	A	
1181	Wall Clock	57196.17	107	0.134593	0.567884	A	
1844	Screen Protector	56872.60	104	0.133831	0.701716	A	

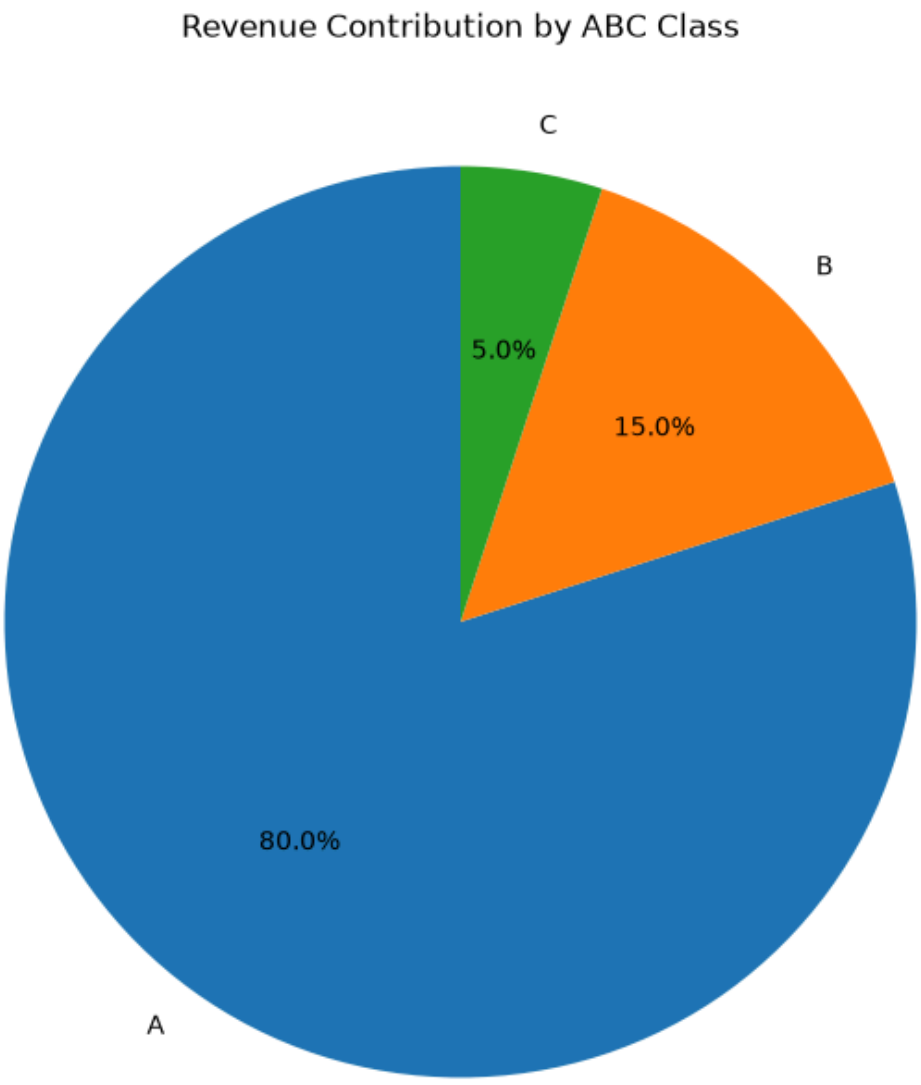
```
======================================================================
ABC SUMMARY
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>ABC_Class</th> <th>Products</th> <th>Revenue</th> <th>Revenue %</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> </tbody> <tbody> </tbody>

A	1247	33985704.89	79.97
B	457	6375835.18	15.00
C	293	2134164.74	5.02








```
======================================================================
TOP A CLASS PRODUCTS
======================================================================
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>product_id</th> <th>product_name</th> <th>Revenue</th> <th>Quantity</th> <th>Revenue %</th> <th>Cumulative Revenue %</th> <th>ABC_Class</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> </tbody> <tbody> </tbody>

697	Water Bottle	63822.93	92	0.150187	0.150187	A
1186	Standing Desk	61073.42	117	0.143717	0.293903	A
1012	Rice Cooker	59233.99	98	0.139388	0.433292	A
1181	Wall Clock	57196.17	107	0.134593	0.567884	A
1844	Screen Protector	56872.60	104	0.133831	0.701716	A
936	Standing Desk	55811.02	82	0.131333	0.833049	A
429	Storage Containers	55027.95	103	0.129491	0.962540	A
670	Wireless Mouse	53935.18	118	0.126919	1.089459	A
986	Standing Desk	53684.79	119	0.126330	1.215789	A
309	Computer Speakers	53674.71	87	0.126306	1.342095	A
1308	External SSD	53164.29	120	0.125105	1.467200	A
1563	Phone Grip	53119.72	102	0.125000	1.592200	A
1707	Bath Towels	52762.82	89	0.124160	1.716361	A
1312	Rice Cooker	52630.35	86	0.123849	1.840209	A
1282	Storage Shelf	52237.86	106	0.122925	1.963134	A

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>ABC_Class</th> <th>Number_of_Products</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> </tbody> <tbody></tbody>

A	1247
B	457
C	293

```
============================================================
ABC ANALYSIS SUMMARY
============================================================

Key Findings

✓ Products classified into A, B and C categories

✓ Revenue contribution calculated

✓ Cumulative revenue analyzed

✓ High-value products identified

✓ Inventory priorities established

Business Recommendations

A-Class Products
• Maintain high inventory levels
• Avoid stock-outs
• Monitor sales daily

B-Class Products
• Review inventory weekly
• Maintain moderate stock

C-Class Products
• Maintain minimum inventory
• Consider promotional offers
• Review product profitability

============================================================
END OF ABC ANALYSIS
============================================================
```

In [62]:
```python
# ============================================================
# PARETO ANALYSIS (80/20 RULE)
# ============================================================
#
# Business Questions
#
# 1. Which products generate 80% of total revenue?
# 2. Which customers generate 80% of total revenue?
# 3. How many products contribute to most of the sales?
# 4. How many customers contribute to most of the revenue?
#
# Business Objective
#
# • Identify high-value products
# • Identify high-value customers
# • Optimize inventory
# • Improve marketing strategy
#
# ============================================================

import pandas as pd
import matplotlib.pyplot as plt
```

```
plt.rcParams["figure.figsize"] = (12,6)

# ============================================================
# PRODUCT PARETO ANALYSIS
# ============================================================

product_pareto = (
    order_items
    .merge(products, on="product_id", how="left")
)

product_pareto["Revenue"] = (
    product_pareto["quantity"] *
    product_pareto["price_at_purchase"]
)

product_pareto = (
    product_pareto
    .groupby(
        ["product_id","product_name"],
        as_index=False
    )
    .agg(
        Revenue=("Revenue","sum")
    )
)

product_pareto = product_pareto.sort_values(
    "Revenue",
    ascending=False
).reset_index(drop=True)

product_pareto["Revenue %"] = (
    product_pareto["Revenue"] /
    product_pareto["Revenue"].sum() * 100
)

product_pareto["Cumulative %"] = (
    product_pareto["Revenue %"].cumsum()
)

top_products = product_pareto[
    product_pareto["Cumulative %"] <= 80
]

print("="*70)
print("PRODUCT PARETO ANALYSIS")
print("="*70)

print(f"Products contributing 80% Revenue : {len(top_products)}")

display(top_products.head(20))

# ============================================================
# PRODUCT PARETO CHART
# ============================================================

plt.figure(figsize=(14,6))

plt.bar(
```

```
    product_pareto.index + 1,
    product_pareto["Revenue"],
    label="Revenue"
)

plt.plot(
    product_pareto.index + 1,
    product_pareto["Cumulative %"],
    color="red",
    linewidth=2,
    label="Cumulative %"
)

plt.axhline(
    y=80,
    color="green",
    linestyle="--",
    label="80% Revenue"
)

plt.title("Pareto Analysis - Products")

plt.xlabel("Products")

plt.ylabel("Revenue / Cumulative %")

plt.legend()

plt.tight_layout()

plt.show()

print("""
Business Insight

A relatively small number of products
generate the majority of total revenue.

These products should receive
high inventory priority.
""")

# ============================================================
# CUSTOMER PARETO ANALYSIS
# ============================================================

customer_pareto = (
    orders
    .groupby(
        "customer_id",
        as_index=False
    )
    .agg(
        Revenue=("total_price","sum")
    )
)

customer_pareto = customer_pareto.sort_values(
    "Revenue",
    ascending=False
```

```
).reset_index(drop=True)

customer_pareto["Revenue %"] = (
    customer_pareto["Revenue"] /
    customer_pareto["Revenue"].sum() * 100
)

customer_pareto["Cumulative %"] = (
    customer_pareto["Revenue %"].cumsum()
)

top_customers = customer_pareto[
    customer_pareto["Cumulative %"] <= 80
]

print("="*70)
print("CUSTOMER PARETO ANALYSIS")
print("="*70)

print(f"Customers contributing 80% Revenue : {len(top_customers)}")

display(top_customers.head(20))

# ============================================================
# CUSTOMER PARETO CHART
# ============================================================

plt.figure(figsize=(14,6))

plt.bar(
    customer_pareto.index + 1,
    customer_pareto["Revenue"],
    label="Revenue"
)

plt.plot(
    customer_pareto.index + 1,
    customer_pareto["Cumulative %"],
    color="red",
    linewidth=2,
    label="Cumulative %"
)

plt.axhline(
    y=80,
    color="green",
    linestyle="--",
    label="80% Revenue"
)

plt.title("Pareto Analysis - Customers")

plt.xlabel("Customers")

plt.ylabel("Revenue / Cumulative %")

plt.legend()

plt.tight_layout()
```

```
plt.show()

print("""
Business Insight

A small percentage of customers
contributes to the majority of revenue.

These customers should be prioritized
through loyalty programs and personalized offers.
""")

# ============================================================
# PARETO ANALYSIS SUMMARY
# ============================================================

print("""
================================================================
PARETO ANALYSIS SUMMARY
================================================================

Key Findings

✓ Products generating 80% of revenue identified

✓ Customers generating 80% of revenue identified

✓ Revenue concentration analyzed

Business Recommendations

1. Prioritize high-value products.

2. Maintain sufficient inventory for key products.

3. Reward high-value customers.

4. Focus marketing on profitable customer segments.

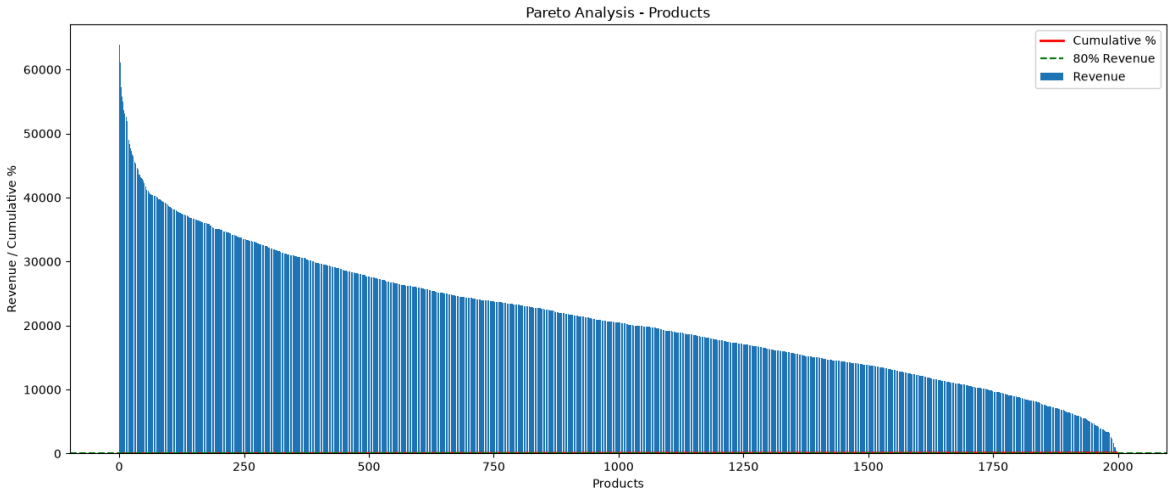
5. Regularly monitor revenue concentration.

================================================================
END OF PARETO ANALYSIS
================================================================
""")
```

```
======================================================================
PRODUCT PARETO ANALYSIS
======================================================================
Products contributing 80% Revenue : 1247
```

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>product_id</th> <th>product_name</th> <th>Revenue</th> <th>Revenue %</th> <th>Cumulative %</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> <th>16</th> <th>17</th> <th>18</th> <th>19</th> </tbody> <tbody></tbody>

697	Water Bottle	63822.93	0.150187	0.150187
1186	Standing Desk	61073.42	0.143717	0.293903
1012	Rice Cooker	59233.99	0.139388	0.433292
1181	Wall Clock	57196.17	0.134593	0.567884
1844	Screen Protector	56872.60	0.133831	0.701716
936	Standing Desk	55811.02	0.131333	0.833049
429	Storage Containers	55027.95	0.129491	0.962540
670	Wireless Mouse	53935.18	0.126919	1.089459
986	Standing Desk	53684.79	0.126330	1.215789
309	Computer Speakers	53674.71	0.126306	1.342095
1308	External SSD	53164.29	0.125105	1.467200
1563	Phone Grip	53119.72	0.125000	1.592200
1707	Bath Towels	52762.82	0.124160	1.716361
1312	Rice Cooker	52630.35	0.123849	1.840209
1282	Storage Shelf	52237.86	0.122925	1.963134
1223	Bookshelf	51951.11	0.122250	2.085385
1586	Standing Desk	51752.21	0.121782	2.207167
1886	Standing Desk	49338.92	0.116103	2.323270
424	Portable Charger	48969.44	0.115234	2.438504
1277	Smart Watch	48757.18	0.114734	2.553238



Business Insight

A relatively small number of products
generate the majority of total revenue.

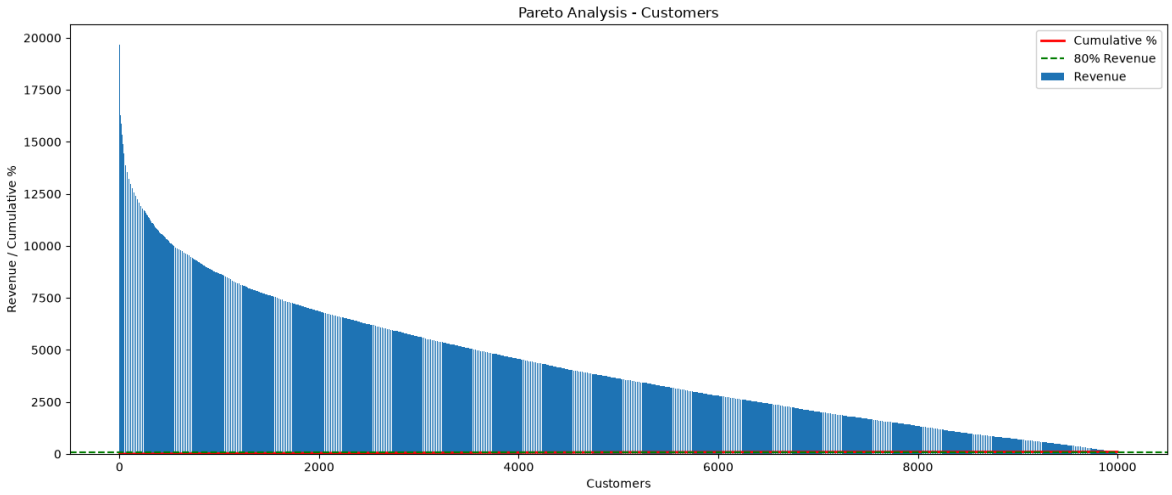
These products should receive
high inventory priority.

==========
CUSTOMER PARETO ANALYSIS
==========
Customers contributing 80% Revenue : 5064

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>customer_id</th> <th>Revenue</th> <th>Revenue %</th> <th>Cumulative %</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> <th>16</th> <th>17</th> <th>18</th> <th>19</th> </tbody> <tbody></tbody>

9803	19651.70	0.046244	0.046244
4114	17567.00	0.041338	0.087582
8941	17124.03	0.040296	0.127878
9953	16956.25	0.039901	0.167779
7387	16952.30	0.039892	0.207671
3613	16813.38	0.039565	0.247236
1416	16522.12	0.038880	0.286115
5737	16442.62	0.038692	0.324808
1733	16292.75	0.038340	0.363148
7922	16193.07	0.038105	0.401253
5445	16179.95	0.038074	0.439327
5053	16165.71	0.038041	0.477368
6992	16162.25	0.038033	0.515401
9841	16119.30	0.037932	0.553332
2770	16024.80	0.037709	0.591041
8371	15949.86	0.037533	0.628574
3182	15885.47	0.037381	0.665956
2388	15861.20	0.037324	0.703280
4540	15802.72	0.037187	0.740467
3662	15702.48	0.036951	0.777417

```
C:\Users\mk393\AppData\Local\Temp\ipykernel_5360\1891496858.py:202: UserWarning:
Creating legend with loc="best" can be slow with large amounts of data.
  plt.tight_layout()
C:\Users\mk393\AppData\Local\Programs\Python\Python313\Lib\site-packages\IPython
\core\pylabtools.py:170: UserWarning: Creating legend with loc="best" can be slow
with large amounts of data.
  fig.canvas.print_figure(bytes_io, **kw)
```



Business Insight

A small percentage of customers
contributes to the majority of revenue.

These customers should be prioritized
through loyalty programs and personalized offers.

```
================================================================
PARETO ANALYSIS SUMMARY
================================================================

Key Findings

✓ Products generating 80% of revenue identified

✓ Customers generating 80% of revenue identified

✓ Revenue concentration analyzed

Business Recommendations

1. Prioritize high-value products.

2. Maintain sufficient inventory for key products.

3. Reward high-value customers.

4. Focus marketing on profitable customer segments.

5. Regularly monitor revenue concentration.

================================================================
END OF PARETO ANALYSIS
================================================================
```

In [ ]: